Waste Lens

Food Industry Waste Analysis System

Project Documentation

Table of Contents

1. System Overview

WasteLens is a multi-user, browser-based Food Industry Waste Analysis System built with Python Streamlit and a MySQL relational database. It enables restaurant and food-service organisations to systematically capture, manage, and analyse food waste across multiple branches. The platform produces real-time KPI dashboards, trend charts, and exportable reports that support data-driven decisions to reduce operational waste and associated costs.

The system is divided into seven functional modules, each delivered as a dedicated Streamlit page:

- app.py – Login, self-registration, and OTP-based password reset
- 1_Dashboard.py – Real-time KPI cards, 14-day trend, category donut, branch comparison
- 2_Waste_Log.py – Inline waste log entry, viewing, editing, and deletion
- 3_Order_Entry.py – Daily customer order count recording and waste-per-order ratio
- 4_Reports.py – Multi-tab analytics with 10+ chart types and CSV export
- 5_User_Management.py – Admin-only user account and branch access management
- 6_Master_Data.py – Inline CRUD for food items, categories, reasons, and branches
- 7_Settings.py – Admin-only Gmail SMTP configuration for OTP emails

Additional supporting files:

- auth.py – Password hashing, login/register/session helpers, role guards
- db.py – MySQL connection, run_query(), run_update(), init_passwords()
- utils.py – Global CSS injection, sidebar rendering, badge helpers
- email_utils.py – Gmail SMTP OTP email sender with local_settings.json override
- config.py – Database credentials, default passwords, email config

Technology Stack

The following technologies power the WasteLens application:

Frontend / UI Layer

- Streamlit (v1.32+) – Python-native web framework that converts Python scripts into interactive browser applications. All UI components (forms, charts, tables, buttons) are rendered via Streamlit widgets.
- Custom CSS – Injected via st.markdown() to apply the WasteLens green brand theme globally across all pages. Defined in utils.py as GLOBAL_CSS.
- Plotly / Plotly Express (v5.x) – Used for all interactive charts: area charts, pie/donut charts, bar charts, scatter plots, treemaps, funnel charts, radar charts.

Backend / Logic Layer

- Python 3.11+ – Core application language.

2.

- bcrypt (v4.x) – Password hashing with random salt. All passwords stored as bcrypt digests; plaintext is never persisted.
- Pandas (v2.x) – DataFrame operations, CSV export, data transformations.
- smtplib (stdlib) – Gmail SMTP connection on port 587 with STARTTLS for OTP email delivery.

Data Layer

- MySQL 8.0 – Relational database running locally on port 3306. Database name: wastelens.
- mysql-connector-python (v8.x) – Official MySQL connector with dictionary cursor support.
- local_settings.json – Per-machine file storing Gmail credentials, takes priority over config.py so teammates can configure their own email without editing shared code.

Authentication & Login Module (app.py)

3.1 Design View

app.py is the entry point of the application. It presents a two-panel full-width layout: a left green branding panel (40% width) containing the WasteLens logo, tagline, and feature highlights; and a right form area (60% width) that hosts one of four views controlled by st.session_state["auth_view"].

The four views and their triggers are:

- "login" – Default view; shown on first visit or after logout. Contains email + password form.
- "register" – Navigated to via the "Create an account" link. Self-registration as Staff role.
- "forgot_email" – Navigated to via "Reset it here". Step 1 of the password-reset flow.
- "forgot_otp" – Automatically shown after OTP is sent. Step 2: enter code + new password.

There are no browser tabs. Navigation between views is handled by st.button() styled as links. Each link sets st.session_state["auth_view"] and calls st.rerun(). The Streamlit sidebar, header, and footer are completely hidden via CSS on this page to present a clean, full-screen app-like appearance.

3.2 Execution View

The startup sequence on each page load is:

1. st.set_page_config() is called with layout="wide" and initial_sidebar_state="collapsed".
2. Global CSS is injected to hide the sidebar nav, header, and footer.

3.

3. If st.session_state["user"] already exists, st.switch_page() redirects to Dashboard immediately.
4. init_passwords() is called silently in a try/except block to seed bcrypt hashes on first run.
5. The two-column layout is rendered. The active view is read from session state.
6. For the "forgot_email" view: a 6-digit random OTP is generated via random.randint(100000, 999999). send_otp_email() from email_utils.py is called. The OTP code and a Unix timestamp expiry (time.time() + 600) are stored in session state.
7. For the "forgot_otp" view: on form submit, time.time() is compared to fp_otp_expiry. Expired OTPs clear session state and redirect back to Step 1.
8. On successful password reset, hash_password() re-hashes the new password and an UPDATE statement writes it to the User table.

3.3 Field Validation Rules

Login Form:

- Email and password must both be non-empty. Error: "Please enter both email and password."

- If credentials are wrong or account is inactive/unapproved, auth.login() returns a specific error string displayed via st.error().

Registration Form:

- First Name and Last Name must both be non-empty.
- Email must be non-empty and must not already exist in the User table.
- Password must have: 8+ characters, one uppercase, one lowercase, one digit, one special character. Real-time strength indicator shows 5 coloured checkmarks.
- Confirm Password must exactly match the Password field.
- Branch must be selected from the active-branch dropdown.

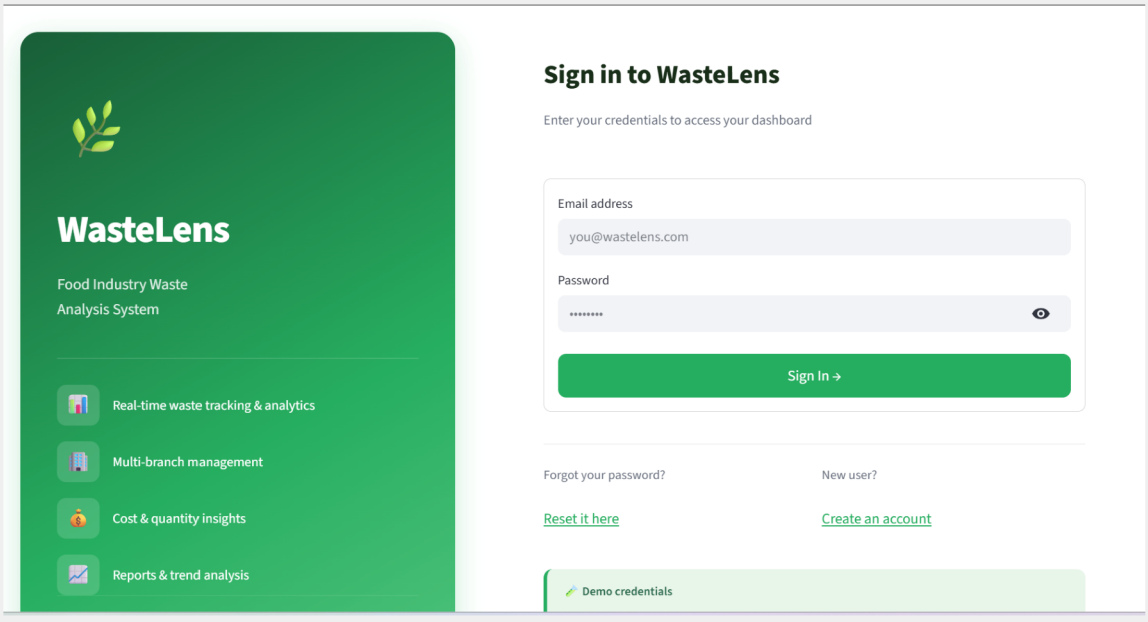
Forgot Password – OTP Flow:

- Email must belong to an active (IsActive=1) account in the database.
- OTP must match the 6-digit code stored in session state.
- OTP must be entered within 10 minutes (600 seconds) of generation.
- New password must pass the same 5-rule strength check as registration.
- Confirm Password must match New Password.

3.4 Screenshots – Login Module

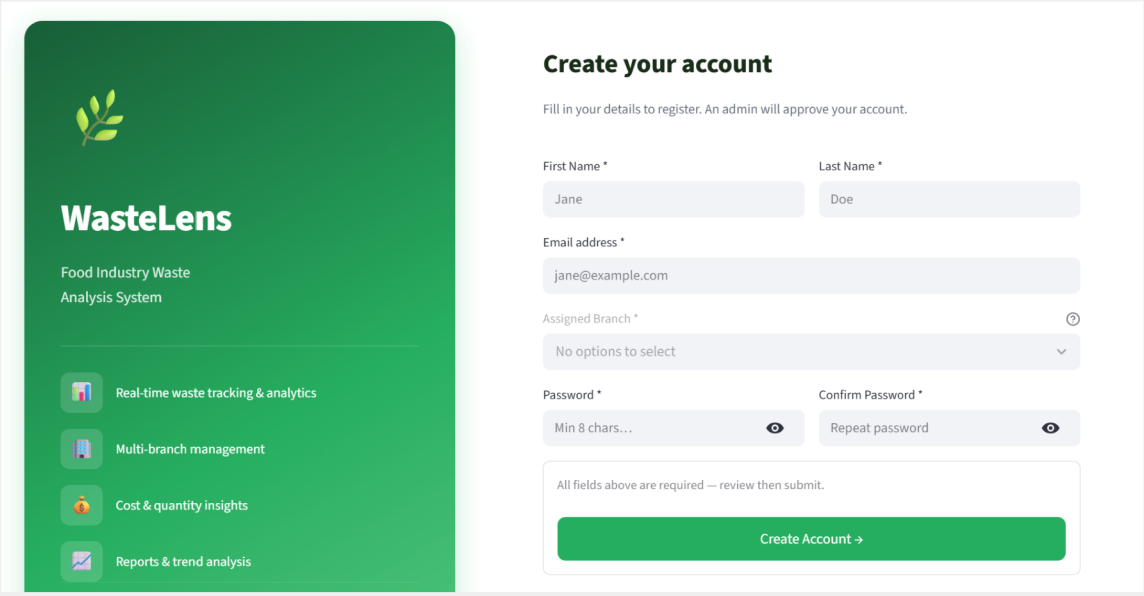
Login Page – Sign In View

[SCREENSHOT — Login page – Sign In form with email, password, navigation links, and demo credentials box]



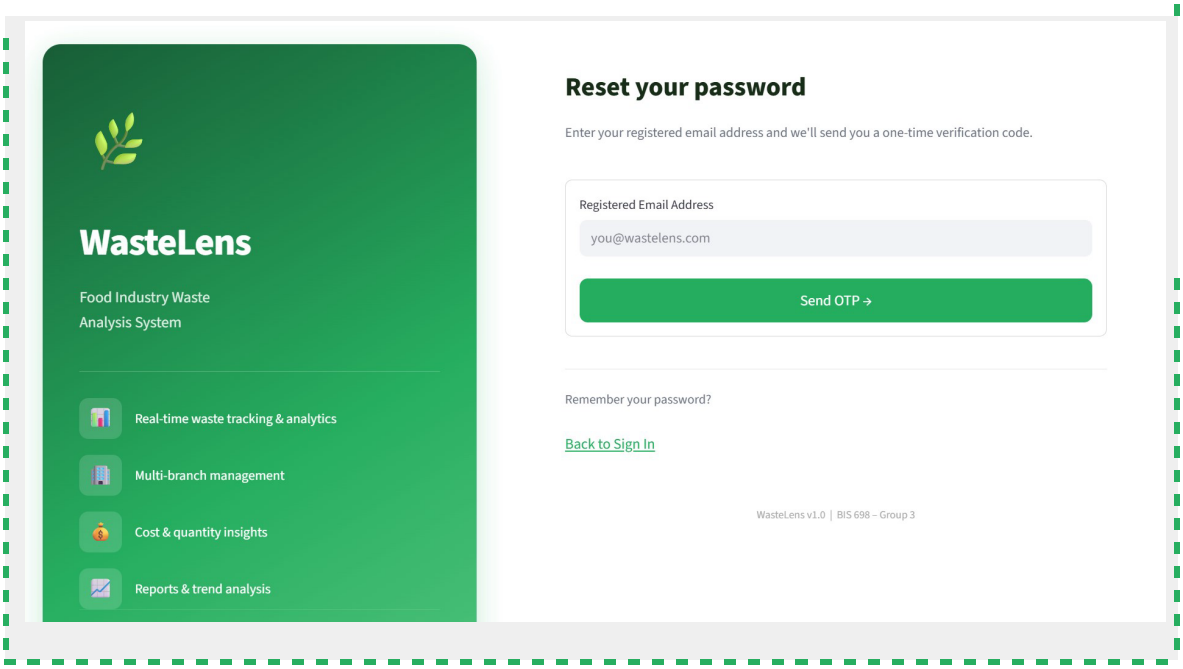
Login Page – Register / Create Account View

[SCREENSHOT — Registration form – First Name, Last Name, Email, Branch, Password with strength indicator]



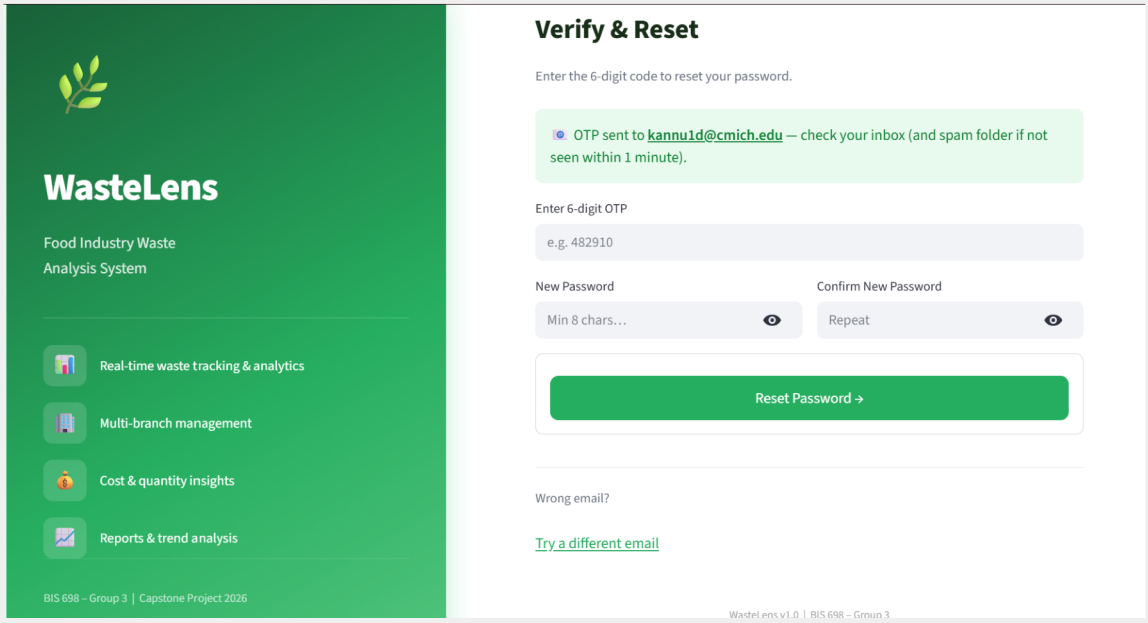
Login Page – Forgot Password Step 1 (Enter Email)

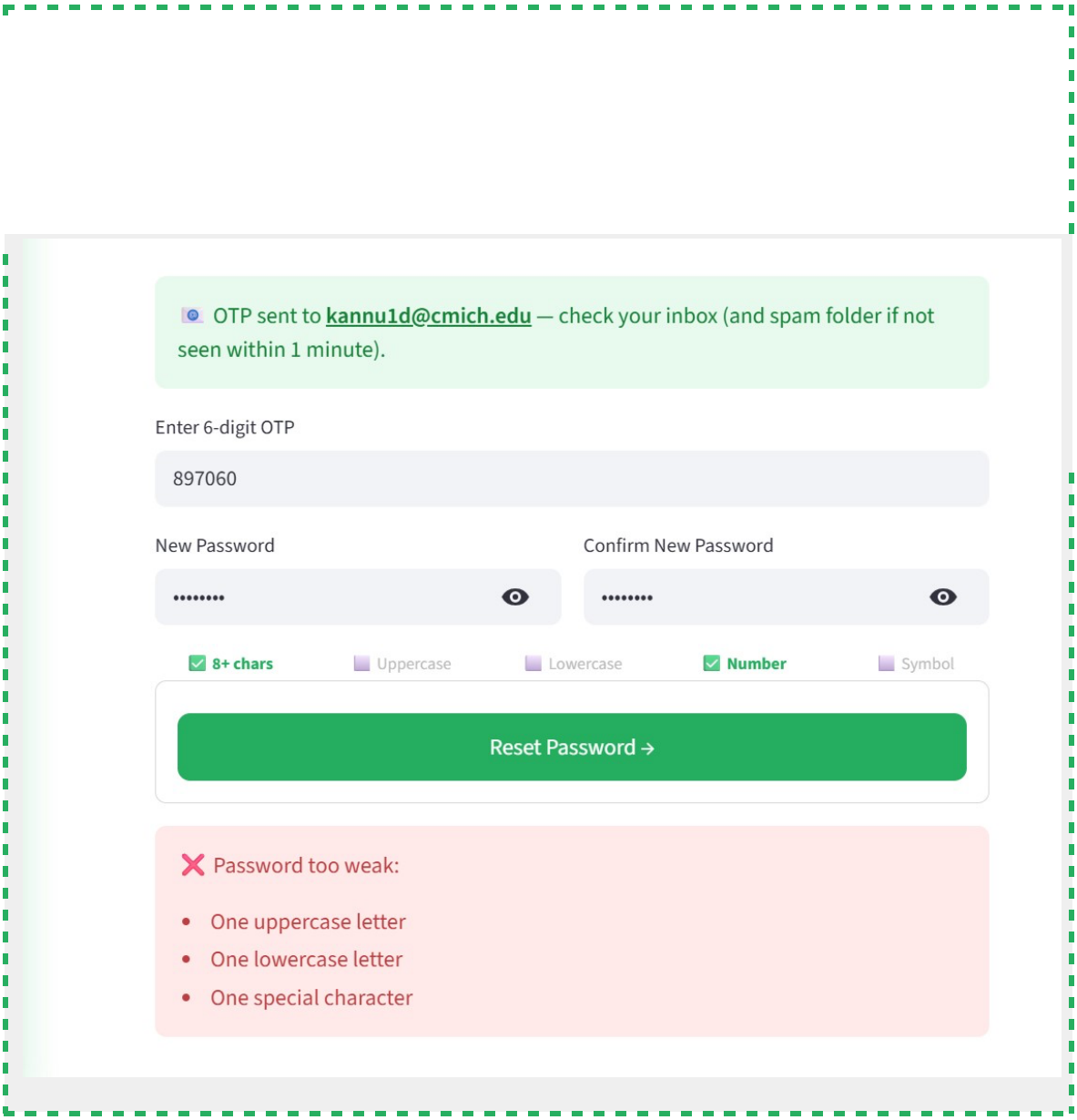
[SCREENSHOT — Forgot Password – Step 1: enter registered email address to receive OTP]


WasteLens
Food Industry Waste
Analysis System
Real-time waste tracking & analytics
Multi-branch management
Cost & quantity insights
Reports & trend analysis
Reset your password
Enter your registered email address and we'll send you a one-time verification code.
Registered Email Address
you@wastelens.com
Send OTP →
Remember your password?
Back to Sign In
WasteLens v1.0 | BIS 698 – Group 3

Login Page – Forgot Password Step 2 (Enter OTP + Reset)

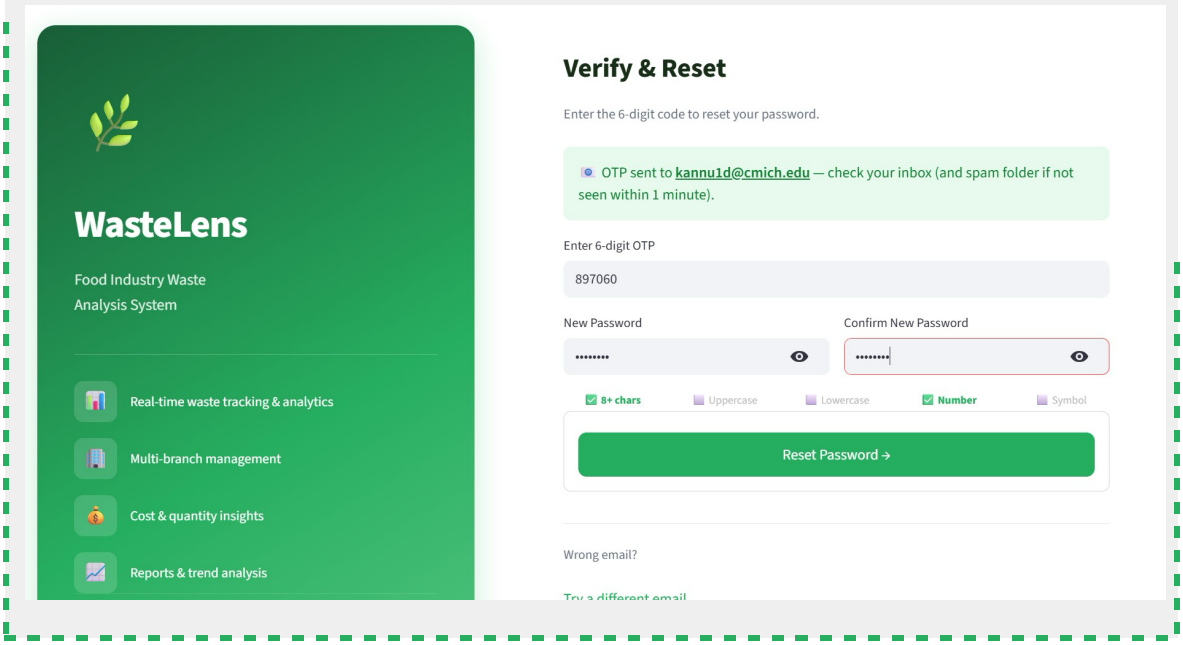
[SCREENSHOT — Forgot Password – Step 2: OTP delivery banner, 6-digit code input, new password fields]





Login Page – Demo Mode OTP Display

[SCREENSHOT — Demo Mode banner showing the OTP code inline when email is not configured]



3.5 Source Code – app.py

```
# ================================================================
#  WasteLens - Entry Point  (Login / Register / Forgot PW)
# ================================================================
import streamlit as st
import random

st.set_page_config(
    page_title="WasteLens",
    page_icon="🌿",
    layout="wide",
    initial_sidebar_state="collapsed",
)

# ── Global CSS ──────────────────────────────────────────────
st.markdown("""
<style>
  #MainMenu, footer { display: none !important; }
  header[data-testid="stHeader"] { display: none !important; }
  [data-testid="stSidebarNav"],    [data-testid="stSidebar"]    {    display:    none
!important;
}
  .block-container { padding: 0 !important; max-width: 100% !important; }
  section[data-testid="stAppViewContainer"] { background: #f0faf5 !important; }

  /* Primary submit buttons */
  [data-testid="stFormSubmitButton"] button {
    background: #27AE60 !important;
    color: white !important;
    border-radius: 9px !important;
    font-weight: 700 !important;
    font-size: 1rem !important;
    padding: 0.7rem !important;
    width: 100% !important;
```

```
    border: none !important;
    transition: all .2s !important;
    margin-top: 0.5rem !important;
  }
  [data-testid="stFormSubmitButton"] button:hover {
    background: #1e8449 !important;
    transform: translateY(-1px) !important;
    box-shadow: 0 4px 14px rgba(39,174,96,.35) !important;
  }

  /* Inputs */
  [data-testid="stTextInput"] input {
    border-radius: 8px !important;
    padding: 0.55rem 0.75rem !important;
    font-size: 0.95rem !important;
  }

  /* Strength row */
  .strength-item { font-size: 0.74rem; text-align: center; padding: 2px; }

  /* Nav link buttons */
  [data-testid="stButton"] button {
    background: transparent !important;
    border: none !important;
    color: #27AE60 !important;
    font-weight: 600 !important;
    padding: 0 !important;
    font-size: 0.88rem !important;
    cursor: pointer !important;
    text-decoration: underline !important;
    text-underline-offset: 2px !important;
  }
  [data-testid="stButton"] button:hover {
    color: #1e8449 !important;
    background: transparent !important;    box-
shadow: none !important;
    transform: none !important;
  }
</style>
""", unsafe_allow_html=True)

# ── Redirect if already logged in ──────────────────────────
if "user" in st.session_state:
    st.switch_page("pages/1_Dashboard.py")

# ── View state: "login" | "register" | "forgot_email" | "forgot_otp"
if "auth_view" not in st.session_state:
```

```
    st.session_state["auth_view"] = "login"

# ── Password helpers ──────────────────────────────────────
def check_password_strength(pwd: str):
    issues = []
    if len(pwd) < 8:
issues.append("At
least 8 characters")
    if not any(c.isupper() for c in pwd):
issues.append("One
uppercase letter")
    if not any(c.islower() for c in pwd):
issues.append("One
lowercase letter")
```

```
    if not any(c.isdigit() for c in pwd):
issues.append("One
number")
    if not any(c in "!@#$%^&*()_+-=[]{}|;':\",./<>?" for c in pwd):
issues.append("One special character")
    return len(issues) == 0, issues

def show_password_rules(pwd: str):
    checks = [
        (len(pwd) >= 8,                                          "8+ chars"),
        (any(c.isupper() for c in pwd),                         "Uppercase"),
        (any(c.islower() for c in pwd),                         "Lowercase"),
        (any(c.isdigit() for c in pwd),                         "Number"),
        (any(c in "!@#$%^&*()_+-=[]{}|;':\",./<>?" for c in pwd),  "Symbol"),
    ]
    cols = st.columns(5)
    for col, (ok, label) in zip(cols, checks):
        col.markdown(
            f"<div  class='strength-item'  style='color:{'#27AE60'  if  ok  else
'#bbb'};"
            f"font-weight:{'700' if ok else '400'}'>{'✅' if ok else '⬜'}
{label}</div>",
            unsafe_allow_html=True,
        )

# ── Seed DB passwords silently ──────────────────────────
try:
    from db import init_passwords
    init_passwords()
except Exception:
    pass

# ═══════════════════════════════════════════════════════
#  LAYOUT  – Left branding panel  |  Right form
# ═══════════════════════════════════════════════════════
left, right = st.columns([4, 6], gap="small")

# ─── LEFT BRANDING PANEL ────────────────────────────────
with left:
    st.markdown("""
    <div style="
        background: linear-gradient(150deg,#1a5e38 0%,#27AE60 60%,#52c47d 100%);
        border-radius: 20px; padding: 3.5rem 2.5rem; color: white;
        min-height: 92vh; display: flex; flex-direction: column;
        justify-content: center; margin: 2vh 0 2vh 2vw;
        box-shadow: 0 8px 32px rgba(39,174,96,.25);
    ">
      <div style="font-size:4rem;margin-bottom:.8rem">🌿</div>
      <h1 style="font-size:2.6rem;font-weight:800;margin:0 0 .4rem;color:white;
                 letter-spacing:-1px">WasteLens</h1>
      <p style="font-size:1.05rem;opacity:.85;margin-bottom:2.5rem;line-
height:1.6">
        Food Industry Waste<br>Analysis System
      </p>
      <div style="border-top:1px solid rgba(255,255,255,.25);padding-top:1.8rem;
                  display:flex;flex-direction:column;gap:1.1rem">
```

```
        <div style="display:flex;align-items:center;gap:.9rem">
          <span style="font-size:1.3rem;background:rgba(255,255,255,.15);
                border-radius:10px;padding:.35rem .55rem">📊</span>
          <span style="font-size:.92rem">Real-time waste tracking &amp;
analytics</span>

        </div>
        <div style="display:flex;align-items:center;gap:.9rem">
          <span style="font-size:1.3rem;background:rgba(255,255,255,.15);
                border-radius:10px;padding:.35rem .55rem">🏢</span>
          <span style="font-size:.92rem">Multi-branch management</span>
        </div>
        <div style="display:flex;align-items:center;gap:.9rem">
          <span style="font-size:1.3rem;background:rgba(255,255,255,.15);
                border-radius:10px;padding:.35rem .55rem">💰</span>
          <span style="font-size:.92rem">Cost &amp; quantity insights</span>
        </div>
        <div style="display:flex;align-items:center;gap:.9rem">
          <span style="font-size:1.3rem;background:rgba(255,255,255,.15);
                border-radius:10px;padding:.35rem .55rem">📈</span>
          <span style="font-size:.92rem">Reports &amp; trend analysis</span>
        </div>
      </div>
      <div style="margin-top:auto;padding-top:2rem;
                  border-top:1px solid rgba(255,255,255,.2);
                  font-size:.78rem;opacity:.65">
        BIS 698 – Group 3  |  Capstone Project 2026
      </div>
    </div>
    """, unsafe_allow_html=True)

# ─── RIGHT FORM AREA ──────────────────────────────────────
with right:
    _, form_col, _ = st.columns([1, 8, 1])
    with form_col:
        view = st.session_state["auth_view"]

        # ═══════════════════════════════════════════════════
        #  VIEW: LOGIN
        # ═══════════════════════════════════════════════════
        if view == "login":
st.markdown("""
            <div style="padding:4vh 0 1.5rem">
              <h2
style="font-size:1.8rem;font-weight:800;color:#1a2e1a;margin-bottom:.3rem">
                Sign in to WasteLens
              </h2>
              <p style="color:#6b7280;font-size:.92rem;margin-bottom:1.8rem">
                Enter your credentials to access your dashboard
              </p>
            </div>
            """, unsafe_allow_html=True)

            with st.form("login_form"):
                email    = st.text_input("Email address",
placeholder="you@wastelens.com")
```

```
                password = st.text_input("Password", type="password",
placeholder="••••••••")
                submitted = st.form_submit_button("Sign In →",
use_container_width=True)

            if submitted:
                if not email or not password:
                    st.error("Please enter both email and password.")

                else:
                    try:
                        from auth import login
                        user, err = login(email.strip().lower(), password)
                        if err:
                            st.error(f"❌ {err}")
                        else:
                            st.session_state["user"] = user
                            st.success("✅ Login successful! Redirecting…")
                            st.switch_page("pages/1_Dashboard.py")
                    except ConnectionError as ce:
                        st.error(f"⚠️ Cannot connect to database. Check
`config.py`.\n\n`{ce}`")

            # ── Nav links ─────────────────────────────────────
            st.markdown(
                "<hr style='border:none;border-top:1px solid #e5e7eb;margin:1.2rem
0'>",
                unsafe_allow_html=True,
            )
            lnk1, lnk2 = st.columns(2)
with lnk1:
                st.markdown(
                    "<span style='color:#6b7280;font-size:.88rem'>Forgot your
password? </span>",
                    unsafe_allow_html=True,
                )
                if st.button("Reset it here", key="goto_forgot"):
                    st.session_state["auth_view"] = "forgot_email"
                    st.rerun()
            with lnk2:
                st.markdown(
                    "<span     style='color:#6b7280;font-size:.88rem'>New     user?
</span>",
                    unsafe_allow_html=True,
                )                 if st.button("Create an account",
key="goto_register"):
                    st.session_state["auth_view"] = "register"
                    st.rerun()

            # ── Demo credentials ──────────────────────────────
            st.markdown("""
            <div style="background:#e8f5e9;border-radius:10px;padding:.9rem
1.1rem;
                        font-size:.82rem;color:#2d6a4f;border-left:4px        solid
#27AE60;
                        margin-top:1.2rem">
              <b>🧪 Demo credentials</b><br><br>
```

```
              🟢 <b>Admin</b>     admin@wastelens.com  / 
<code>Admin@123</code><br>
              🔵 <b>Manager</b>   manager@wastelens.com  / 
<code>Manager@123</code><br>
              ⚪ <b>Staff</b>      staff@wastelens.com
 /  <code>Staff@123</code>
            </div>
            """, unsafe_allow_html=True)

        # ═══════════════════════════════════════════════════
        #  VIEW: REGISTER
        # ═══════════════════════════════════════════════════
        elif view == "register":
```

```
            st.markdown("""
            <div style="padding:4vh 0 1rem">
              <h2
style="font-size:1.8rem;font-weight:800;color:#1a2e1a;margin-bottom:.3rem">
                Create your account
              </h2>
              <p style="color:#6b7280;font-size:.92rem;margin-bottom:1.5rem">
                Fill in your details to register. An admin will approve your account.
              </p>
            </div>
            """, unsafe_allow_html=True)

            try:
                from db import run_query
                branches = run_query(
                    "SELECT BranchID, BranchName FROM Branch WHERE IsActive=1
ORDER
BY BranchName"
                )
                branch_map = {b["BranchName"]: b["BranchID"] for b in branches} if
branches else {}
            except Exception:
branch_map = {}

            rn1, rn2 = st.columns(2)
            with rn1:
                r_first = st.text_input("First Name *", placeholder="Jane",
key="reg_first")
            with rn2:
                r_last  = st.text_input("Last Name *",  placeholder="Doe",
key="reg_last")

            r_email      = st.text_input("Email address *",
placeholder="jane@example.com", key="r_email")
            r_branch_sel = st.selectbox(
                "Assigned Branch *", options=list(branch_map.keys()),
help="You will be assigned to this branch as Staff.",
key="reg_branch",
            )
            rp1, rp2 = st.columns(2)
            with rp1:
                r_pass    = st.text_input("Password *", type="password",
```

```
                                        placeholder="Min 8 chars…",
key="r_pass")
            with rp2:
                r_confirm = st.text_input("Confirm Password *", type="password",
                                        placeholder="Repeat password",
key="r_confirm")
            if r_pass:
                show_password_rules(r_pass)

            with st.form("register_form"):
                st.caption("All fields above are required — review then submit.")
                reg_sub = st.form_submit_button("Create Account →",
use_container_width=True)

            if reg_sub:
                r_name = f"{r_first.strip()} {r_last.strip()}".strip()
                pwd_ok, pwd_issues = check_password_strength(r_pass)
```

```
                if not all([r_first, r_last, r_email, r_branch_sel, r_pass,
r_confirm]):
                    st.error("Please fill in all fields.")
                elif r_pass != r_confirm:
                    st.error("❌ Passwords do not match.")
                elif not pwd_ok:
                    st.error("❌ Password too weak:\n- " + "\n- ".join(pwd_issues))
else:
                    try:
                        from auth import register
                        ok, msg = register(
                            r_name, r_email.strip().lower(), r_pass,
                            branch_map.get(r_branch_sel, 1),
                        )
                        if ok:
                            st.success(f"✅ {msg}")
                            st.session_state["auth_view"] = "login"
                            st.rerun()
                        else:
                            st.error(f"❌ {msg}")
                    except ConnectionError as ce:
                        st.error(f"⚠️ Database error: {ce}")

            st.markdown(
                "<hr style='border:none;border-top:1px solid
#e5e7eb;margin:1.2rem
0'>",
                unsafe_allow_html=True,
            )
            st.markdown(
                "<span style='color:#6b7280;font-size:.88rem'>Already have an
account? </span>",
                unsafe_allow_html=True,
            )
            if st.button("Sign in here", key="goto_login_from_reg"):
                st.session_state["auth_view"] = "login"
st.rerun()

        # ═══════════════════════════════════════════════════
```

```
        #  VIEW: FORGOT PASSWORD – STEP 1 (enter email)
        # ═══════════════════════════════════════════════════
        elif view == "forgot_email":
            st.markdown("""
            <div style="padding:4vh 0 1rem">
              <h2
style="font-size:1.8rem;font-weight:800;color:#1a2e1a;margin-bottom:.3rem">
                Reset your password
              </h2>
              <p style="color:#6b7280;font-size:.92rem;margin-bottom:1.5rem">
                Enter your registered email address and we'll send you a
                one-time verification code.
              </p>
            </div>
            """, unsafe_allow_html=True)

            with st.form("fp_email_form"):
                fp_email_input = st.text_input(
                    "Registered Email Address", placeholder="you@wastelens.com",
                )
```

```
                send_btn = st.form_submit_button("Send OTP →",
use_container_width=True)

            if send_btn:
                if not fp_email_input.strip():
                    st.error("Please enter your email address.")
                else:
try:
                        from db import run_query as _rq
                        row = _rq(
                            "SELECT UserID, UserName FROM User WHERE Email=%s AND
IsActive=1 LIMIT 1",
                            (fp_email_input.strip().lower(),),
                        )
                        if not row:
                            st.error("❌ No active account found with that email
address.")
                        else:
                            otp_code  = str(random.randint(100000, 999999))
                            user_name = row[0]["UserName"]

                            # ── Try to send real email ──────────
                            from email_utils import send_otp_email
                            sent, msg = send_otp_email(
                                fp_email_input.strip().lower(), otp_code, user_name
                            )

                            import time
                            st.session_state["fp_otp"]        = otp_code
                            st.session_state["fp_otp_expiry"] = time.time() + 600  #
10 min
                            st.session_state["fp_email"]      =
fp_email_input.strip().lower()
                            st.session_state["fp_email_sent"] = sent
```

```
                            st.session_state["auth_view"]    = "forgot_otp"

                            if sent:
                                st.session_state["fp_send_msg"] = "real"
                            elif msg == "EMAIL_NOT_CONFIGURED":
                                st.session_state["fp_send_msg"] = "demo"
                            else:
                                st.session_state["fp_send_msg"] = f"error:{msg}"

                            st.rerun()
                    except Exception as e:
                        st.error(f"⚠️ Error: {e}")

            st.markdown(
                "<hr style='border:none;border-top:1px solid #e5e7eb;margin:1.2rem
0'>",
                unsafe_allow_html=True,
            )
            st.markdown(
                "<span     style='color:#6b7280;font-size:.88rem'>Remember     your
password?
</span>",
                unsafe_allow_html=True,
            )
            if st.button("Back to Sign In", key="back_to_login_fp"):
```

```
                st.session_state["auth_view"] = "login"
                st.rerun()

        # ═══════════════════════════════════════════════════
        #  VIEW: FORGOT PASSWORD – STEP 2 (verify OTP + reset)
        # ═══════════════════════════════════════════════════
        elif view == "forgot_otp":
            fp_email_display = st.session_state.get("fp_email", "")
            otp_correct      = st.session_state.get("fp_otp", "")

            send_msg = st.session_state.get("fp_send_msg", "demo")

            st.markdown(f"""
            <div style="padding:4vh 0 1rem">
              <h2
style="font-size:1.8rem;font-weight:800;color:#1a2e1a;margin-bottom:.3rem">
                Verify &amp;amp; Reset
              </h2>
              <p style="color:#6b7280;font-size:.92rem;margin-bottom:.5rem">
                Enter the 6-digit code to reset your password.
              </p>
            </div>
            """, unsafe_allow_html=True)

            # ── Show delivery status banner ──────────────────
            if send_msg == "real":
                st.success(
                    f"📧 OTP sent to **{fp_email_display}** — check your inbox "
                    f"(and spam folder if not seen within 1 minute)."
                )
```

```
            elif send_msg == "demo":
                st.info(
                    f"🧪 **Demo Mode** — email not configured yet.\n\n"
                    f"Your OTP code is: **`{otp_correct}`**\n\n"
                    f"*(Add your Gmail App Password in `config.py` to enable real
emails)*"
                )
            elif send_msg.startswith("error:"):
                err_detail = send_msg.replace("error:", "")
                st.warning(
                    f"⚠️ Could not send email ({err_detail}).\n\n"
                    f"OTP code: **`{otp_correct}`**"
                )

            otp_entered = st.text_input(
                "Enter 6-digit OTP", placeholder="e.g. 482910",
                max_chars=6, key="fp_otp_input",
            )
            fp1, fp2 = st.columns(2)
            with fp1:
                fp_new  = st.text_input("New Password", type="password",
                                        placeholder="Min 8 chars…", key="fp_new")
            with fp2:
                fp_conf = st.text_input("Confirm New Password", type="password",
                                        placeholder="Repeat", key="fp_conf")
            if fp_new:
                show_password_rules(fp_new)
```

```
            with st.form("fp_reset_form"):
                reset_btn = st.form_submit_button("Reset Password →",
use_container_width=True)

            if reset_btn:
                import time
                fp_ok, fp_issues = check_password_strength(fp_new)
otp_expiry = st.session_state.get("fp_otp_expiry", 0)
                if not otp_entered.strip():
                    st.error("Please enter the OTP code.")
                elif time.time() > otp_expiry:
                    st.error("⏰ OTP has expired (10 min limit). Please request a
new
one.")
                    for k in ("fp_otp", "fp_email", "fp_otp_expiry",
"fp_send_msg"):
                        st.session_state.pop(k, None)
                    st.session_state["auth_view"] = "forgot_email"
                    st.rerun()
                elif otp_entered.strip() != otp_correct:
                    st.error("❌ Incorrect OTP. Please check and try again.")
                elif not fp_new or not fp_conf:
                    st.error("Please enter and confirm your new password.")
elif fp_new != fp_conf:
                    st.error("❌ Passwords do not match.")
                elif not fp_ok:
                    st.error("❌ Password too weak:\n- " + "\n- ".join(fp_issues))
```

```
                else:
                    try:
                        from db import run_query as _rq3, run_update as _ru
                        from auth import hash_password
                        uid_row = _rq3(
                            "SELECT UserID FROM User WHERE Email=%s AND IsActive=1
LIMIT 1",
                            (fp_email_display,),
                        )
                        if uid_row:
                            _ru("UPDATE User SET Password=%s WHERE UserID=%s",
                                (hash_password(fp_new), uid_row[0]["UserID"]))
                            for k in ("fp_otp", "fp_email"):
                                st.session_state.pop(k, None)
                            st.session_state["auth_view"] = "login"
                            st.success("✅ Password reset! You can now sign in.")
                            st.rerun()
                        else:
                            st.error("Account not found.")
                    except Exception as e:
                        st.error(f"⚠️ Error: {e}")

            st.markdown(
                "<hr style='border:none;border-top:1px solid #e5e7eb;margin:1.2rem
0'>",
                unsafe_allow_html=True,
            )
            st.markdown(
                "<span style='color:#6b7280;font-size:.88rem'>Wrong email?
</span>",
                unsafe_allow_html=True,
            )
            if st.button("Try a different email", key="back_to_fp_email"):
                for k in ("fp_otp", "fp_email"):
                    st.session_state.pop(k, None)
                st.session_state["auth_view"] = "forgot_email"
                st.rerun()

        # ── Footer ──────────────────────────────────────────
        st.markdown(
            "<div style='text-align:center;color:#aaa;font-size:.74rem;padding-
top:2rem'>"
            "WasteLens v1.0  |  BIS 698 – Group 3</div>",
            unsafe_allow_html=True,
        )
```

4. Authentication Helpers (auth.py)

4.1 Design View

auth.py is a shared utility module imported by app.py and every page. It provides all security-related functions so that no page needs to implement its own auth logic. It exposes the following public API:

- hash_password(plain: str) -> str – bcrypt-hashes a plaintext password, returns UTF-8 string.
- verify_password(plain: str, hashed: str) -> bool – Checks a plaintext password against a stored bcrypt hash.
- login(email, password) – Full login flow: DB lookup, bcrypt verify, active/approved check. Returns (user_dict, None) on success or (None, error_string) on failure. user_dict includes branch_ids: list[int].
- register(name, email, password, branch_id) – Inserts a new Staff user with IsApproved=0 and assigns branch access. Returns (True, success_msg) or (False, error_msg).
- logout() – Clears all keys from st.session_state.
- require_auth() – Page guard. If no user is in session, renders a lock screen and calls st.stop() to halt all further code execution.
- require_role(allowed: list[str]) – Extends require_auth(). Also checks that the logged-in user's RoleName is in the allowed list. Calls st.stop() if not.
- branch_filter_sql(user, alias="wl") -> str – Returns a SQL AND fragment (e.g., "AND wl.BranchID IN (1,2,3)") for non-Admin users to scope queries to their accessible branches. Returns empty string for Admin.

4.2 Execution View

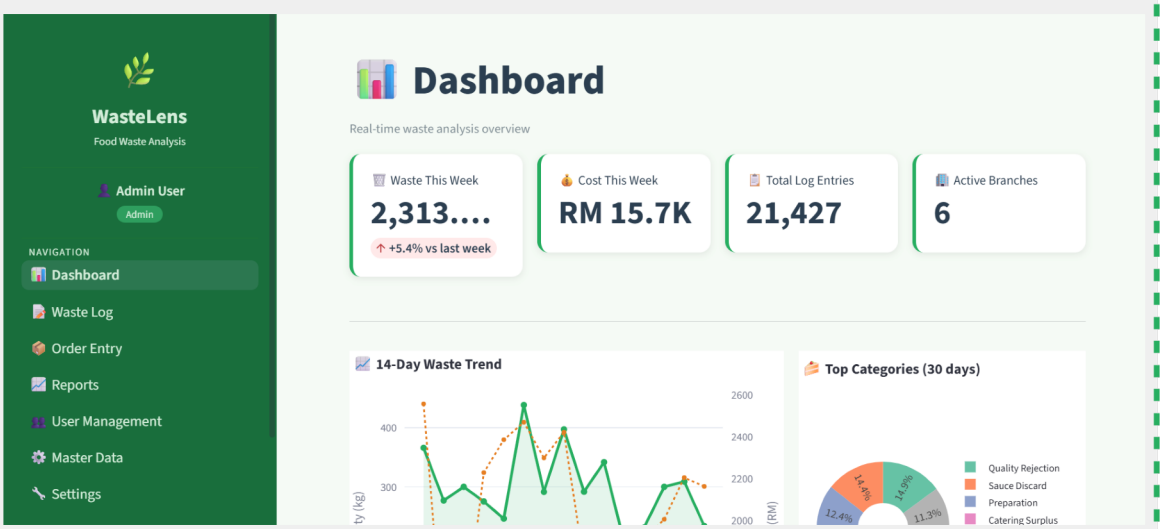
Every protected page starts with either user = require_auth() or user = require_role(["Admin"]) / require_role(["Admin","Manager"]). These are the first executed statements after inject_css(). If the user is not authenticated or lacks the required role, st.stop() is called immediately and no further code on that page renders. This prevents partial rendering of restricted content.

branch_filter_sql() is called once near the top of each data page and the resulting SQL fragment bf is reused in all subsequent queries. This keeps branch-scoping logic in a single location and avoids repetition across Dashboard, Waste Log, Reports, and Order Entry.

4.3 Screenshots – Auth Guards

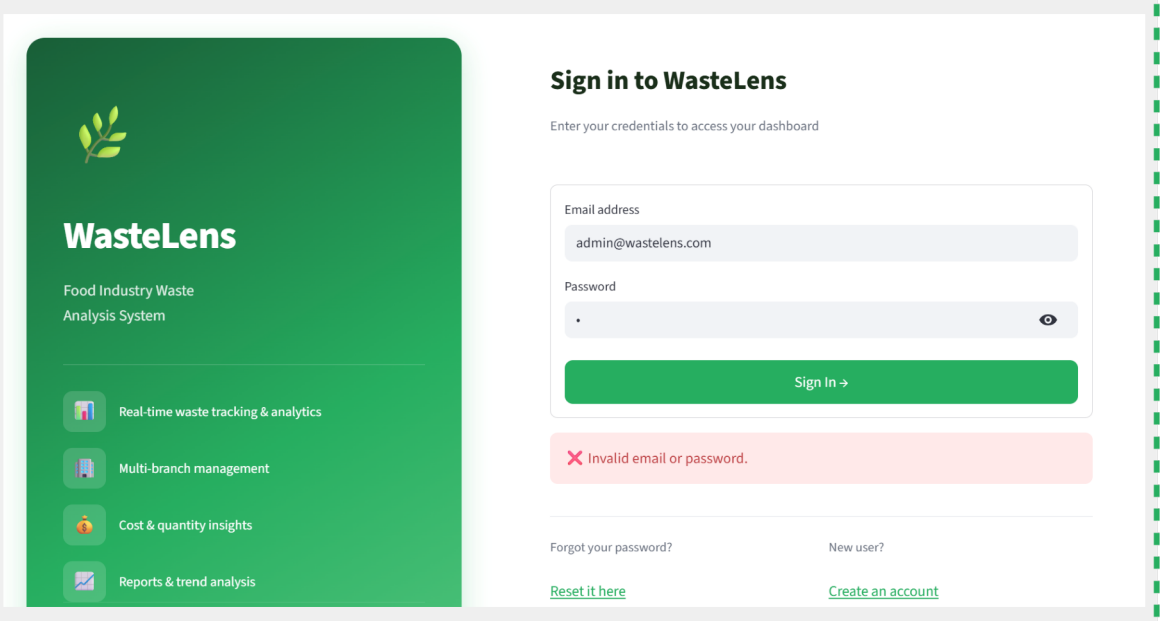
Session Expired / Not Logged In Screen

[SCREENSHOT — Lock screen shown when accessing a page without a valid session – lock icon and "Go to Login" button]



Role Access Denied Screen

[SCREENSHOT — Access denied message when a user tries to access a page above their role (e.g. Staff accessing Reports)]



4.4 Source Code – auth.py

```
# ============================================================
```

```
#  WasteLens – Authentication & Session Helpers
# ============================================================
import bcrypt
import streamlit as st
from db import run_query, run_update



# ── password helpers ──────────────────────────────────────
def hash_password(plain: str) -> str:
    return bcrypt.hashpw(plain.encode(), bcrypt.gensalt()).decode()



def verify_password(plain: str, hashed: str) -> bool:
    try:
        return bcrypt.checkpw(plain.encode(), hashed.encode())
    except Exception:
        return False



# ── login ─────────────────────────────────────────────────
def login(email: str, password: str):
    """
    Return (user_dict, None) on success or (None, error_str) on failure.
    user_dict includes  branch_ids: list[int].
    """
    sql = """
        SELECT u.UserID, u.UserName, u.Email, u.Password,
               u.IsApproved, u.IsActive, r.RoleName,
               GROUP_CONCAT(uba.BranchID ORDER BY uba.BranchID) AS BranchIDs
        FROM   User u
        JOIN   Role r  ON r.RoleID  = u.RoleID
        LEFT JOIN UserBranchAccess uba ON uba.UserID = u.UserID
        WHERE  u.Email = %s
        GROUP BY u.UserID
    """
    rows = run_query(sql, (email,))
    if not rows:
        return None, "Invalid email or password."

    u = rows[0]
    if not verify_password(password, u["Password"]):
        return None, "Invalid email or password."
    if not u["IsActive"]:
        return None, "Account deactivated. Contact your administrator."
    if not u["IsApproved"]:
        return None, "Account pending approval. Please wait for admin."

    u["branch_ids"] = (
        [int(x) for x in u["BranchIDs"].split(",")]
        if u["BranchIDs"]
        else []
    )
    return u, None
```

```
# ── register ──────────────────────────────────────────────
def register(username: str, email: str, password: str, branch_id: int):
    """Return (True, success_msg) or (False, error_msg)."""
```

```
    if run_query("SELECT UserID FROM User WHERE Email = %s", (email,)):
        return False, "Email already registered."

    role = run_query("SELECT RoleID FROM Role WHERE RoleName = 'Staff'")
    if not role:
        return False, "System configuration error – Staff role missing."

    hashed = hash_password(password)
    try:
        uid = run_update(
            """INSERT INTO User (UserName, Email, Password, IsApproved, IsActive,
RoleID)
               VALUES (%s, %s, %s, 0, 1, %s)""",
            (username, email, hashed, role[0]["RoleID"]),
        )
        if uid and branch_id:
            run_update(
                "INSERT IGNORE INTO UserBranchAccess (UserID, BranchID,
AccessType)"
                " VALUES (%s, %s, 'Write')",
                (uid, branch_id),
            )
        return True, "Registration successful! Your account is awaiting admin
approval."
    except Exception as e:
        return False, f"Registration failed: {e}"


# ── session helpers ───────────────────────────────────────
def logout():
    """Clear user session and navigate back to login."""
    for key in list(st.session_state.keys()):
        del st.session_state[key]


def require_auth():
    """Stops the page if user is not logged-in. Returns user dict."""
    if "user" not in st.session_state:
        st.markdown(
            """
            <div style='text-align:center;padding:3rem 1rem'>
                <div style='font-size:3rem'>🔒</div>
                <h3 style='color:#27AE60'>Session Expired</h3>
                <p style='color:#666'>Please log in to continue.</p>
            </div>
            """,
            unsafe_allow_html=True,
        )
        col1, col2, col3 = st.columns([1, 2, 1])
with col2:
            if st.button("🔑  Go to Login", use_container_width=True,
```

```
type="primary"):
                st.switch_page("app.py")
        st.stop()
    return st.session_state["user"]



def require_role(allowed: list[str]):
    """Stops the page if user's role is not in *allowed*. Returns user dict."""
    user = require_auth()
    if user["RoleName"] not in allowed:
        st.error(f"⛔ Access restricted to: {', '.join(allowed)}")
        st.stop()
    return user

 def branch_filter_sql(user: dict, alias: str = "wl") ->
str:
    """
    Return a SQL fragment like ' AND wl.BranchID IN (1,2,3)'
    for non-Admin users, or '' for Admin (sees everything).
    """
    if user["RoleName"] == "Admin":
        return ""
    ids = user.get("branch_ids", [])
    if not ids:
        return " AND 1=0"   # no branches → see nothing
    return f" AND {alias}.BranchID IN ({','.join(str(i) for i in ids)})"
```

5. Database Layer (db.py)

5.1 Design View

db.py is the complete database abstraction layer. Every MySQL interaction in the application is routed through three functions. This centralises connection management and error handling.

- get_connection() – Reads DB_CONFIG from config.py and calls mysql.connector.connect(). Raises ConnectionError with a human-readable message if the connection fails.
- run_query(sql, params) – Opens a dictionary cursor (returns list[dict]), executes the parameterised SQL, returns all rows, and closes the connection in a finally block.
- run_update(sql, params) – Opens a standard cursor, executes INSERT/UPDATE/DELETE, commits, returns cursor.lastrowid for INSERT operations. Rolls back and re-raises on any mysql.connector.Error.
- init_passwords() – Called at startup. Two operations: (1) Replace "Placeholder" password strings with real bcrypt hashes for the seeded demo accounts. (2) Auto-create team-member accounts (Dhana Lakshmi, Kannu, Chiri) if they do not already exist in the User table.

5.2 Execution View

All SQL in the application uses parameterised queries with %s or %(key)s placeholders. No user-supplied string is ever concatenated directly into a SQL statement. The dictionary=True cursor option means every row is returned as a Python dict keyed by column name, making column access readable and refactor-safe.

5.3 Source Code – db.py

```
# ============================================================
#  WasteLens - Database Layer
# ============================================================
import mysql.connector
from mysql.connector import Error
from config import DB_CONFIG


def get_connection():
    """Return a live MySQL connection or raise ConnectionError."""
    try:
        return mysql.connector.connect(**DB_CONFIG)
    except Error as e:
        raise ConnectionError(f"DB connection failed: {e}")


def run_query(sql: str, params: tuple = None) -> list[dict]:
    """Execute a SELECT and return list-of-dicts."""
    conn = get_connection()
try:
        cur = conn.cursor(dictionary=True)
        cur.execute(sql, params or ())
```

```
        return cur.fetchall()
    finally:
        conn.close()



def run_update(sql: str, params: tuple = None) -> int:
    """Execute INSERT/UPDATE/DELETE, commit, return lastrowid."""
conn = get_connection()
    try:
        cur = conn.cursor()
        cur.execute(sql, params or ())
        conn.commit()
        return cur.lastrowid
    except Error:
        conn.rollback()
        raise
    finally:
        conn.close()



def init_passwords():
    """
    Replace SQL placeholder hashes with real bcrypt hashes on first run.
    Also creates any missing team-member accounts automatically.
    Safe to call repeatedly - no-op for rows that are already correct.
    """
    import bcrypt
    from config import DEFAULT_PASSWORDS

    # ── 1. Fix placeholder hashes for existing users ──────────
    for email, plain in DEFAULT_PASSWORDS.items():
        rows = run_query("SELECT UserID, Password FROM User WHERE Email = %s",
(email,))
        if rows and "Placeholder" in (rows[0]["Password"] or ""):
            hashed = bcrypt.hashpw(plain.encode(), bcrypt.gensalt()).decode()
run_update("UPDATE User SET Password = %s WHERE Email = %s", (hashed,
email))

    # ── 2. Auto-create team members if they don't exist yet ─────
    TEAM_EXTRAS = {
        "dhanalakshmikm25@gmail.com": ("Dhana Lakshmi", "Admin"),
        "kannu1d@cmich.edu":          ("Kannu",         "Manager"),
        "chiri1s@cmich.edu":          ("Chiri",         "Staff"),
    }

    for email, (name, role_name) in TEAM_EXTRAS.items():
        existing = run_query("SELECT UserID FROM User WHERE Email=%s", (email,))
        if existing:
            continue   # already in DB - skip

        role_row = run_query("SELECT RoleID FROM Role WHERE RoleName=%s",
(role_name,))
        if not role_row:
            continue
```

```
        plain  = DEFAULT_PASSWORDS.get(email, "Welcome@123")
        hashed = bcrypt.hashpw(plain.encode(), bcrypt.gensalt()).decode()

        try:
            uid = run_update(
                """INSERT INTO User (UserName, Email, Password, IsApproved, IsActive,
RoleID)
                   VALUES (%s, %s, %s, 1, 1, %s)""",
                (name, email, hashed, role_row[0]["RoleID"]),
            )
            # assign to first available branch
            branch = run_query("SELECT BranchID FROM Branch ORDER BY BranchID LIMIT
1")
            if branch and uid:
                run_update(
                    "INSERT IGNORE INTO UserBranchAccess (UserID, BranchID,
AccessType)"
                    " VALUES (%s, %s, 'Write')",
                    (uid, branch[0]["BranchID"]),
                )
        except Exception:
            pass   # silently skip if any constraint issue
```

6. Configuration Files

6.1 config.py

config.py is the central configuration file. It contains the MySQL connection parameters, default plaintext passwords for seeded demo accounts, app cosmetic settings, and the fallback Gmail SMTP configuration. This file is the only place that needs editing when deploying to a new machine with different MySQL credentials.

- DB_CONFIG – host, port, database name, user, and password for MySQL.
- DEFAULT_PASSWORDS – Plaintext passwords for the six demo/team accounts. Used only by init_passwords() to seed bcrypt hashes on first run.
- APP_TITLE, APP_ICON, PRIMARY_CLR – Cosmetic constants.
- EMAIL_CONFIG – Fallback Gmail SMTP settings used when local_settings.json does not exist or is incomplete.

6.2 email_utils.py

email_utils.py handles all outbound email. It exposes a single public function: send_otp_email(to_email, otp_code, user_name). Internally it first calls _get_email_cfg() which reads local_settings.json (per-machine override) and falls back to config.py EMAIL_CONFIG. It then sends a styled HTML email with the OTP code via smtplib on Gmail port 587 with STARTTLS.

Return values from send_otp_email():

- (True, "sent") – Email sent successfully.
- (False, "EMAIL_NOT_CONFIGURED") – No app_password configured; app falls back to Demo Mode.
- (False, "AUTH_FAILED") – Wrong App Password or 2-Step Verification not enabled.
- (False, "INVALID_EMAIL") – Recipient address rejected by Gmail.
- (False, "<exception string>") – Any other SMTP or network error.

6.3 Source Code – config.py

```
# ============================================================
#  WasteLens - Configuration
#  Edit DB_CONFIG to match your MySQL setup
# ============================================================

DB_CONFIG = {
    "host": "localhost",
    "port": 3306,
    "database": "wastelens",
    "user": "root",
    "password": "Root@123",          # <— set your MySQL root password here
}

# Default passwords injected on first run (replaces SQL placeholder hashes)
DEFAULT_PASSWORDS = {
    "admin@wastelens.com":        "Admin@123",
    "manager@wastelens.com":      "Manager@123",
```

```
    "staff@wastelens.com":        "Staff@123",
    "dhanalakshmikm25@gmail.com": "Dhana@123",
    "kannu1d@cmich.edu":          "123456",
    "chiri1s@cmich.edu":          "Chiri@123",
}

APP_TITLE   = "WasteLens"
APP_ICON    = "🌿"
PRIMARY_CLR = "#27AE60"

# ── Email / SMTP (for OTP password reset) ──────────────────────
# Step 1: Enable 2-Step Verification on your Gmail account
# Step 2: Go to https://myaccount.google.com/apppasswords
# Step 3: Create an App Password → copy the 16-char code below
EMAIL_CONFIG = {
    "smtp_host":   "smtp.gmail.com",
    "smtp_port":   587,
    "sender_email": "dhanalakshmikm25@gmail.com",   # <── your Gmail
    "app_password": "soal ypde vhvs mkie",                                  # <── 16-
char
App Password
    "sender_name":  "WasteLens",
}
```

6.4 Source Code – email_utils.py

```
# ============================================================
#  WasteLens – Email Utility  (Gmail SMTP + OTP mailer)
# ============================================================
import smtplib, json, os
from email.mime.multipart import MIMEMultipart
from email.mime.text      import MIMEText
from config import EMAIL_CONFIG

_SETTINGS_FILE = os.path.join(os.path.dirname(__file__), "local_settings.json")

def _get_email_cfg() -> dict:
    """Read from local_settings.json first; fall back to config.py."""
    if os.path.exists(_SETTINGS_FILE):
        try:
            with open(_SETTINGS_FILE, "r") as f:
                local = json.load(f)
            if local.get("app_password") and local.get("sender_email"):
                return local
        except Exception:
            pass
    return EMAIL_CONFIG



def send_otp_email(to_email: str, otp_code: str, user_name: str = "") -> tuple[bool,
str]:
    """
    Send a styled OTP email via Gmail SMTP.
    Returns (success: bool, message: str).
    """
```

```
    cfg = _get_email_cfg()
    if not cfg.get("app_password"):
        return False, "EMAIL_NOT_CONFIGURED"
```

```
    subject = "WasteLens – Your Password Reset Code"
    greeting = f"Hi {user_name}," if user_name else "Hello,"

    html_body = f"""
    <!DOCTYPE html>
    <html>
    <head>
      <meta charset="UTF-8">
      <style>
        body {{ font-family: 'Segoe UI', Arial, sans-serif; background: #f0faf5;
margin:0; padding:0; }}
        .wrapper {{ max-width:520px; margin:40px auto; background:white;
                    border-radius:16px; overflow:hidden;
                    box-shadow:0 4px 24px rgba(0,0,0,.10); }}
        .header  {{ background:linear-gradient(135deg,#1a5e38,#27AE60);
                    padding:2rem; text-align:center; }}
        .header  h1  {{  color:white;  font-size:1.6rem;  margin:0;  letter-spacing:-
0.5px;
}}
        .header p  {{ color:rgba(255,255,255,.8); font-size:.9rem; margin:.4rem 0 0;
}}
        .body    {{ padding:2rem 2.5rem; }}
        .otp-box {{ background:#f0faf5; border:2px dashed #27AE60;
                    border-radius:12px; text-align:center;
                    padding:1.5rem; margin:1.5rem 0; }}
        .otp-code {{ font-size:2.6rem; font-weight:900; letter-spacing:10px;
                     color:#1a5e38; font-family:monospace; }}
        .otp-label {{ font-size:.78rem; color:#6b7280; margin-top:.4rem; }}
        .note    {{ background:#fff9e6; border-left:4px solid #f0ad4e;
                    border-radius:6px; padding:.8rem 1rem;
                    font-size:.82rem; color:#92400e; margin-top:1rem; }}
        .footer  {{ background:#f9fafb; padding:1rem 2.5rem;
                    text-align:center; font-size:.75rem; color:#9ca3af;
                    border-top:1px solid #e5e7eb; }}
        p {{ color:#374151; line-height:1.6; font-size:.92rem; }}
      </style>
    </head>
    <body>
      <div class="wrapper">
        <div class="header">
          <h1>🌿 WasteLens</h1>
          <p>Food Industry Waste Analysis System</p>
        </div>
        <div class="body">
          <p>{greeting}</p>
          <p>We received a request to reset your WasteLens password.
             Use the code below to continue. This code is valid for
             <strong>10 minutes</strong>.</p>

          <div class="otp-box">
            <div class="otp-code">{otp_code}</div>
            <div class="otp-label">One-Time Password (OTP)</div>
```

```
          </div>

          <div class="note">
            ⚠️ <strong>Do not share this code</strong> with anyone.
            WasteLens staff will never ask for your OTP.
          </div>

          <p style="margin-top:1.2rem">
            If you did not request a password reset, you can safely
            ignore this email — your password will not change.
          </p>
        </div>
        <div class="footer">
          WasteLens v1.0  |  BIS 698 – Group 3  | 
          This is an automated message, please do not reply.
        </div>
      </div>
    </body>
    </html>
    """

    try:
        msg = MIMEMultipart("alternative")
        msg["Subject"] = subject
        msg["From"]    = f"{cfg['sender_name']} <{cfg['sender_email']}>"
        msg["To"]      = to_email

        # plain-text fallback
        plain = (f"{greeting}\n\nYour WasteLens password reset code is:
{otp_code}\n\n"
                 f"This code expires in 10 minutes.\n\n"
                 f"If you did not request this, ignore this email.")
        msg.attach(MIMEText(plain, "plain"))
        msg.attach(MIMEText(html_body, "html"))

        with smtplib.SMTP(cfg["smtp_host"], cfg["smtp_port"]) as server:
            server.ehlo()
            server.starttls()
            server.login(cfg["sender_email"], cfg["app_password"])
            server.sendmail(cfg["sender_email"], to_email, msg.as_string())

        return True, "sent"

    except smtplib.SMTPAuthenticationError:
        return False, "AUTH_FAILED"
    except smtplib.SMTPRecipientsRefused:
        return False, "INVALID_EMAIL"
    except Exception as e:
        return False, str(e)
```

7. Dashboard (1_Dashboard.py)

7.1 Design View

The Dashboard is the first page users see after login. It delivers an immediate visual overview of waste performance through metric cards, trend charts, and a recent activity table. The layout adapts by role: Admin and Manager see the Branch Comparison chart; Staff see only their branch data.

KPI Row (4 metric cards):

- Waste This Week (kg) – Sum of Quantity for the last 7 days. Shows delta percentage vs. the prior 7-day window.
- Cost This Week (RM) – Sum of EstimatedCost for the last 7 days. Large values formatted as K/M shorthand.
- Total Log Entries – Count of all WasteLog rows accessible to the user. ● Active Branches – Count of Branch rows where IsActive=1.

14-Day Waste Trend (dual-axis chart):

- X-axis: one tick per calendar day, formatted as "Mon DD". dt.normalize() strips time components.
- Primary y-axis (left, green): Quantity (kg) rendered as a filled area trace.
- Secondary y-axis (right, orange): Cost (RM) rendered as a dashed dot line. ● Hover mode: "x unified" shows both values at once on cursor hover.

Top Categories Donut Chart (30-day window):

- Up to 8 waste categories ranked by quantity.
- Percent labels shown inside the donut slices.
- Vertical legend anchored to the right of the chart.

Branch Comparison Bar Chart (Admin / Manager only):

- Current month waste per branch as a colour-gradient bar chart. ● Not rendered for Staff role.

Recent Waste Logs Table:

- Last 15 waste log entries in a styled st.dataframe(), scoped by branch access.

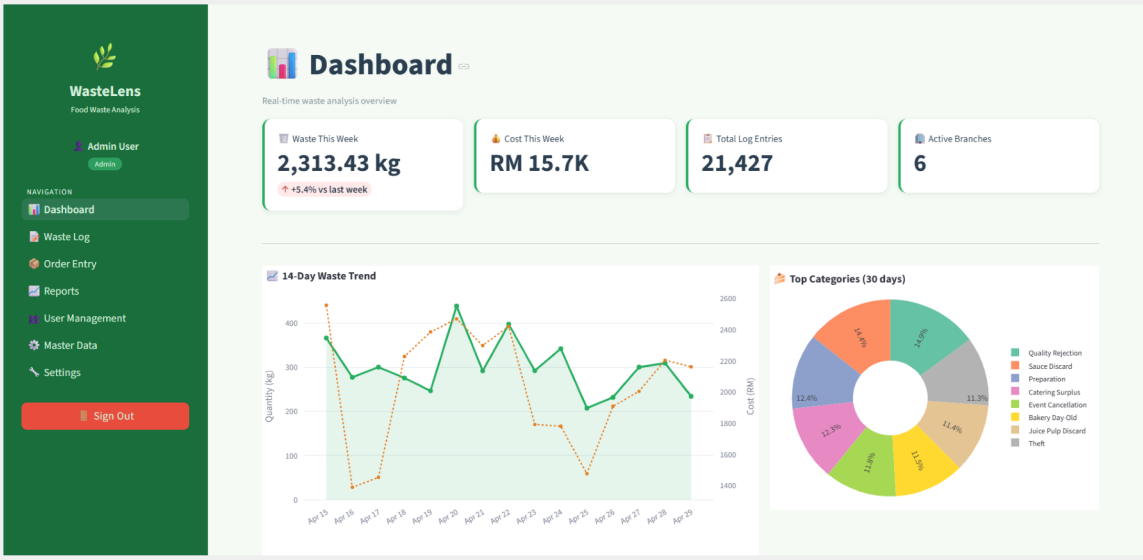
7.2 Execution View

require_auth() is called first, then branch_filter_sql(user) produces the bf SQL fragment. All five database queries run on every page load. The delta percentage on the weekly waste KPI is only computed and displayed when the prior week had non-zero data (wlw > 0). The fmt_rm() helper function formats large RM values as "RM 1.2K" or "RM 3.4M" to prevent metric card text overflow.

7.3 Screenshots – Dashboard

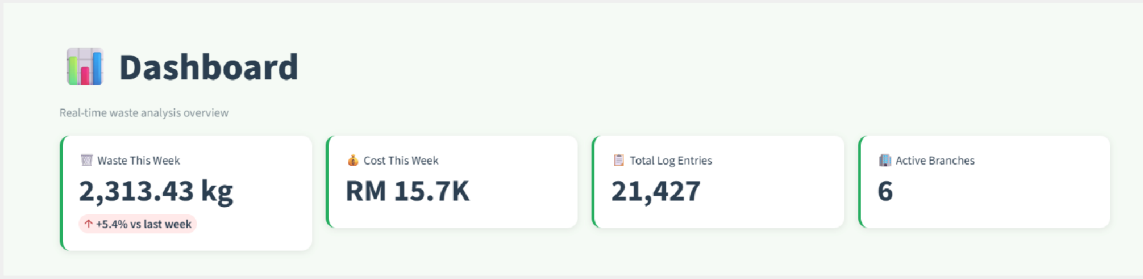
Dashboard – Full View (Admin)

[SCREENSHOT — Full Dashboard page – KPI cards, 14-day trend chart, category donut, branch comparison, recent logs]



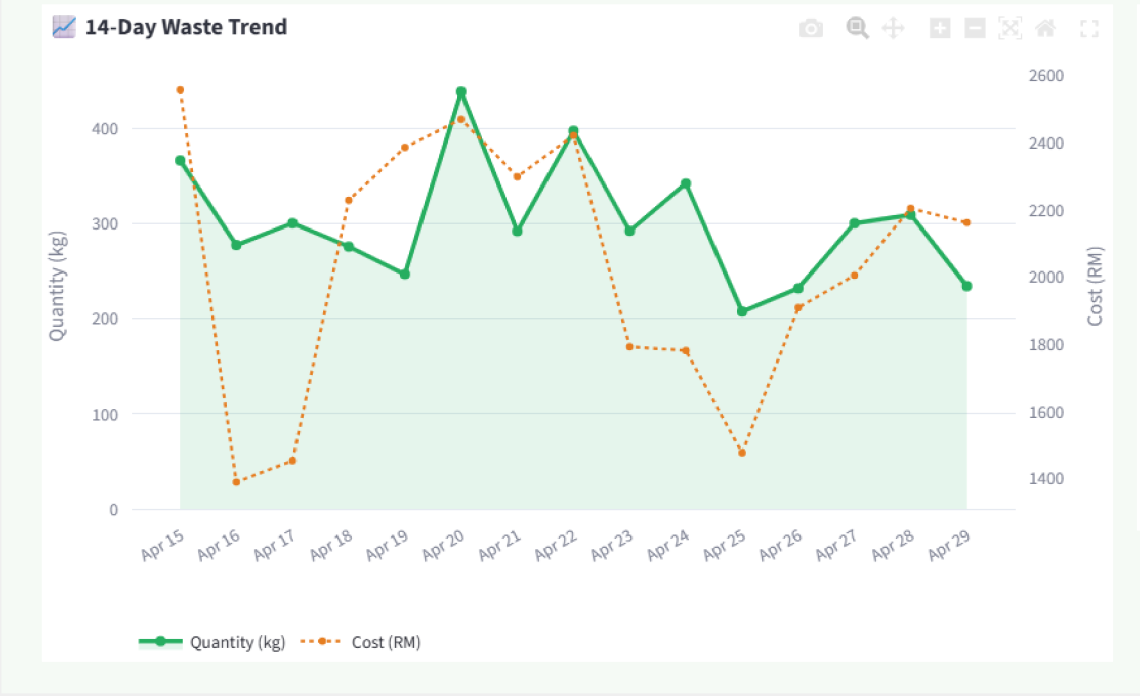
KPI Metric Cards Row

[SCREENSHOT — Close-up of the 4 KPI cards: Waste This Week, Cost This Week, Total Entries, Active Branches]



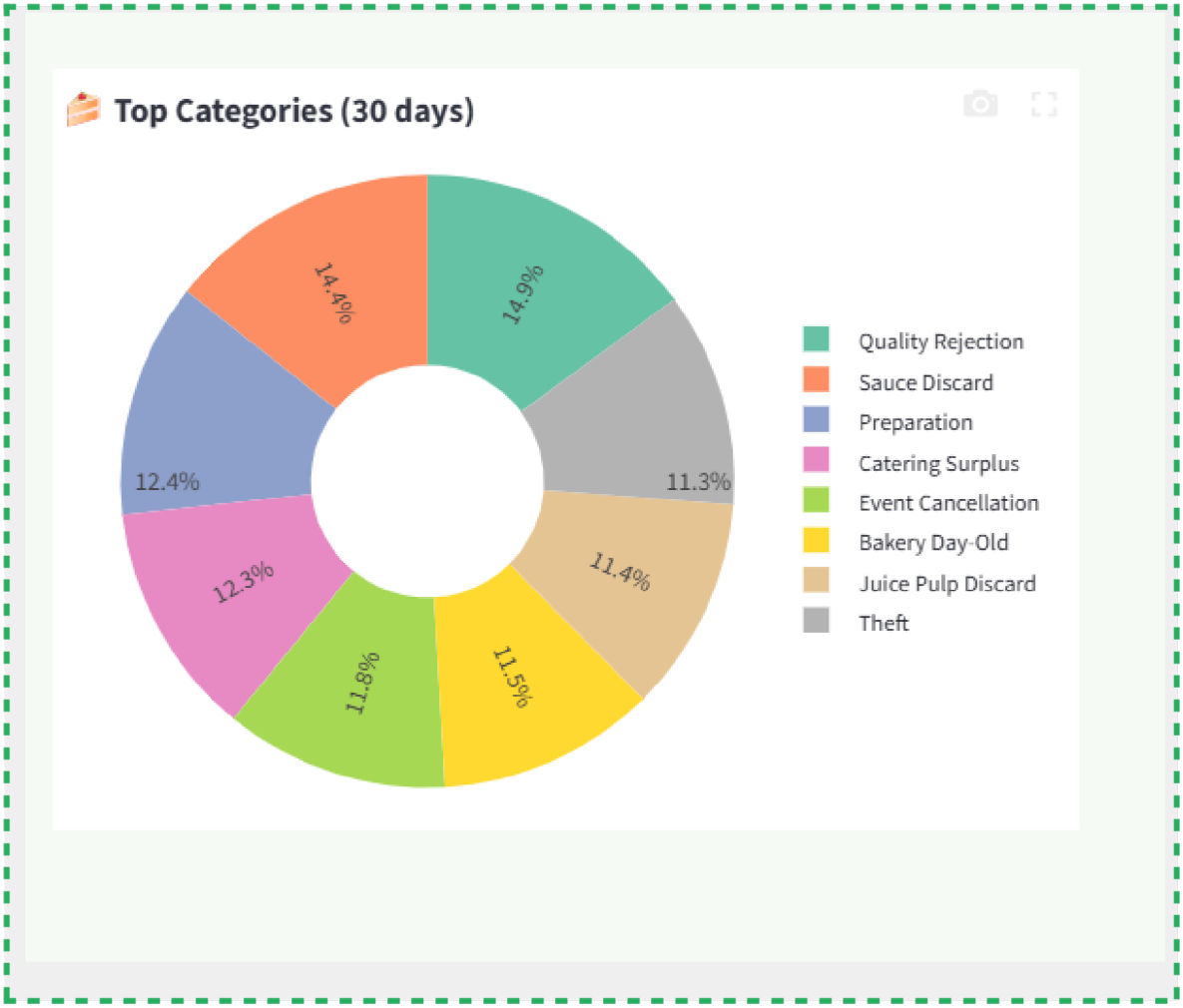
14-Day Waste Trend Chart

[SCREENSHOT — 14-day dual-axis area+line chart – green area (quantity) and dashed orange line (cost)]



Top Categories Donut Chart

[SCREENSHOT — Category donut/pie chart with percent labels inside slices and vertical legend on right]

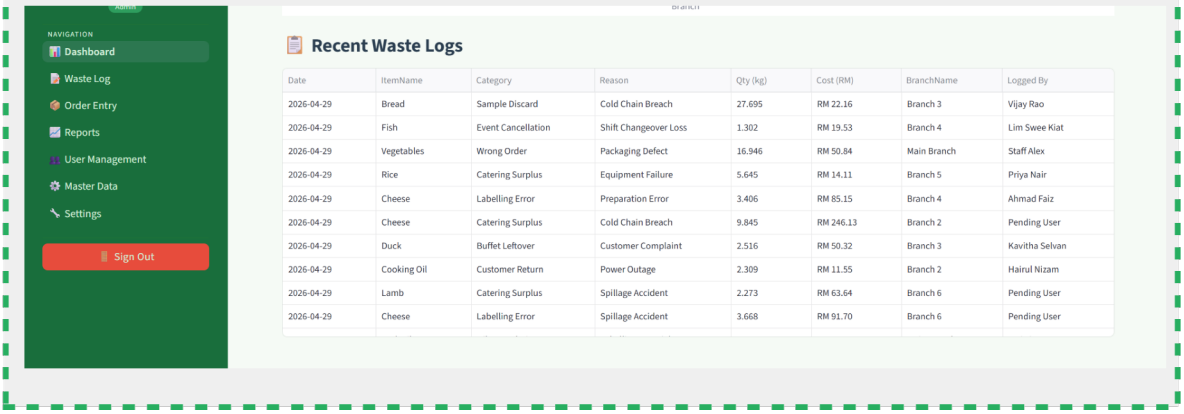


Branch Comparison Bar Chart

[SCREENSHOT — Branch comparison colour-gradient bar chart – waste by branch for current month (Admin/Manager view)]

Recent Waste Logs Table

[SCREENSHOT — Recent 15 waste log entries in styled dataframe table]



7.4 Source Code – 1_Dashboard.py

```
# ============================================================
#  WasteLens - Dashboard
# ============================================================
import streamlit as st

st.set_page_config(
    page_title="Dashboard | WasteLens",
    page_icon="📊",
    layout="wide",
)
```

```
import pandas as pd
import plotly.express as px
import plotly.graph_objects as go

from auth import require_auth, branch_filter_sql
from db import run_query
from utils import inject_css, render_sidebar

inject_css()
user = require_auth()
render_sidebar(user)

bf = branch_filter_sql(user)   # branch filter for WasteLog queries

# == KPI CARDS ============================================
st.title("📊 Dashboard")
st.caption("Real-time waste analysis overview")

c1, c2, c3, c4 = st.columns(4)

def _val(rows, key="val"):
    return (rows[0][key] or 0) if rows else 0

waste_this  = run_query(
    f"SELECT ROUND(SUM(Quantity),2) AS val FROM WasteLog wl"
    f" WHERE WasteDateTime >= CURDATE()-INTERVAL 7 DAY {bf}"
)
waste_last  = run_query(
    f"SELECT ROUND(SUM(Quantity),2) AS val FROM WasteLog wl"
    f" WHERE WasteDateTime >= CURDATE()-INTERVAL 14 DAY"
    f" AND WasteDateTime < CURDATE()-INTERVAL 7 DAY {bf}"
)
cost_this   = run_query(
    f"SELECT ROUND(SUM(EstimatedCost),2) AS val FROM WasteLog wl"
    f" WHERE WasteDateTime >= CURDATE()-INTERVAL 7 DAY {bf}"
)
total_logs  = run_query(
    f"SELECT COUNT(*) AS val FROM WasteLog wl WHERE 1=1 {bf}"
)
active_br   = run_query("SELECT COUNT(*) AS val FROM Branch WHERE IsActive=1")

ww  = float(_val(waste_this))
wlw = float(_val(waste_last))
cw  = float(_val(cost_this))
tl  = int(_val(total_logs))
ab  = int(_val(active_br))

# only show delta when last week had data
if wlw > 0:
    delta_pct = round((ww - wlw) / wlw * 100, 1)
    delta_str = f"{delta_pct:+.1f}% vs last week"
    d_color   = "inverse"   # red = more waste = bad
else:
    delta_str = None
    d_color   = "off"
```

```
# shorten large RM values so they don't truncate
def fmt_rm(val):
    if val >= 1_000_000:
        return f"RM {val/1_000_000:.2f}M"
    if val >= 1_000:
        return f"RM {val/1_000:.1f}K"
    return f"RM {val:,.2f}"

with c1:
    st.metric("🗑️ Waste This Week", f"{ww:,.2f} kg",
              delta=delta_str, delta_color=d_color)
with c2:
    st.metric("💰 Cost This Week", fmt_rm(cw))
with c3:
    st.metric("📋 Total Log Entries", f"{tl:,}")
with c4:
    st.metric("🏬 Active Branches", f"{ab}")

st.markdown("---")

# ── CHARTS ROW ──────────────────────────────────────────
col_line, col_pie = st.columns([3, 2])
 with
col_line:
    trend = run_query(
        f"""SELECT DATE(wl.WasteDateTime) AS d,
                   ROUND(SUM(wl.Quantity),2)      AS Quantity_kg,
                   ROUND(SUM(wl.EstimatedCost),2) AS Cost_RM
            FROM WasteLog wl WHERE 1=1 {bf}
            AND wl.WasteDateTime >= CURDATE()-INTERVAL 14 DAY
            GROUP BY DATE(wl.WasteDateTime)
            ORDER BY d"""
    )
    if trend:
        df_t = pd.DataFrame(trend)
        # normalise to date-only so x-axis shows clean dates, not timestamps
        df_t["d"] = pd.to_datetime(df_t["d"]).dt.normalize()
df_t["Quantity_kg"] = pd.to_numeric(df_t["Quantity_kg"],
errors="coerce").fillna(0)
        df_t["Cost_RM"]     = pd.to_numeric(df_t["Cost_RM"],
errors="coerce").fillna(0)

        fig = go.Figure()
        # Area fill for quantity
        fig.add_trace(go.Scatter(
            x=df_t["d"], y=df_t["Quantity_kg"],
            name="Quantity (kg)", mode="lines+markers",
            fill="tozeroy", fillcolor="rgba(39,174,96,.12)",
            line=dict(color="#27AE60", width=3),
            marker=dict(size=7, color="#27AE60"),
        ))
        # Cost as a separate dashed line on secondary y-axis
        fig.add_trace(go.Scatter(
            x=df_t["d"], y=df_t["Cost_RM"],
```

```
            name="Cost (RM)", mode="lines+markers",
            line=dict(color="#e67e22", width=2, dash="dot"),
            marker=dict(size=5, color="#e67e22"),
            yaxis="y2",
        ))
        fig.update_layout(

            title="📈 14-Day Waste Trend",
            xaxis=dict(
                tickformat="%b %d",
                dtick=86_400_000,        # one tick per day (ms)
                tickangle=-30,
            ),
            yaxis =dict(title="Quantity (kg)", color="#27AE60"),
            yaxis2=dict(title="Cost (RM)",     overlaying="y", side="right",
                        color="#e67e22", showgrid=False),
            legend=dict(orientation="h", y=-0.25),
            plot_bgcolor="white", paper_bgcolor="white",
            margin=dict(t=40, b=20),
            hovermode="x unified",
        )
        st.plotly_chart(fig, use_container_width=True)
    else:
        st.info("No waste trend data for the last 14 days.")

with col_pie:
    cat_data = run_query(
        f"""SELECT wc.TypeName AS Category,
                   ROUND(SUM(wl.Quantity),2) AS Quantity_kg
            FROM WasteLog wl
            JOIN WasteCategory wc ON wc.WasteCategoryID=wl.WasteCategoryID
            WHERE 1=1 {bf}
            AND wl.WasteDateTime >= CURDATE()-INTERVAL 30 DAY
            GROUP BY wc.TypeName ORDER BY Quantity_kg DESC LIMIT 8"""
    )
    if cat_data:
        df_c = pd.DataFrame(cat_data)
        fig2 = px.pie(
            df_c, names="Category", values="Quantity_kg",
            title="🥧 Top Categories (30 days)",
            color_discrete_sequence=px.colors.qualitative.Set2,
            hole=0.38,
        )
        fig2.update_traces(
            textposition="inside",
            textinfo="percent",
            insidetextorientation="radial",
            hovertemplate="<b>%{label}</b><br>%{value:.2f} kg
(%{percent})<extra></extra>",
        )
        fig2.update_layout(
            paper_bgcolor="white",
            showlegend=True,
            legend=dict(
                orientation="v",
x=1.02,
```

```
                y=0.5,
                xanchor="left",
                font=dict(size=11),
            ),
            margin=dict(t=50, b=20, l=20, r=150),
            height=360,
        )
        st.plotly_chart(fig2, use_container_width=True)
    else:
        st.info("No category data available.")

# == BRANCH BAR CHART (Admin/Manager) ============================
if user["RoleName"] in ("Admin", "Manager"):
    st.markdown("### 🏬 Branch Comparison - This Month")
    branch_cmp = run_query(
        """SELECT b.BranchName,
                  ROUND(SUM(wl.Quantity),2)      AS Waste_kg,
                  ROUND(SUM(wl.EstimatedCost),2) AS Cost_RM
           FROM WasteLog wl
           JOIN Branch b ON b.BranchID=wl.BranchID
           WHERE wl.WasteDateTime >= DATE_FORMAT(CURDATE(),'%Y-%m-01')
           GROUP BY b.BranchName ORDER BY Waste_kg DESC"""
    )
    if branch_cmp:
        df_b = pd.DataFrame(branch_cmp)
        fig3 = px.bar(
            df_b, x="BranchName", y="Waste_kg",
            color="Waste_kg",
            color_continuous_scale=["#a8d5b5", "#27AE60", "#1a6e3c"],
            labels={"BranchName": "Branch", "Waste_kg": "Waste (kg)"},
title="Waste by Branch",
        )
        fig3.update_layout(
            plot_bgcolor="white", paper_bgcolor="white",
            coloraxis_showscale=False, margin=dict(t=40, b=10),
        )
        st.plotly_chart(fig3, use_container_width=True)

# == RECENT LOGS =================================================
st.markdown("### 📋 Recent Waste Logs")
recent = run_query(
    f"""SELECT DATE(wl.WasteDateTime) AS Date,
               i.ItemName, wc.TypeName AS Category,
               wr.ReasonText AS Reason,
wl.Quantity   AS `Qty (kg)`,
               wl.EstimatedCost AS `Cost (RM)`,
               b.BranchName, u.UserName AS `Logged By`
        FROM WasteLog wl
        JOIN Item          i  ON i.ItemID           = wl.ItemID
        JOIN WasteCategory wc ON wc.WasteCategoryID = wl.WasteCategoryID
        JOIN WasteReason   wr ON wr.ReasonID        = wl.ReasonID
        JOIN Branch        b  ON b.BranchID         = wl.BranchID
        JOIN User          u  ON u.UserID           = wl.LoggedByUserID
        WHERE 1=1 {bf}
        ORDER BY wl.WasteDateTime DESC LIMIT 15"""
)
```

```
if recent:
    df_r = pd.DataFrame(recent)
    df_r["Cost (RM)"] = df_r["Cost (RM)"].apply(lambda x: f"RM {x:.2f}")
    df_r["Qty (kg)"]  = df_r["Qty (kg)"].apply(lambda x: f"{x:.3f}")
    st.dataframe(df_r, use_container_width=True, hide_index=True)
else:
    st.info("No waste logs found.")
```

8. Waste Log (2_Waste_Log.py)

8.1 Design View

The Waste Log module is the primary data capture and review screen. It is accessible to all roles. It contains two tabs: "View Logs" for browsing and inline editing of existing records, and "Add Entry" for submitting new waste events.

Tab 1 – View Logs:

- Filter bar with From date, To date, Category filter selectbox, and a Search button.
- Three summary metrics above the table: Entries shown, Total Qty (kg), Total Cost (RM).
- st.data_editor table with inline-editable columns: Category, Reason, Qty (kg), and Notes.
- A "Delete" checkbox column (tick to mark a row for deletion).
- "Save All Changes" button: iterates all rows, applies UPDATEs for changed rows and DELETEs for ticked rows. Respects permission: Staff can only modify their own entries.
- Cost is automatically recalculated as CostPerUnit x new_qty when quantity changes.

Tab 2 – Add Entry:

- Food item selectbox OUTSIDE the form so item details (unit, cost per unit) show as a live green preview box immediately on selection.
- "Other (specify in Notes)" option for items not yet in Master Data.
- Three-column layout inside the form: left (Category + Quantity), centre (Reason + Branch), right (Date + Time).
- Blue estimated-cost preview box inside the form, updated reactively.
- On success: green success message, balloons animation, auto-navigate to View Logs tab.

8.2 Execution View

editable_ids is built as a set of WasteLogIDs that the current user may edit. Admins and Managers can edit all visible entries. Staff can only edit entries where LoggedByUserID == uid. On "Save All Changes", rows not in editable_ids increment the skipped counter and are ignored. Changed rows are detected by comparing edited DataFrame values against the original query result list.

The item selectbox is placed OUTSIDE st.form() so its value is available before the form renders. This allows the cost-per-unit info box to appear live. Inside the form, a_qty is a NumberInput with min_value=0.001. The estimated cost is computed as round(CostPerUnit * a_qty, 2) and shown in the preview box.

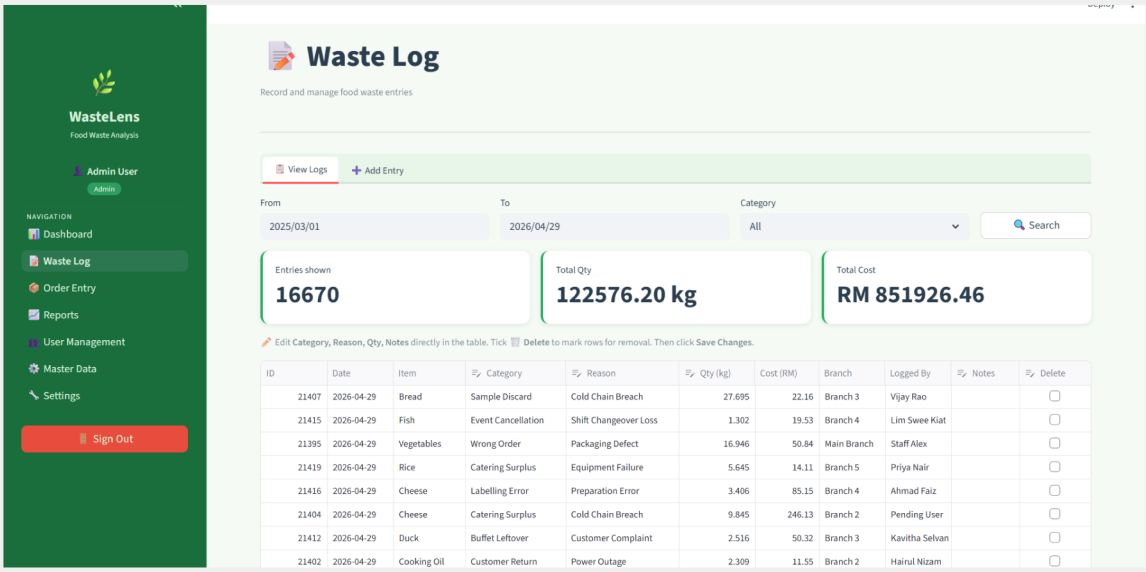
8.3 Field Validation Rules

- Date range: From and To are date_input() widgets, no server-side inversion check.
- Quantity: min_value=0.001 enforced by NumberColumn in data_editor; Add Entry form uses min_value=0.001.
- Notes (Other item): required if "Other (specify in Notes)" is selected. Error: "Please describe the item in the Notes field."
- Branch: must be selected from the user's accessible branches.

8.4 Screenshots – Waste Log

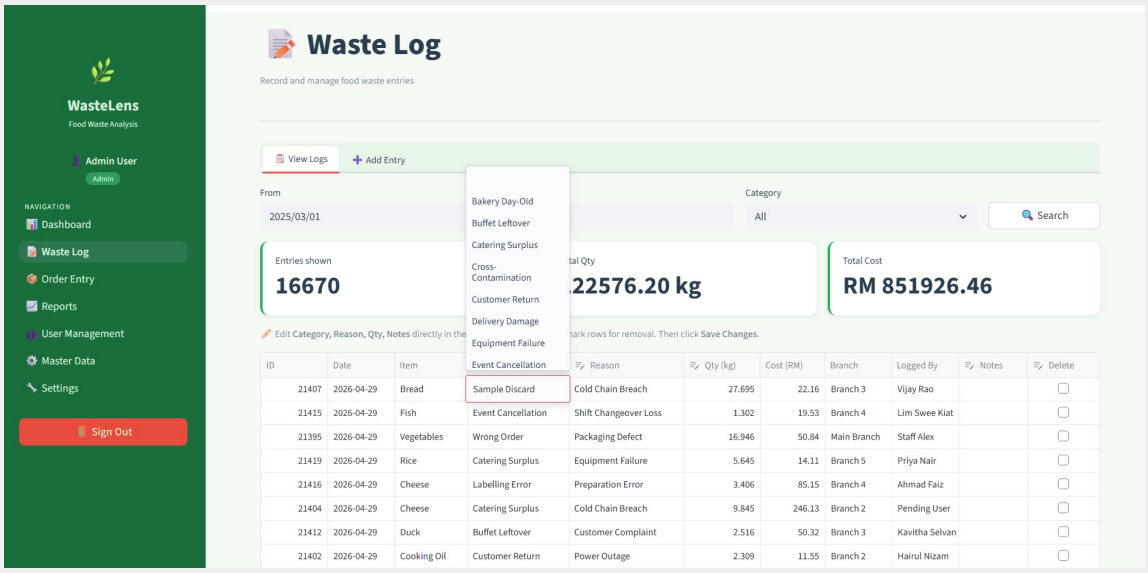
Waste Log – View Logs Tab (Default)

[SCREENSHOT — Waste Log View Logs tab – filter bar, 3 summary metrics, inline-editable data_editor table]



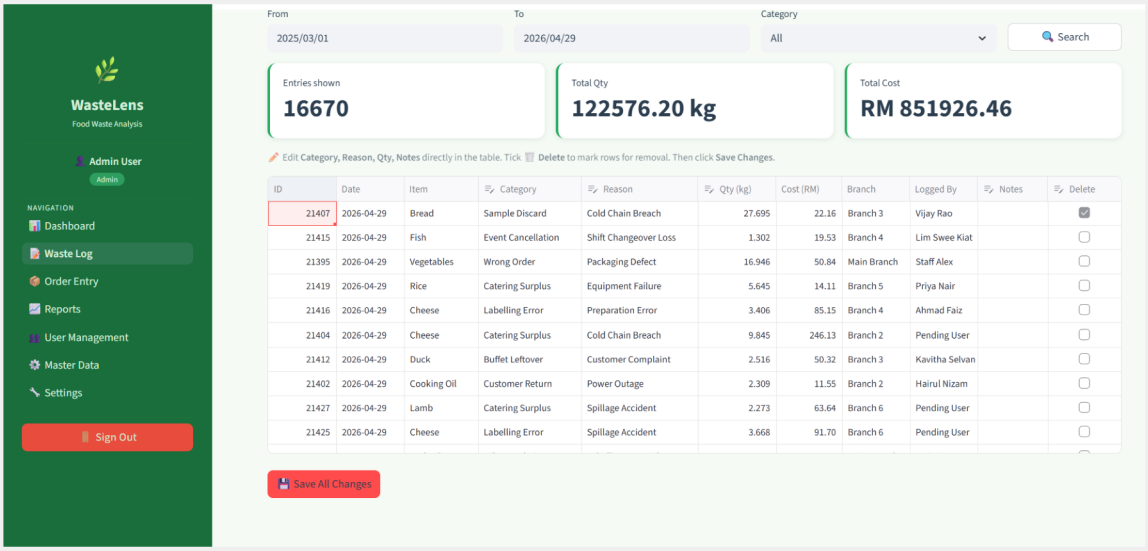
Waste Log – Inline Editing a Row

[SCREENSHOT — Waste Log – a row being edited inline (Category/Reason dropdowns, Qty changed)]



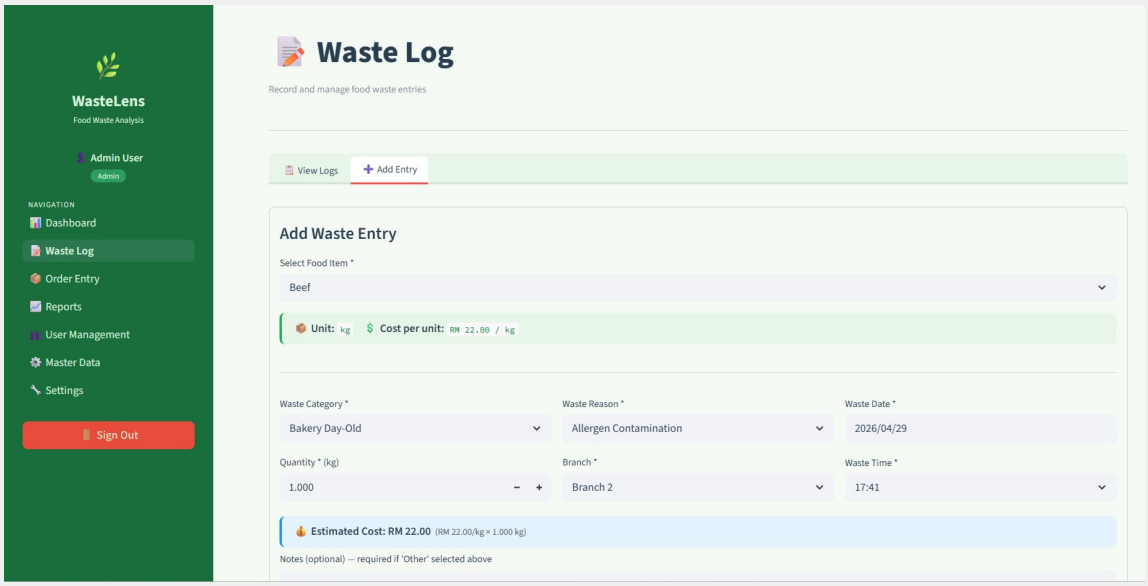
Waste Log – Delete Checkbox Marked

[SCREENSHOT — Waste Log – Delete checkbox ticked on one or more rows before clicking Save Changes]



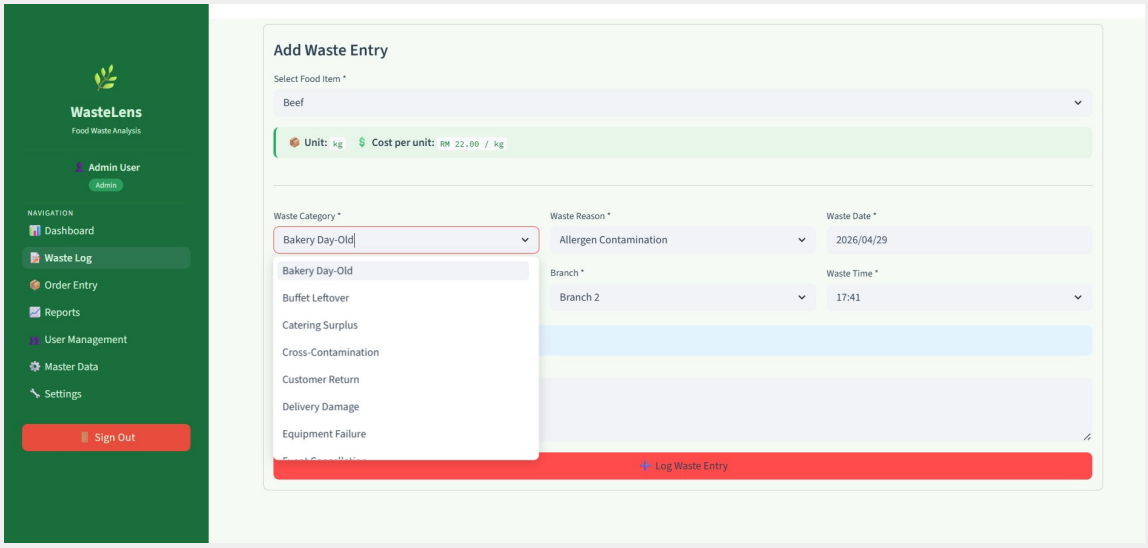
Waste Log – Save Changes Success Banner

[SCREENSHOT — Waste Log – success banner after saving: "X row(s) updated | Y row(s) deleted"]

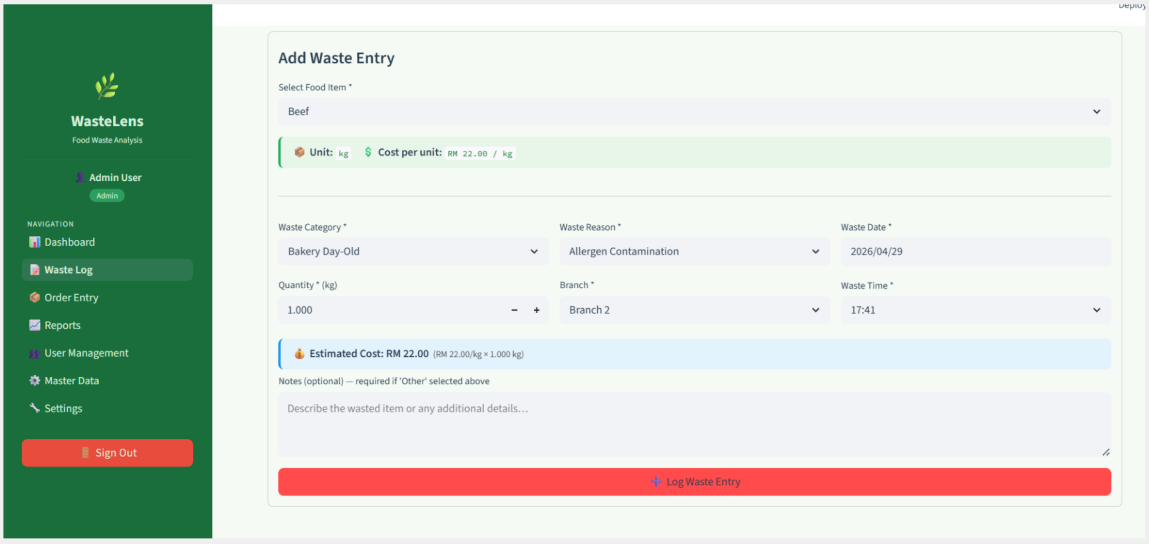


Waste Log – Add Entry Tab (Item Selected with Preview)

[SCREENSHOT — Waste Log Add Entry form – Category, Qty, Reason, Branch, Date, Time, blue cost preview box]

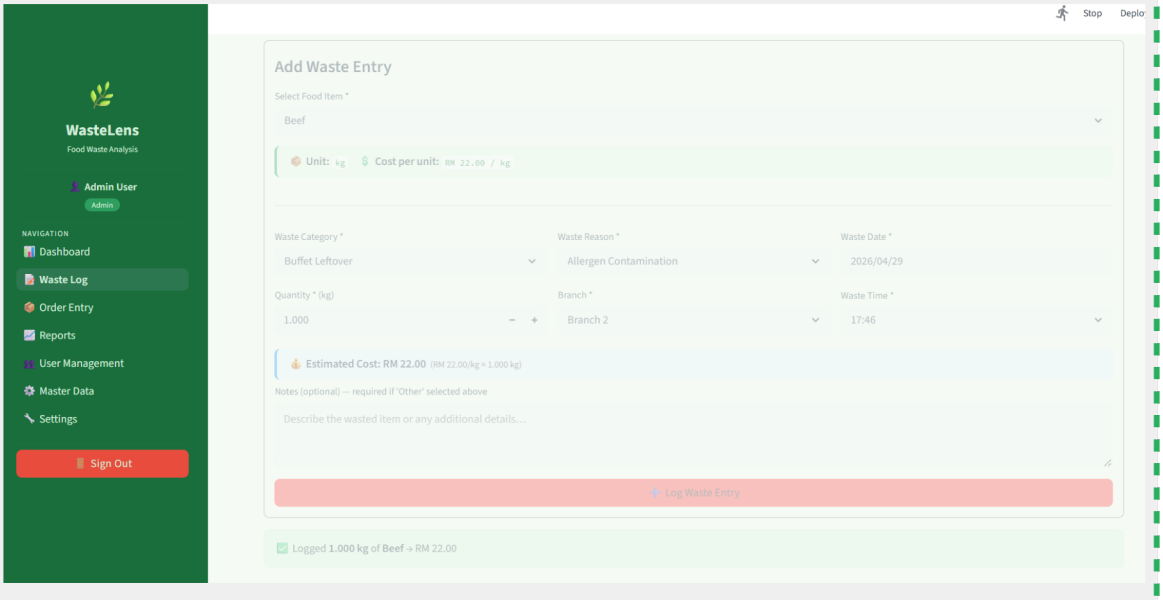


[SCREENSHOT — Waste Log Add Entry tab – item selected with green unit/cost preview box visible]



Waste Log – Add Entry Tab (Form with Estimated Cost)
Waste Log – Entry Successfully Logged

[SCREENSHOT — Waste Log – success message new waste entry is logged]



8.5 Source Code – 2_Waste_Log.py

```
# ============================================================
#  WasteLens – Waste Log (Add / View / Edit / Delete)
# ============================================================
import streamlit as st

st.set_page_config(
    page_title="Waste Log | WasteLens",
    page_icon="📝",
    layout="wide",
)

import pandas as pd
from datetime import datetime, date
from auth import require_auth, branch_filter_sql
from db import run_query, run_update
from utils import inject_css, render_sidebar

inject_css()
user = require_auth()
render_sidebar(user)

role = user["RoleName"]
uid  = user["UserID"]
bf   = branch_filter_sql(user)

st.title("📝 Waste Log")
st.caption("Record and manage food waste entries")

st.markdown("---")

# ── TABS: VIEW / ADD ──────────────────────────────────────
# after a successful add we flip to view tab automatically
_default_tab = st.session_state.pop("wl_goto_view", False)
tab_view, tab_add = st.tabs(["📋  View Logs", "➕  Add Entry"])
if _default_tab:
    # force the view tab to appear "active" via a rerun flag already consumed
    pass

# ──────────────────────────────────────────────────────────
# TAB 1 – VIEW  (with inline edit / delete per row)
# ──────────────────────────────────────────────────────────
with tab_view:
    fc1, fc2, fc3, fc4 = st.columns([2, 2, 2, 1])
with fc1:
        date_from = st.date_input("From", value=date(2025, 3, 1))
    with fc2:
        date_to   = st.date_input("To",   value=date.today())
    cats = run_query("SELECT WasteCategoryID, TypeName FROM WasteCategory ORDER BY
TypeName")
    cat_map = {"All": None} | {c["TypeName"]: c["WasteCategoryID"] for c in (cats
or
[])}
    with fc3:
```

```
        sel_cat = st.selectbox("Category", options=list(cat_map.keys()))
    with fc4:
```

```
        st.markdown("<br>", unsafe_allow_html=True)
        st.button("🔍 Search", use_container_width=True)

    cat_clause = f" AND wl.WasteCategoryID = {cat_map[sel_cat]}" if cat_map[sel_cat]
else ""

    logs = run_query(
        f"""SELECT wl.WasteLogID,
                   DATE(wl.WasteDateTime) AS Date,
                   i.ItemName, wc.TypeName AS Category,
                   wr.ReasonText AS Reason,
                   wl.Quantity   AS Qty,
                   wl.EstimatedCost AS Cost,
                   b.BranchName,
                   u.UserName    AS LoggedBy,
                   wl.Notes,
                   wl.LoggedByUserID
            FROM WasteLog wl
            JOIN Item          i  ON i.ItemID           = wl.ItemID
            JOIN WasteCategory wc ON wc.WasteCategoryID = wl.WasteCategoryID
            JOIN WasteReason   wr ON wr.ReasonID        = wl.ReasonID
            JOIN Branch        b  ON b.BranchID         = wl.BranchID
            JOIN User          u  ON u.UserID           = wl.LoggedByUserID
            WHERE DATE(wl.WasteDateTime) BETWEEN %s AND %s
              {bf} {cat_clause}
            ORDER BY wl.WasteDateTime DESC""",
        (str(date_from), str(date_to)),
    )

    if not logs:
        st.info("No waste logs found for the selected filters.")
    else:
        df = pd.DataFrame(logs)
        s1, s2, s3 = st.columns(3)
        s1.metric("Entries shown", len(df))
        s2.metric("Total Qty",     f"{float(df['Qty'].sum()):.2f} kg")
        s3.metric("Total Cost",    f"RM {float(df['Cost'].sum()):.2f}")

        # decide which rows this user can edit
        editable_ids = {
            r["WasteLogID"]
            for r in logs
            if role in ("Admin", "Manager") or r["LoggedByUserID"] == uid
        }

        # ── fetch option lists for dropdowns ──────────────────────
        cats_e    = run_query("SELECT WasteCategoryID, TypeName FROM WasteCategory
ORDER BY TypeName")
        reasons_e = run_query("SELECT ReasonID, ReasonText FROM WasteReason ORDER
BY
ReasonText")
        items_e   = run_query("SELECT ItemID, ItemName, Unit, CostPerUnit FROM
Item
ORDER BY ItemName")
```

```
        cat_opts    = [c["TypeName"]    for c in (cats_e    or [])]
        reason_opts = [r["ReasonText"]  for r in (reasons_e or [])]
        item_map_s  = {i["ItemName"]: i for i in (items_e   or [])}
        cat_map_s   = {c["TypeName"]: c["WasteCategoryID"] for c in (cats_e    or
[])}
```

```
        reason_map_s= {r["ReasonText"]: r["ReasonID"]      for r in (reasons_e or
[])}

        # ── build editable dataframe ──────────────────────────
        edit_df = pd.DataFrame({
            "ID":        [r["WasteLogID"]         for r in logs],
            "Date":      [str(r["Date"])           for r in logs],
            "Item":      [r["ItemName"]            for r in logs],
            "Category":  [r["Category"]            for r in logs],
            "Reason":    [r["Reason"]              for r in logs],
            "Qty (kg)":  [float(r["Qty"])          for r in logs],
            "Cost (RM)": [float(r["Cost"])         for r in logs],
            "Branch":    [r["BranchName"]          for r in logs],
            "Logged By": [r["LoggedBy"]            for r in logs],
            "Notes":     [r["Notes"] or ""         for r in logs],
            "🗑 Delete": [False                    for _  in logs],
        })

        st.caption("✏️ Edit **Category, Reason, Qty, Notes** directly in the table.
"
                   "Tick **🗑 Delete** to mark rows for removal. Then click **Save
Changes**.")

        edited_df = st.data_editor(
            edit_df,
            column_config={
                "ID":        st.column_config.NumberColumn("ID",
disabled=True, width="small"),
                "Date":      st.column_config.TextColumn("Date",
disabled=True, width="small"),
                "Item":      st.column_config.TextColumn("Item",
disabled=True),
                "Category":  st.column_config.SelectboxColumn("Category",
options=cat_opts),
                "Reason":    st.column_config.SelectboxColumn("Reason",
options=reason_opts),
                "Qty (kg)":  st.column_config.NumberColumn("Qty (kg)",
min_value=0.001, format="%.3f"),
                "Cost (RM)": st.column_config.NumberColumn("Cost (RM)",
disabled=True, format="%.2f"),
                "Branch":    st.column_config.TextColumn("Branch",
disabled=True, width="small"),
                "Logged By": st.column_config.TextColumn("Logged By",
disabled=True, width="small"),
                "Notes":     st.column_config.TextColumn("Notes"),
                "🗑        Delete":       st.column_config.CheckboxColumn("Delete",
default=False,
width="small"),
            },
```

```
            disabled=[
                c for c in edit_df.columns
                if c not in ("Category", "Reason", "Qty (kg)", "Notes", "
Delete")
            ],
            hide_index=True,
            use_container_width=True,
            num_rows="fixed",
            key="waste_log_editor",
        )
```

```
        if st.button(" Save All Changes", type="primary",
use_container_width=False):
            saved = deleted = skipped = 0
            for _, row in edited_df.iterrows():
                log_id = int(row["ID"])
                # only allow edit/delete if user has permission
                if log_id not in editable_ids:
                    skipped += 1
                    continue

                if row[" Delete"]:
                    run_update("DELETE   FROM   WasteLog   WHERE   WasteLogID=%s",
(log_id,))
                    deleted += 1
                    continue

                # find original row to detect changes
                orig = next((r for r in logs if r["WasteLogID"] == log_id), None)
                if not orig:
                    continue

                new_qty    = float(row["Qty (kg)"])
new_cat    = row["Category"]
                new_reason = row["Reason"]
                new_notes  = row["Notes"].strip() or None

                changed = (
                    abs(new_qty - float(orig["Qty"])) > 0.0001
                    or new_cat    != orig["Category"]
                    or new_reason != orig["Reason"]
                    or new_notes  != (orig["Notes"] or None)
                )
                if changed:
                    idata    = item_map_s.get(row["Item"])
                    new_cost = round(float(idata["CostPerUnit"]) * new_qty, 2) if
idata else float(orig["Cost"])
run_update(
                        """UPDATE WasteLog
                           SET WasteCategoryID=%s, ReasonID=%s,
                               Quantity=%s, EstimatedCost=%s, Notes=%s
                           WHERE WasteLogID=%s""",
                        (cat_map_s.get(new_cat),
                         reason_map_s.get(new_reason),
                         new_qty, new_cost, new_notes, log_id),
```

```
                    )
                    saved += 1

            parts = []
            if saved:   parts.append(f"✅ {saved} row(s) updated")
if deleted: parts.append(f"🗑️ {deleted} row(s) deleted")
            if skipped: parts.append(f"⚠️ {skipped} row(s) skipped (no
permission)")
            if parts:
                st.success("  ·  ".join(parts))
                st.rerun()
            else:
                st.info("No changes detected.")

# ═══════════════════════════════════════════════════════════════
# TAB 2 - ADD NEW ENTRY  (all fields in one aligned container)
```

```
# ═══════════════════════════════════════════════════════════════
with tab_add:
    items_add   = run_query("SELECT ItemID, ItemName, Unit, CostPerUnit FROM Item
ORDER BY ItemName")
    cats_add    = run_query("SELECT WasteCategoryID, TypeName FROM WasteCategory
ORDER BY TypeName")
    reasons_add = run_query("SELECT ReasonID, ReasonText FROM WasteReason ORDER BY
ReasonText")

    if role == "Admin":
        branches_add = run_query(
            "SELECT BranchID, BranchName FROM Branch WHERE IsActive=1 ORDER BY
BranchName"
        )
    else:
        ids_str = ",".join(str(i) for i in user["branch_ids"]) or "0"
        branches_add = run_query(
            f"SELECT BranchID, BranchName FROM Branch WHERE IsActive=1 AND
BranchID
IN ({ids_str}) ORDER BY BranchName"
        )

    # "Other" option lets staff specify a non-listed item in the Notes field
    item_names   = [i["ItemName"] for i in (items_add or [])] + ["Other (specify
in
Notes)"]
    item_map_add = {i["ItemName"]: i for i in (items_add or [])}
    cat_map_add  = {c["TypeName"]: c["WasteCategoryID"] for c in (cats_add or [])}
    reason_map_add = {r["ReasonText"]: r["ReasonID"] for r in (reasons_add or [])}
    branch_map_add = {b["BranchName"]: b["BranchID"] for b in (branches_add or
[])}

    # ── item selector OUTSIDE form for live preview ──────────────
    st.markdown("**Select Food Item \\***")
    a_item = st.selectbox("Food Item", item_names,
                          label_visibility="collapsed", key="wl_item")

    is_other = (a_item == "Other (specify in Notes)")

    if not is_other and a_item in item_map_add:
```

```
        it = item_map_add[a_item]
        st.markdown(
            f"""<div     style='background:#e8f5e9;border-radius:8px;padding:0.6rem
1rem;
                margin-bottom:0.5rem;border-left:4px solid #27AE60'>
                <b>📦 Unit:</b> <code>{it['Unit']}</code>
                  
                <b>💲 Cost per unit:</b> <code>RM {float(it['CostPerUnit']):.2f} /
{it['Unit']}</code>
            </div>""",
            unsafe_allow_html=True,
        )     elif
is_other:
        st.info("ℹ️ 'Other' selected — describe the item in the Notes field below.
"
                "Cost will be calculated as RM 0.00. Ask your Admin to add it to
Master Data later.")

    # ── all form fields in ONE unified box ──────────────────────
    with st.form("add_waste_form", clear_on_submit=True):
        st.markdown(f"**Selected item:** `{a_item}`")
        st.markdown("---")
```

```
        col1, col2, col3 = st.columns(3)

        with col1:
            a_cat  = st.selectbox("Waste Category *", list(cat_map_add.keys()))
            unit_lbl = item_map_add[a_item]["Unit"] if (a_item in item_map_add) else
"unit"
            a_qty        =    st.number_input(
f"Quantity * ({unit_lbl})",
                min_value=0.001, value=1.0, step=0.1, format="%.3f",
            )

        with col2:
            a_reason = st.selectbox("Waste Reason *", list(reason_map_add.keys()))
            a_branch = st.selectbox("Branch *", list(branch_map_add.keys()))

        with col3:
            a_date = st.date_input("Waste Date *", value=date.today())
            a_time = st.time_input("Waste Time *", value=datetime.now().time())

        # cost preview
        if not is_other and a_item in item_map_add:
            cpu      = float(item_map_add[a_item]["CostPerUnit"])
            est_cost = round(cpu * a_qty, 2)
            st.markdown(
                f"""<div style='background:#e3f2fd;border-
radius:8px;padding:0.6rem
1rem;
                    border-left:4px solid #2196F3;margin:0.3rem 0'>
                    <b>💰 Estimated Cost: RM {est_cost:.2f}</b>
                    <span style='color:#555;font-size:0.82rem'>
                     (RM {cpu:.2f}/{unit_lbl} × {a_qty:.3f} {unit_lbl})
                    </span>
                </div>""",
```

```
                unsafe_allow_html=True,
            )
        else:
            est_cost = 0.0

        a_notes = st.text_area("Notes (optional) — required if 'Other' selected
above",
                               placeholder="Describe the wasted item or any
additional details…")

        add_sub = st.form_submit_button("➕ Log Waste Entry",
use_container_width=True)

    if add_sub:
        if is_other and not a_notes.strip():
            st.error("Please describe the item in the Notes field when 'Other' is
selected.")
        elif not a_branch:
            st.error("Please select a branch.")
        else:
            if is_other:
                # Use the first item in DB as placeholder (or a dedicated "Other"
item if it exists)
                other_item = run_query(
                    "SELECT ItemID, CostPerUnit, Unit FROM Item WHERE
ItemName='Other' LIMIT 1"
                )
                if not other_item:
                    # fallback: use first item in list
                    other_item = run_query("SELECT ItemID, CostPerUnit, Unit FROM
Item LIMIT 1")
                item_id  = other_item[0]["ItemID"]
                est_cost = 0.0
else:
                item_id  = item_map_add[a_item]["ItemID"]
                cpu      = float(item_map_add[a_item]["CostPerUnit"])
                est_cost = round(cpu * a_qty, 2)

            waste_dt = datetime.combine(a_date, a_time)
            run_update(
                """INSERT INTO WasteLog
                   (WasteDateTime, Quantity, EstimatedCost, Notes,
                    LoggedByUserID, BranchID, WasteCategoryID, ReasonID, ItemID)
                   VALUES (%s, %s, %s, %s, %s, %s, %s, %s, %s)""",
                (waste_dt, a_qty, est_cost, a_notes or None,
                 uid, branch_map_add[a_branch], cat_map_add[a_cat],
                 reason_map_add[a_reason], item_id),
            )
            label = a_item if not is_other else f"Other ({a_notes[:30]})"
            st.success(f"✅ Logged **{a_qty:.3f} {unit_lbl}** of **{label}** → RM
{est_cost:.2f}")
            st.balloons()
            st.session_state["wl_goto_view"] = True
            st.rerun()   # rerun → lands on View Logs tab
```

9. Order Entry (3_Order_Entry.py)

9.1 Design View

The Order Entry module records daily customer order counts per branch. It is accessible to all roles. Data is stored in the OrderEntry table with a UNIQUE constraint on (BranchID, OrderDate), preventing duplicate entries. The module has three tabs.

Tab 1 – Record Orders:

- Branch selectbox and Date input are OUTSIDE the form for live duplicate detection.
- A live preview check runs on every interaction: green success box if no record exists for that branch+date, amber warning box if an existing record is found (shows existing count and warns that submitting will UPDATE it).
- Inside the form: Order Count (integer, min 1) and optional Notes textarea.
- INSERT … ON DUPLICATE KEY UPDATE ensures idempotent upsert behaviour.

Tab 2 – Order History:

- Date range + branch filter bar.
- Three summary metrics: Records count, Total Orders, Daily Average.
- Scrollable data table.
- When more than 1 record exists: a trend chart with green bars (daily orders) and a dashed green line (7-day moving average).
- Admin/Manager only: Edit / Delete expander with inline form and delete button per entry.

Tab 3 – Today's Summary:

- Shows all active branches for today's date.
- Colour-highlights rows in amber where the Status is "Missing" (no order recorded yet).
- Four KPIs: Total Orders Today, Branches Recorded, Branches Missing, Avg Orders/Branch.
- Warning message if any branches are missing today's count.
- Waste-per-Order Ratio bar chart (current month data from vw_DailyBranchSummary view).

9.2 Execution View

The branch and date pickers are outside st.form() intentionally. Streamlit re-runs the entire script on each widget interaction, so the duplicate-check query fires automatically when the user changes branch or date – before they even submit. This gives immediate feedback without needing a separate "Check" button.

The waste-per-order ratio chart queries the vw_DailyBranchSummary database view. If the view returns no data (e.g., no orders recorded this month), st.info() is shown instead of a blank chart.

9.3 Field Validation Rules

- Branch: must be selected from accessible branches (scoped by role).
- Order Count: min_value=0 in the widget; server-side check for > 0 before saving.

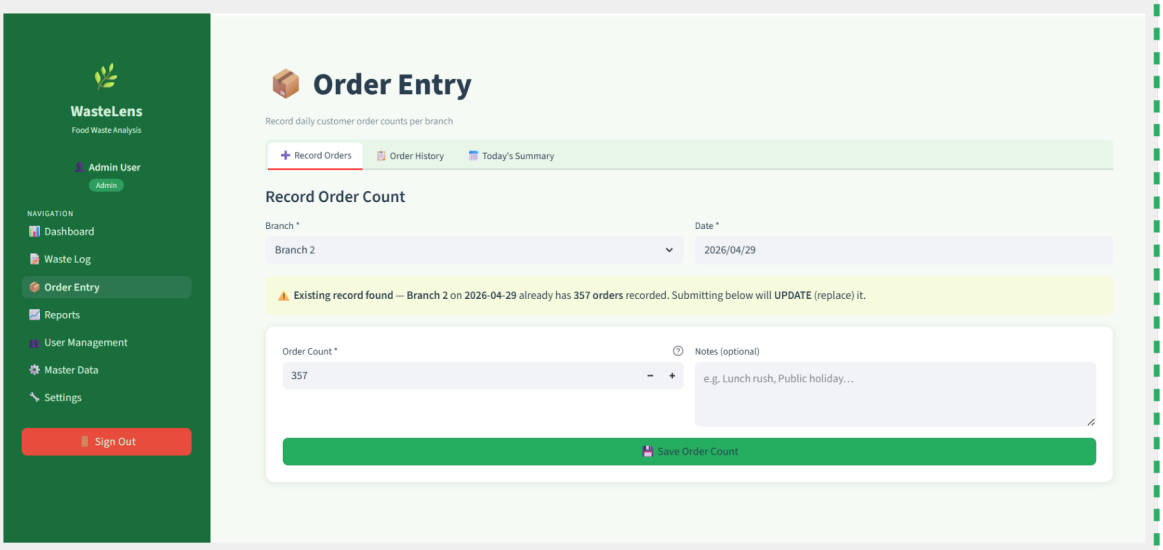
Error: "Order count must be greater than 0."

- Date: date_input() widget, must be a valid calendar date.
- Duplicate: ON DUPLICATE KEY UPDATE handles the upsert transparently; the amber preview warning informs the user before submission.

9.4 Screenshots – Order Entry

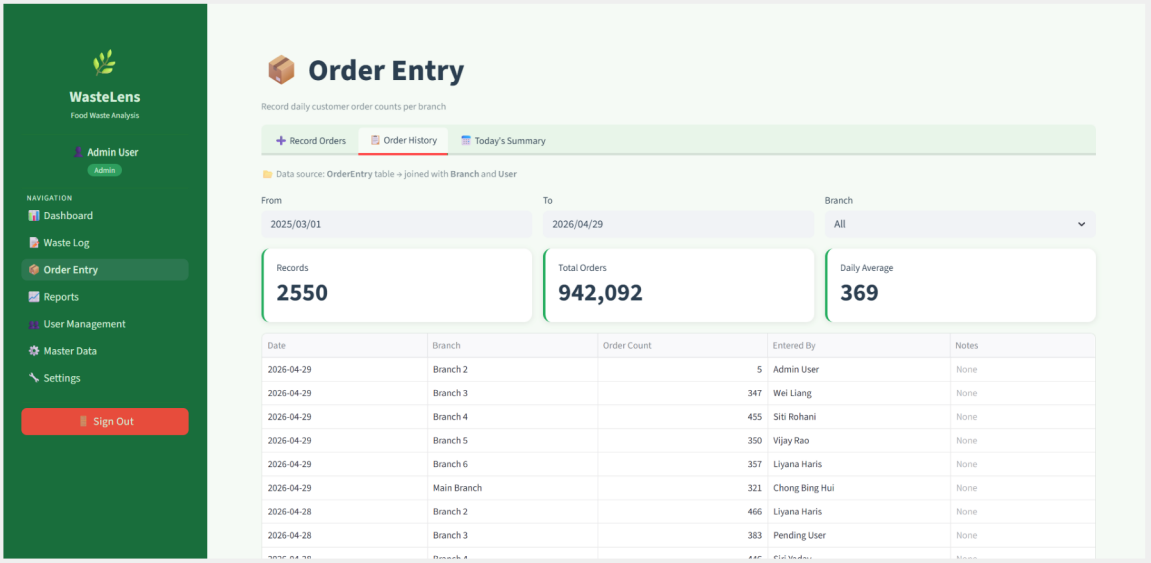
Order Entry – Record Orders Tab (New Record)

[SCREENSHOT — Order Entry – Record Orders tab showing green "no record yet" confirmation for selected branch+date]



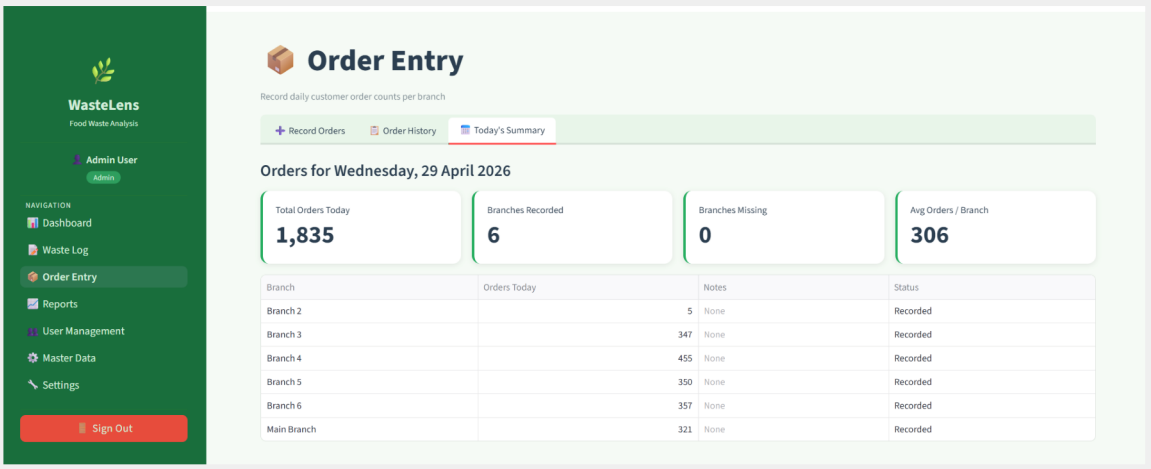
Order Entry – Order History Tab

[SCREENSHOT — Order Entry – Order History tab with summary metrics, data table, and trend bar+line chart]



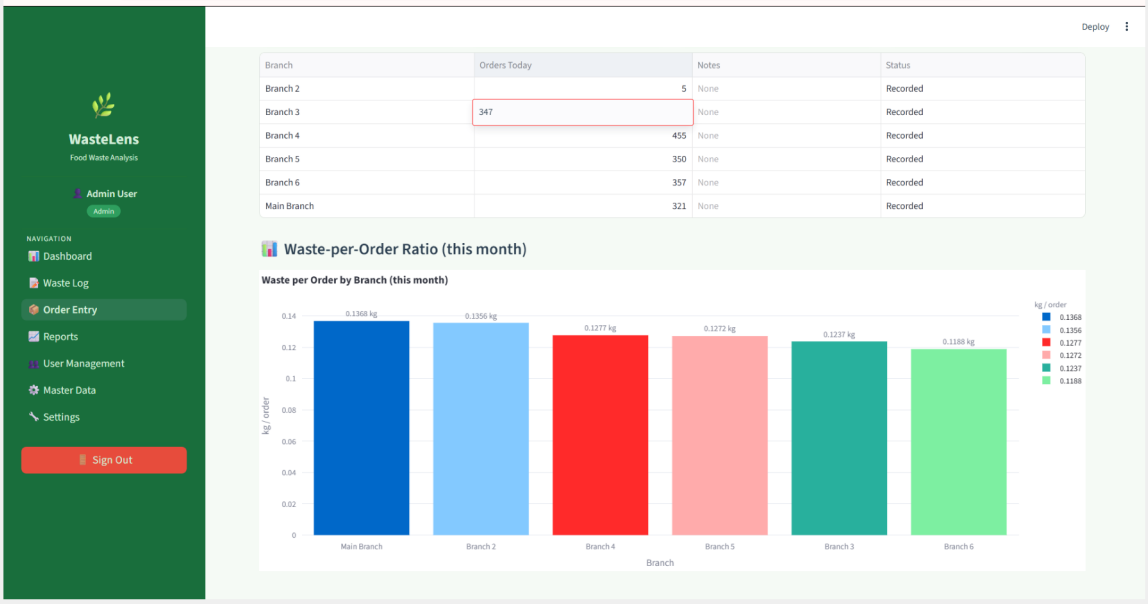
Order Entry – Edit/Delete Expander (Admin/Manager)

[SCREENSHOT — Order Entry – Edit expander open showing editable order count and notes with Save/Delete buttons]



Order Entry – Today's Summary Tab

[SCREENSHOT — Order Entry – Today's Summary tab with KPI row, branch status table (Missing rows highlighted amber), and waste-per-order chart]



9.5 Source Code – 3_Order_Entry.py

```
# ================================================================
#  WasteLens – Order Entry  (fixes: upsert preview + logout)
# ================================================================
import streamlit as st

st.set_page_config(
    page_title="Order Entry | WasteLens",
page_icon="📦",
    layout="wide",
)

import pandas as pd
import plotly.express as px
from datetime import date
from auth import require_auth
from db import run_query, run_update
from utils import inject_css, render_sidebar

inject_css()
user = require_auth()
render_sidebar(user)          # ← sidebar already has Sign-Out button

role = user["RoleName"]
uid  = user["UserID"]

st.title("📦 Order Entry")
```

```
st.caption("Record daily customer order counts per branch")


# ── branch access ──────────────────────────────────────────
if role == "Admin":
    branches = run_query(
        "SELECT BranchID, BranchName FROM Branch WHERE IsActive=1 ORDER BY
BranchName"
    )
else:    ids_str  = ",".join(str(i) for i in
user["branch_ids"]) or "0"
    branches = run_query(
        f"SELECT BranchID, BranchName FROM Branch"
        f" WHERE IsActive=1 AND BranchID IN ({ids_str}) ORDER BY BranchName"
    )
branch_map = {b["BranchName"]: b["BranchID"] for b in (branches or [])}

# ═══ TABS ═══════════════════════════════════════════════════
tab_add, tab_view, tab_today = st.tabs(
    ["➕  Record Orders", "📋  Order History", "📅  Today's Summary"]
)

# ──────────────────────────────────────────────────────────────
# TAB 1 - RECORD ORDERS
# Branch + date are OUTSIDE the form so we can preview existing record
# ──────────────────────────────────────────────────────────────
with tab_add:
    st.markdown("#### Record Order Count")

    # ── selectors OUTSIDE the form (dynamic preview) ──────────
    sel1, sel2 = st.columns(2)
    with sel1:
        o_branch    =    st.selectbox("Branch    *",    list(branch_map.keys()),
key="oe_branch")
    with sel2:
        o_date = st.date_input("Date *", value=date.today(), key="oe_date")

    # ── live preview: check if record already exists ──────────
    existing = run_query(
        "SELECT OrderID, OrderCount, Notes FROM OrderEntry "
        "WHERE BranchID=%s AND OrderDate=%s",
        (branch_map[o_branch], str(o_date)),
    )

    if existing:
        ex = existing[0]
        st.warning(
            f"⚠️  **Existing record found** — "
            f"**{o_branch}** on **{o_date}** already has "
            f"**{ex['OrderCount']:,} orders** recorded.  "
            f"Submitting below will **UPDATE** (replace) it."
        )
        default_count = int(ex["OrderCount"])
        default_notes = ex["Notes"] or ""
    else:
        st.success(f"✅  No record yet for **{o_branch}** on **{o_date}** — will
```

```
create new entry.")
        default_count = 0
        default_notes = ""

    # ── form (only count + notes inside) ──────────────────────
    with st.form("order_entry_form", clear_on_submit=True):
```

```
        fc1, fc2 = st.columns(2)
        with fc1:
            o_count = st.number_input(
                "Order Count *",
                min_value=0,
                value=default_count,
                step=1,
                help="Total customer orders served this day",
            )
        with fc2:
            o_notes = st.text_area(
                "Notes (optional)",
                value=default_notes,
                placeholder="e.g. Lunch rush, Public holiday…",
            )
        sub = st.form_submit_button("💾 Save Order Count",
use_container_width=True)

    if sub:
        if o_count <= 0:
            st.error("Order count must be greater than 0.")
        else:
            action = "Updated" if existing else "Saved"
            run_update(
                """INSERT INTO OrderEntry
                        (OrderDate, OrderCount, Notes, EnteredByUserID, BranchID)
                   VALUES (%s, %s, %s, %s, %s)
                   ON DUPLICATE KEY UPDATE
                       OrderCount       = VALUES(OrderCount),
                       Notes            = VALUES(Notes),
                       EnteredByUserID  = VALUES(EnteredByUserID)""",
                (str(o_date), o_count, o_notes or None, uid,
branch_map[o_branch]),
            )
            st.success(
                f"✅ {action} — **{o_branch}** | **{o_date}** | **{o_count:,}
orders**"
            )
            st.rerun()   # refresh so the preview updates immediately

# ──────────────────────────────────────────────────────────────
# TAB 2 – ORDER HISTORY
# Source: OrderEntry table (JOIN Branch + User)
# ──────────────────────────────────────────────────────────────
with tab_view:
    st.caption("📂 Data source: **OrderEntry** table → joined with **Branch** and
**User**")

    hf1, hf2, hf3 = st.columns([2, 2, 2])
```

```
    with hf1:
        h_from  = st.date_input("From", value=date(2025, 3, 1), key="h_from")
    with hf2:
        h_to    = st.date_input("To",   value=date.today(),     key="h_to")
    with hf3:
        h_branch = st.selectbox("Branch", ["All"] + list(branch_map.keys()),
key="h_branch")

    br_clause = ""
    if h_branch != "All" and h_branch in branch_map:
        br_clause = f" AND oe.BranchID = {branch_map[h_branch]}"
```

```
    elif role != "Admin" and user["branch_ids"]:
        ids_str = ",".join(str(i) for i in user["branch_ids"])
        br_clause = f" AND oe.BranchID IN ({ids_str})"

    history = run_query(
        f"""SELECT oe.OrderID,
                   oe.OrderDate         AS Date,
                   b.BranchName         AS Branch,
                   oe.OrderCount        AS `Order Count`,
                   u.UserName           AS `Entered By`,
                   oe.Notes
            FROM   OrderEntry oe
            JOIN   Branch b ON b.BranchID = oe.BranchID
            JOIN   User   u ON u.UserID   = oe.EnteredByUserID
            WHERE  oe.OrderDate BETWEEN %s AND %s {br_clause}
            ORDER  BY oe.OrderDate DESC, b.BranchName""",
        (str(h_from), str(h_to)),
    )

    if not history:
        st.info("No order records found for the selected filters.")
else:
        df_h = pd.DataFrame(history)
        s1, s2, s3 = st.columns(3)
        s1.metric("Records",       len(df_h))
        s2.metric("Total Orders",  f"{df_h['Order Count'].sum():,}")
        s3.metric("Daily Average", f"{df_h['Order Count'].mean():,.0f}")

        st.dataframe(
            df_h.drop(columns=["OrderID"]),
            use_container_width=True,
            hide_index=True,
        )

        # trend chart - bar + line combo
if len(df_h) > 1:
            import plotly.graph_objects as go
            df_chart = df_h.groupby("Date")["Order Count"].sum().reset_index()
            df_chart["MA7"] = (
                df_chart["Order Count"].rolling(7, min_periods=1).mean().round(0)
            )
            fig = go.Figure()
            fig.add_trace(go.Bar(
                x=df_chart["Date"], y=df_chart["Order Count"],
```

```
                name="Daily Orders",
                marker_color="#a8d5b5",
            ))
            fig.add_trace(go.Scatter(
                x=df_chart["Date"], y=df_chart["MA7"],
                name="7-Day Avg",
                mode="lines+markers",
                line=dict(color="#27AE60", width=3),
                marker=dict(size=5),
            ))
            fig.update_layout(
                title="Order Count Trend (Bar) + 7-Day Moving Average",
                xaxis_title="Date", yaxis_title="Orders",
                plot_bgcolor="white", paper_bgcolor="white",
```

```
                legend=dict(orientation="h", y=-0.2),
                hovermode="x unified",
                margin=dict(t=40, b=10),
            )
            st.plotly_chart(fig, use_container_width=True)

        # Edit / Delete (Admin / Manager)
if role in ("Admin", "Manager"):
            st.markdown("#### ✏️ Edit / Delete an Order Entry")
            ord_labels = {
                f"#{r['OrderID']} – {r['Date']} – {r['Branch']} ({r['Order
Count']}
orders)": r["OrderID"]
                for r in history
            }
            sel_ord = st.selectbox("Select entry", list(ord_labels.keys()),
key="sel_ord")
            sel_oid = ord_labels[sel_ord]
            sel_row = next(r for r in history if r["OrderID"] == sel_oid)

            with st.expander("Edit this entry"):
                with st.form(f"edit_ord_{sel_oid}"):
                    new_count = st.number_input("Order Count", min_value=0,
                                                value=int(sel_row["Order Count"]))
                    new_notes = st.text_area("Notes", value=sel_row["Notes"] or
"")
                    save_ord  = st.form_submit_button("💾 Save")
                if save_ord:
                    run_update(
                        "UPDATE OrderEntry SET OrderCount=%s, Notes=%s WHERE
OrderID=%s",
                        (new_count, new_notes or None, sel_oid),
                    )
                    st.success("✅ Updated.")
                    st.rerun()

            if st.button(f"🗑️ Delete order #{sel_oid}", type="primary"):
                run_update("DELETE FROM OrderEntry WHERE OrderID=%s", (sel_oid,))
                st.success("Deleted.")
                st.rerun()

# ──────────────────────────────────────────────────────────
```

```
# TAB 3 – TODAY'S SUMMARY
# ──────────────────────────────────────────────────────────
with tab_today:
    st.markdown(f"#### Orders for **{date.today().strftime('%A, %d %B %Y')}**")

    br_today_clause = ""
    if role != "Admin" and user["branch_ids"]:
        ids_str = ",".join(str(i) for i in user["branch_ids"])
br_today_clause = f" AND b.BranchID IN ({ids_str})"

    today_data = run_query(
        f"""SELECT b.BranchName                    AS Branch,
                   COALESCE(oe.OrderCount, 0)      AS `Orders Today`,
                   oe.Notes,
                   CASE WHEN oe.OrderID IS NOT NULL THEN 'Recorded'
                        ELSE 'Missing' END         AS Status
            FROM   Branch b
            LEFT JOIN OrderEntry oe
                   ON oe.BranchID  = b.BranchID
                  AND oe.OrderDate = CURDATE()
            WHERE  b.IsActive = 1 {br_today_clause}
            ORDER  BY b.BranchName"""
    )

    if today_data:
        df_td       = pd.DataFrame(today_data)
        total_today = df_td["Orders Today"].sum()
        recorded    = (df_td["Status"] == "Recorded").sum()
        missing     = (df_td["Status"] == "Missing").sum()

        col_a, col_b, col_c, col_d = st.columns(4)
        col_a.metric("Total Orders Today",  f"{total_today:,}")
        col_b.metric("Branches Recorded",   f"{recorded}")
        col_c.metric("Branches Missing",    f"{missing}")
        col_d.metric("Avg Orders / Branch",
                     f"{total_today / max(recorded, 1):,.0f}")

        def highlight_missing(row):
            if row["Status"] == "Missing":
                return ["background-color:#fff3cd"] * len(row)
            return [""] * len(row)

        st.dataframe(
            df_td.style.apply(highlight_missing, axis=1),
            use_container_width=True,
            hide_index=True,
        )

        if missing > 0:
            st.warning(f"⚠️  **{missing}** branch(es) have not recorded orders
today.
"
                       "Go to ➕ Record Orders to add them.")

    # Waste-per-Order ratio chart
```

```
st.markdown("#### 📊 Waste-per-Order Ratio (this month)")
ratio_data = run_query(
    """SELECT b.BranchName,
              ROUND(SUM(ds.TotalWaste_kg),2)  AS Waste_kg,
              SUM(ds.OrderCount)               AS Orders,
              ROUND(SUM(ds.TotalWaste_kg)/NULLIF(SUM(ds.OrderCount),0),4)
                  AS WastePerOrder_kg
       FROM   vw_DailyBranchSummary ds
       JOIN   Branch b ON b.BranchID = ds.BranchID
       WHERE  ds.SummaryDate >= DATE_FORMAT(CURDATE(),'%Y-%m-01')
       GROUP  BY b.BranchName
       HAVING Orders > 0
       ORDER  BY WastePerOrder_kg DESC"""
)
if ratio_data:
    df_ratio = pd.DataFrame(ratio_data)
    fig2 = px.bar(
        df_ratio, x="BranchName", y="WastePerOrder_kg",
        color="WastePerOrder_kg",
        color_continuous_scale=["#a8d5b5", "#27AE60", "#1a6e3c"],
        labels={"BranchName": "Branch", "WastePerOrder_kg": "kg / order"},
        title="Waste per Order by Branch (this month)",
        text="WastePerOrder_kg",
    )
    fig2.update_traces(texttemplate="%{text:.4f} kg", textposition="outside")
    fig2.update_layout(
        plot_bgcolor="white", paper_bgcolor="white",
        coloraxis_showscale=False, margin=dict(t=40, b=10),
    )
    st.plotly_chart(fig2, use_container_width=True)
else:
    st.info("No data yet for waste-per-order ratio this month.")
```

10. Reports & Analytics (4_Reports.py)

10.1 Design View

The Reports module is restricted to Admin and Manager. It provides a comprehensive analytics interface with a global filter bar, eight KPI summary cards, and five analytical tabs covering items, categories, branches, and data export.

Global Filters (always visible):

- From date, To date, Branch filter (Admin can select a specific branch; Manager sees their branches via bf), Primary Metric radio button (Quantity kg or Cost RM).

8 KPI Cards (2 rows of 4):

- Row 1: Total Entries, Total Waste (kg), Total Cost (RM), Branches Active
- Row 2: Avg Qty per Entry, Avg Cost per Entry, Unique Items, Categories Used

Tab 1 – Overview:

- Top-10 items by cost or quantity: horizontal colour-gradient bar chart.
- Top reasons by metric: donut pie chart with inside percent labels and horizontal legend.
- Item Quantity vs Cost: bubble scatter plot where bubble size = number of entries.

Tab 2 – Items:

- Items summary table (all items, entries count, totals, averages).
- Treemap: each item is a tile sized by its waste quantity or cost.
- Waste Volume Funnel: top 10 items shown as a funnel chart by quantity.

Tab 3 – Categories:

- Category share donut chart with inside percent labels.
- Category summary table.
- Stacked bar chart: waste by branch, each bar segment = a waste category.

Tab 4 – Branches:

- Grouped bar chart with dual y-axis: green bars for quantity, light-green bars for cost on secondary axis.
- Branch radar chart (spider chart) normalised 0-100: shown only when 3+ branches have data.
- Branch summary table.

Tab 5 – Export:

- Preview of first 25 rows of the filtered waste log.
- Download Full Waste Log CSV button.
- Download Branch Summary CSV button.

10.2 Execution View

All queries use named MySQL parameters %(fd)s and %(td)s via the params dict. The y_col variable ("Qty_kg" or "Cost_RM") is derived from the metric radio button and used in every query ORDER BY, GROUP BY context, and chart axis label. The radar chart normalises each metric column to 0-100 by dividing by the column maximum, allowing disparate metrics (entries, kg, RM) to appear on the same 5-axis spider chart. pd.to_numeric() is applied before normalisation because MySQL Decimal columns return as Python Decimal objects, not float.

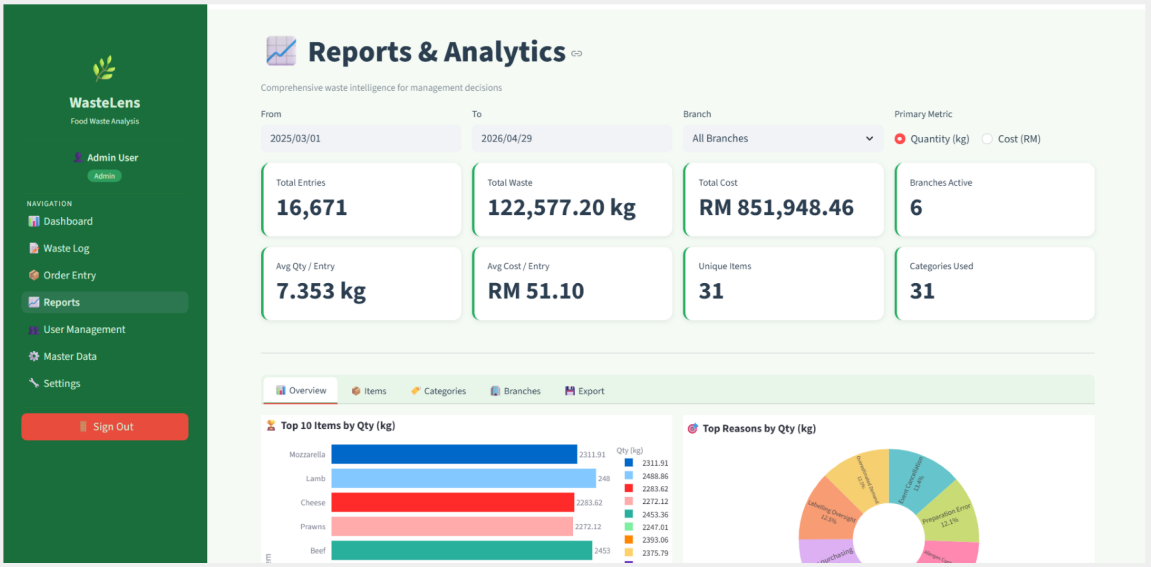
10.3 Field Validation and Guards

- All chart sections check if the query returned data before calling Plotly. Empty results show st.info() messages instead of blank charts.
- Radar chart: explicitly guards with "if len(df_br) >= 3" to avoid a degenerate 2-point polygon.
- Admin branch filter: only rendered for Admin role; Manager sees an st.info() indicating their accessible branches.

10.4 Screenshots – Reports & Analytics

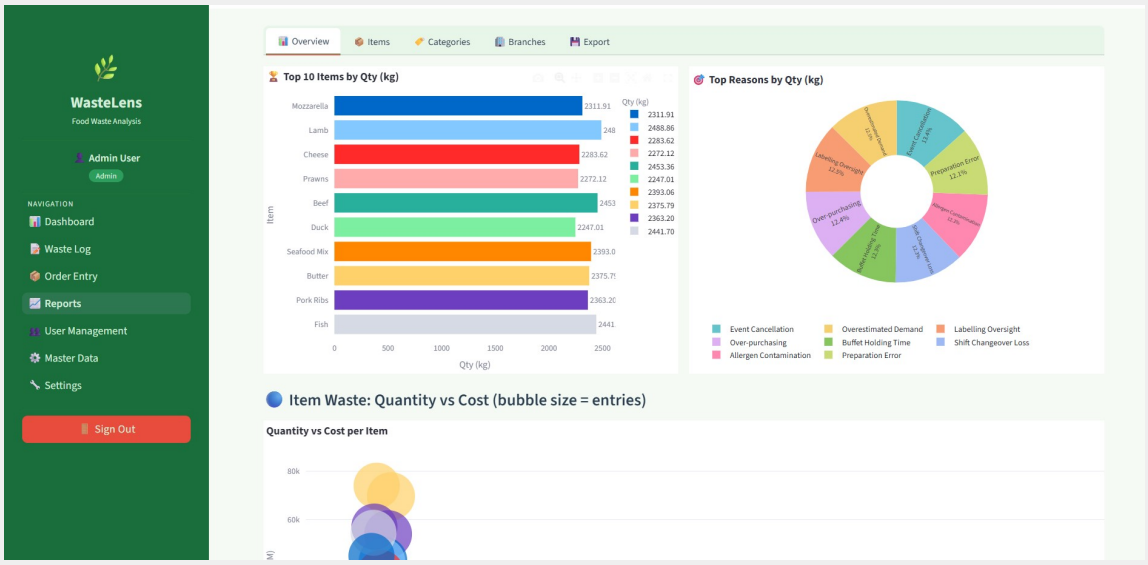
Reports – Global Filter Bar + KPI Cards

[SCREENSHOT — Reports page – filter row (dates, branch, metric), 8 KPI metric cards in 2 rows]



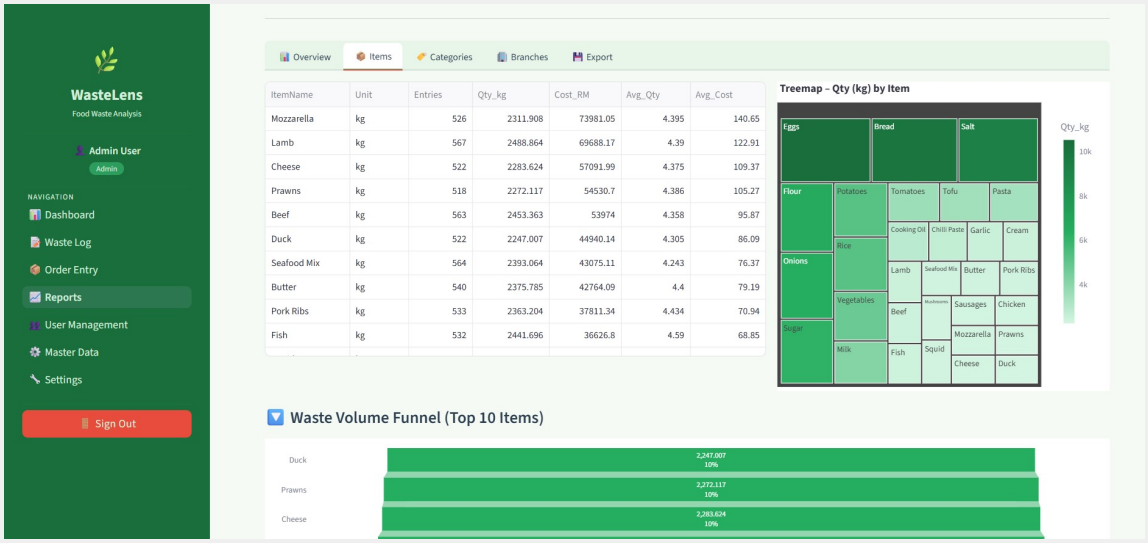
Reports – Overview Tab (Top Items Bar + Reasons Pie)

[SCREENSHOT — Reports Overview tab – top-10 items horizontal bar chart and reasons donut chart side by side]



Reports – Overview Tab (Qty vs Cost Scatter)

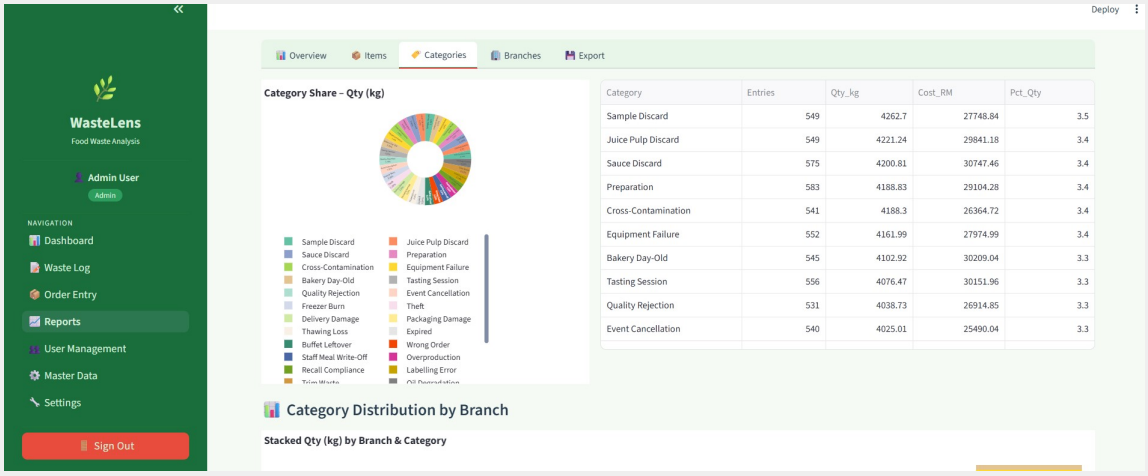
[SCREENSHOT — Reports Items tab – items summary table on left, treemap chart on right]



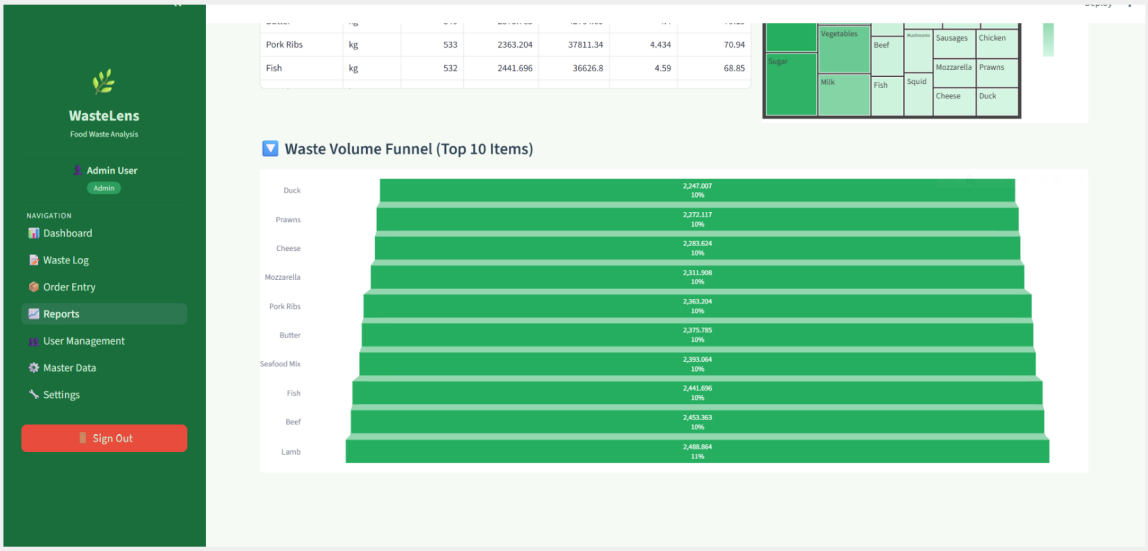
[SCREENSHOT — Reports Overview tab – bubble scatter chart of Quantity vs Cost coloured by item]

Reports – Items Tab (Table + Treemap) Reports – Items Tab (Funnel Chart)

[SCREENSHOT — Reports Categories tab – category donut chart, summary table, stacked bar by branch and category]

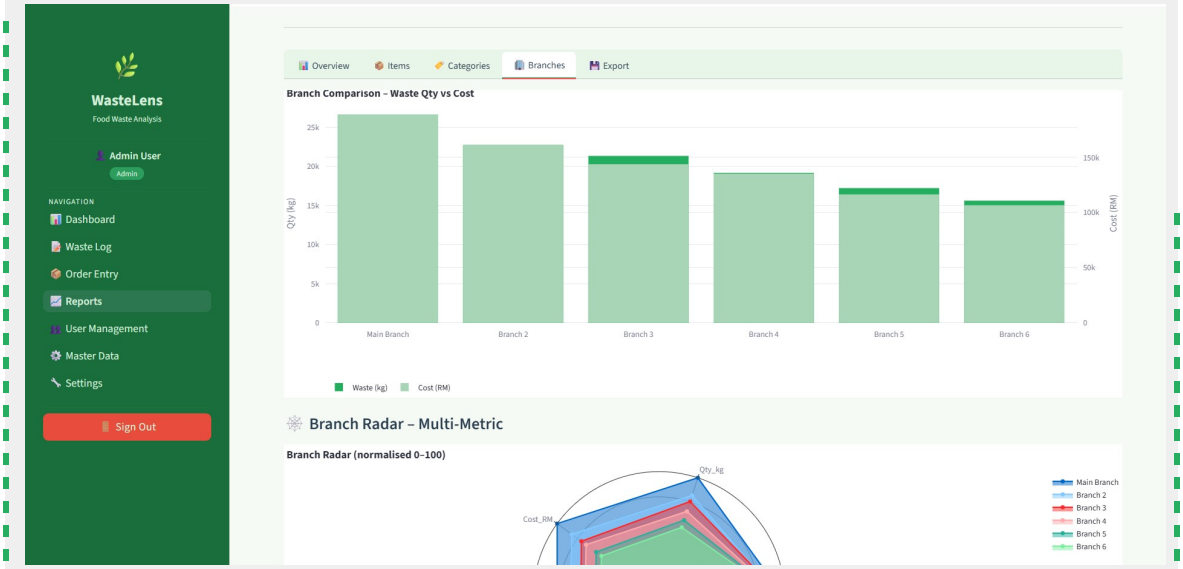


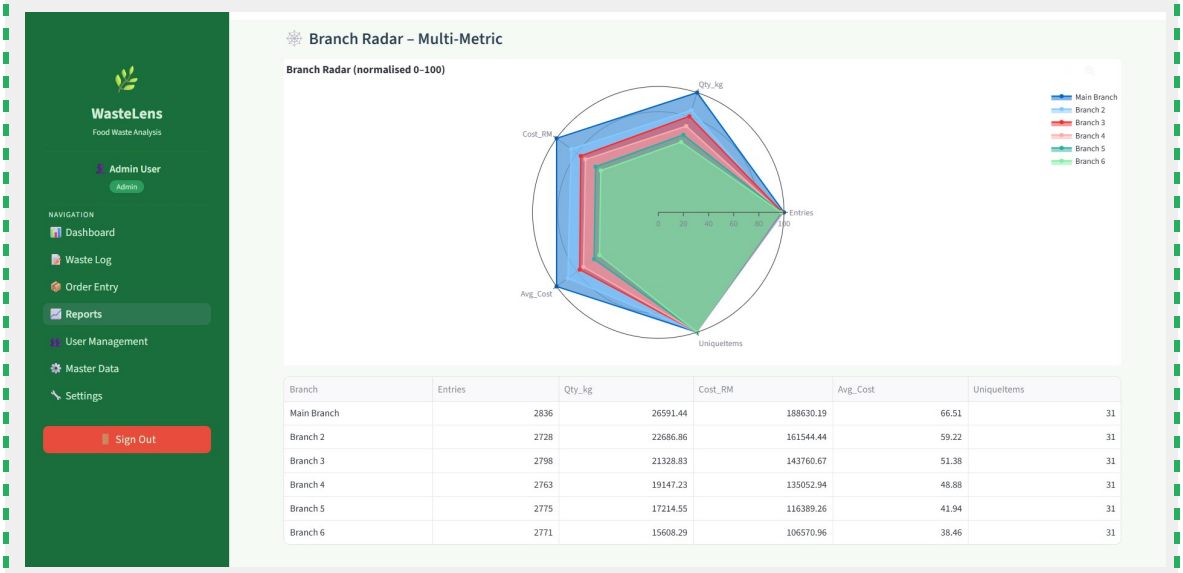
[SCREENSHOT — Reports Items tab – waste volume funnel chart for top 10 items]



Reports – Categories Tab (Donut + Stacked Bar)
Reports – Branches Tab (Grouped Bar + Radar)

[SCREENSHOT — Reports Branches tab – grouped bar (qty+cost), spider/radar chart for branch comparison, summary table]

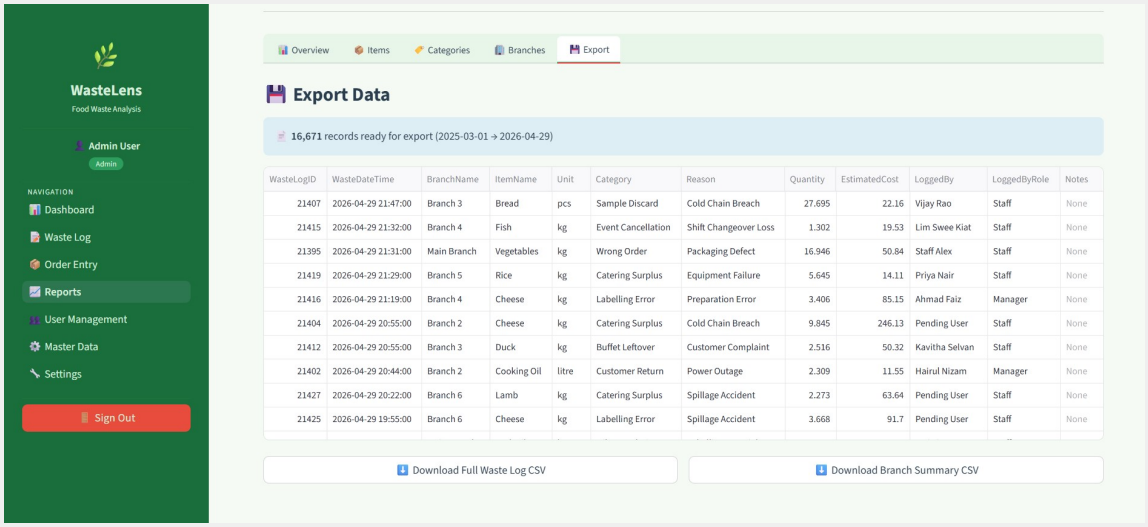




Branch	Entries	Qty_kg	Cost_RM	Avg_Cost	UniqueItems
Main Branch	2836	26591.44	188630.19	66.51	31
Branch 2	2728	22686.86	161544.44	59.22	31
Branch 3	2798	21328.83	143760.67	51.38	31
Branch 4	2763	19147.23	135052.94	48.88	31
Branch 5	2775	17214.55	116389.26	41.94	31
Branch 6	2771	15608.29	106570.96	38.46	31

Reports – Export Tab

[SCREENSHOT — Reports Export tab – record count info, 25-row preview table, CSV download buttons]



10.5 Source Code – 4_Reports.py

```
# ============================================================
#  WasteLens – Reports & Analytics  (Manager / Admin)
#  Enriched with 10+ chart types
# ============================================================
import streamlit as st

st.set_page_config(
    page_title="Reports | WasteLens",
    page_icon="📈",
    layout="wide",
)

import pandas as pd
import plotly.express as px
import plotly.graph_objects as go
from datetime import date, timedelta
from auth import require_role, branch_filter_sql
from db import run_query
from utils import inject_css, render_sidebar

inject_css()
user = require_role(["Admin", "Manager"])
render_sidebar(user)

bf = branch_filter_sql(user)

st.title("📈 Reports & Analytics")
st.caption("Comprehensive waste intelligence for management decisions")

# ═══ FILTERS (always visible, collapsed by default) ═══════════
```

```
rc1, rc2, rc3, rc4 = st.columns(4)

with rc1:
    rpt_from = st.date_input("From", value=date(2025, 3, 1))
with rc2:
    rpt_to   = st.date_input("To",   value=date.today())
with rc3:
    if user["RoleName"] == "Admin":
        all_br   = run_query("SELECT BranchID, BranchName FROM Branch WHERE
IsActive=1 ORDER BY BranchName")
        br_opts  = ["All Branches"] + [b["BranchName"] for b in all_br]
        sel_br   = st.selectbox("Branch", br_opts)
        if sel_br != "All Branches":
            bid      = next(b["BranchID"] for b in all_br if b["BranchName"] ==
sel_br)
            extra_bf = f" AND wl.BranchID = {bid}"
        else:
            extra_bf = bf
    else:
        extra_bf = bf
        st.info("Viewing your accessible branches")
with rc4:
    metric_type = st.radio("Primary Metric", ["Quantity (kg)", "Cost (RM)"],
horizontal=True)
    y_col    = "Qty_kg"   if metric_type == "Quantity (kg)" else "Cost_RM"
    y_label  = "Qty (kg)" if metric_type == "Quantity (kg)" else "Cost (RM)"

date_clause = " AND DATE(wl.WasteDateTime) BETWEEN %(fd)s AND %(td)s"
params = {"fd": str(rpt_from), "td": str(rpt_to)}

# == SUMMARY KPI CARDS (update immediately when filter changes) =
summary = run_query(
    f"""SELECT COUNT(*)                            AS Entries,
               ROUND(SUM(wl.Quantity),2)          AS Qty_kg,
               ROUND(SUM(wl.EstimatedCost),2)     AS Cost_RM,
               ROUND(AVG(wl.Quantity),3)          AS Avg_Qty,
               ROUND(AVG(wl.EstimatedCost),2)     AS Avg_Cost,
               COUNT(DISTINCT wl.BranchID)        AS Branches,
               COUNT(DISTINCT wl.ItemID)          AS UniqueItems,
               COUNT(DISTINCT wl.WasteCategoryID) AS Categories
        FROM WasteLog wl WHERE 1=1 {extra_bf} {date_clause}""",
    params,
)
if summary and summary[0]["Entries"]:
    r = summary[0]
    m1, m2, m3, m4 = st.columns(4)
    m1.metric("Total Entries",   f"{r['Entries']:,}")
    m2.metric("Total Waste",     f"{float(r['Qty_kg'] or 0):,.2f} kg")
    m3.metric("Total Cost",      f"RM {float(r['Cost_RM'] or 0):,.2f}")
```

```
    m4.metric("Branches Active", f"{r['Branches']}")
m5, m6, m7, m8 = st.columns(4)
    m5.metric("Avg Qty / Entry",  f"{float(r['Avg_Qty'] or 0):,.3f} kg")
    m6.metric("Avg Cost / Entry", f"RM {float(r['Avg_Cost'] or 0):,.2f}")
    m7.metric("Unique Items",     f"{r['UniqueItems']}")
    m8.metric("Categories Used",  f"{r['Categories']}")
else:
    st.info("No data found for the selected filters.")
st.markdown("---")

# == TABS ================================================
```

```
tabs = st.tabs([
    "📊 Overview", "📦 Items", "📁 Categories",
    "🏬 Branches", "💾 Export"
])

# ──────────────────────────────────────────────────────────────
# TAB 1 – OVERVIEW
# ──────────────────────────────────────────────────────────────
with tabs[0]:
    c_left, c_right = st.columns(2)

    # Top 10 items by cost – horizontal bar
    with c_left:
        top_items = run_query(
            f"""SELECT i.ItemName,
                       ROUND(SUM(wl.Quantity),2)      AS Qty_kg,
                       ROUND(SUM(wl.EstimatedCost),2) AS Cost_RM
                FROM WasteLog wl JOIN Item i ON i.ItemID=wl.ItemID
                WHERE 1=1 {extra_bf} {date_clause}
                GROUP BY i.ItemName ORDER BY Cost_RM DESC LIMIT 10""",
            params,
        )
        if top_items:
            df_ti = pd.DataFrame(top_items)
            fig = px.bar(df_ti, x=y_col, y="ItemName", orientation="h",
                         color=y_col, text=y_col,
                         color_continuous_scale=["#a8d5b5","#27AE60","#1a6e3c"],
                         title=f"🏆 Top 10 Items by {y_label}",
                         labels={"ItemName":"Item", y_col: y_label})
            fig.update_traces(texttemplate="%{text:.2f}", textposition="outside")
            fig.update_layout(plot_bgcolor="white", paper_bgcolor="white",
                              coloraxis_showscale=False,
                              yaxis=dict(autorange="reversed"),
                              margin=dict(t=40,b=10))
            st.plotly_chart(fig, use_container_width=True)

    # Waste by reason – pie
    with c_right:
        reason_data = run_query(
            f"""SELECT wr.ReasonText AS Reason,
                       ROUND(SUM(wl.Quantity),2)      AS Qty_kg,
                       ROUND(SUM(wl.EstimatedCost),2) AS Cost_RM
                FROM WasteLog wl JOIN WasteReason wr ON wr.ReasonID=wl.ReasonID
                WHERE 1=1 {extra_bf} {date_clause}
                GROUP BY wr.ReasonText ORDER BY {y_col} DESC LIMIT 8""",
            params,
        )
        if reason_data:
            df_rr = pd.DataFrame(reason_data)
            fig2 = px.pie(df_rr, names="Reason", values=y_col, hole=0.4,
                          color_discrete_sequence=px.colors.qualitative.Pastel,
                          title=f"🎯 Top Reasons by {y_label}")
            fig2.update_traces(textinfo="percent+label", textposition="inside",
                               insidetextorientation="radial")
            fig2.update_layout(showlegend=True,
                               legend=dict(orientation="h", y=-0.2),
```

```
                                paper_bgcolor="white",
                                margin=dict(t=50, b=60, l=20, r=20))
```

```
            st.plotly_chart(fig2, use_container_width=True)

    # Scatter: Qty vs Cost per item
    st.markdown("#### 🔵 Item Waste: Quantity vs Cost (bubble size = entries)")
    scatter_data = run_query(
        f"""SELECT i.ItemName,
                   ROUND(SUM(wl.Quantity),2)      AS Qty_kg,
                   ROUND(SUM(wl.EstimatedCost),2) AS Cost_RM,
                   COUNT(*)                        AS Entries
            FROM WasteLog wl JOIN Item i ON i.ItemID=wl.ItemID
            WHERE 1=1 {extra_bf} {date_clause}
            GROUP BY i.ItemName ORDER BY Qty_kg DESC""",
        params,
    )
    if scatter_data:
        df_sc = pd.DataFrame(scatter_data)
        fig3  = px.scatter(df_sc, x="Qty_kg", y="Cost_RM", size="Entries",
                           color="ItemName", hover_name="ItemName",
                           labels={"Qty_kg":"Quantity (kg)","Cost_RM":"Cost
(RM)"},
                           title="Quantity vs Cost per Item",
                           size_max=50)
        fig3.update_layout(plot_bgcolor="white", paper_bgcolor="white",
                           showlegend=False, margin=dict(t=40,b=10))
        st.plotly_chart(fig3, use_container_width=True)

# ─────────────────────────────────────────────────────────────
# TAB 2 – ITEMS
# ─────────────────────────────────────────────────────────────
with tabs[1]:
    item_data = run_query(
        f"""SELECT i.ItemName, i.Unit,
                   COUNT(*)                          AS Entries,
                   ROUND(SUM(wl.Quantity),3)         AS Qty_kg,
                   ROUND(SUM(wl.EstimatedCost),2)    AS Cost_RM,
                   ROUND(AVG(wl.Quantity),3)         AS Avg_Qty,
                   ROUND(AVG(wl.EstimatedCost),2)    AS Avg_Cost
            FROM WasteLog wl JOIN Item i ON i.ItemID=wl.ItemID
            WHERE 1=1 {extra_bf} {date_clause}
            GROUP BY i.ItemName, i.Unit ORDER BY Cost_RM DESC""",
        params,
    )
    if item_data:
        df_i = pd.DataFrame(item_data)
        col_a, col_b = st.columns([3,2])
        with col_a:
            st.dataframe(df_i, use_container_width=True, hide_index=True)
        with col_b:
            fig_tm = px.treemap(df_i, path=["ItemName"], values=y_col,
color=y_col,

color_continuous_scale=["#d5f5e3","#27AE60","#1a6e3c"],
                                title=f"Treemap – {y_label} by Item")
```

```
            fig_tm.update_layout(margin=dict(t=30,b=5))
            st.plotly_chart(fig_tm, use_container_width=True)

        # Funnel: top items by quantity
        st.markdown("#### 🔽 Waste Volume Funnel (Top 10 Items)")
        df_funnel = df_i.head(10).sort_values("Qty_kg")

        fig_f = go.Figure(go.Funnel(
            y=df_funnel["ItemName"], x=df_funnel["Qty_kg"],
            textinfo="value+percent total",
            marker=dict(color="#27AE60"),
        ))
        fig_f.update_layout(paper_bgcolor="white", margin=dict(t=10,b=10))
        st.plotly_chart(fig_f, use_container_width=True)

# ─────────────────────────────────────────────────────────────
# TAB 3 - CATEGORIES
# ─────────────────────────────────────────────────────────────
with tabs[2]:
    cat_data = run_query(
        f"""SELECT wc.TypeName AS Category,
                   COUNT(*)                          AS Entries,
                   ROUND(SUM(wl.Quantity),2)         AS Qty_kg,
                   ROUND(SUM(wl.EstimatedCost),2)    AS Cost_RM,
                   ROUND(100.0*SUM(wl.Quantity)/
                         SUM(SUM(wl.Quantity)) OVER(),1) AS Pct_Qty
            FROM WasteLog wl
            JOIN WasteCategory wc ON wc.WasteCategoryID=wl.WasteCategoryID
            WHERE 1=1 {extra_bf} {date_clause}
            GROUP BY wc.TypeName ORDER BY Qty_kg DESC""",
        params,
    )
    if cat_data:
        df_cat = pd.DataFrame(cat_data)
        col_a, col_b = st.columns([2,3])
        with col_a:
            fig_pie = px.pie(df_cat, names="Category", values=y_col, hole=0.4,
                             color_discrete_sequence=px.colors.qualitative.Set2,
                             title=f"Category Share – {y_label}")
            fig_pie.update_traces(textinfo="percent+label", textposition="inside",
                                  insidetextorientation="radial")
            fig_pie.update_layout(showlegend=True,
                                  legend=dict(orientation="h", y=-0.2),
                                  paper_bgcolor="white",
                                  margin=dict(t=50, b=60, l=20, r=20))
            st.plotly_chart(fig_pie, use_container_width=True)
        with col_b:
            st.dataframe(df_cat, use_container_width=True, hide_index=True)

        # Stacked bar: category by branch
        st.markdown("#### 📊 Category Distribution by Branch")
        stacked = run_query(
            f"""SELECT b.BranchName AS Branch,
                       wc.TypeName  AS Category,
```

```
                    ROUND(SUM(wl.Quantity),2)      AS Qty_kg,
                    ROUND(SUM(wl.EstimatedCost),2) AS Cost_RM
                FROM WasteLog wl
                JOIN WasteCategory wc ON wc.WasteCategoryID=wl.WasteCategoryID
                JOIN Branch b ON b.BranchID=wl.BranchID
                WHERE 1=1 {extra_bf} {date_clause}
                GROUP BY b.BranchName, wc.TypeName
                ORDER BY Branch, Qty_kg DESC""",
            params,
        )
        if stacked:
```

```
            df_st = pd.DataFrame(stacked)
            fig_st = px.bar(df_st, x="Branch", y=y_col, color="Category",
                            barmode="stack",
                            color_discrete_sequence=px.colors.qualitative.Set2,
                            labels={"Branch":"Branch", y_col: y_label},
                            title=f"Stacked {y_label} by Branch & Category")
            fig_st.update_layout(plot_bgcolor="white", paper_bgcolor="white",
legend=dict(orientation="h",y=-0.3),
                                 margin=dict(t=40,b=10))
            st.plotly_chart(fig_st, use_container_width=True)

# ─────────────────────────────────────────────────────────────
# TAB 4 - BRANCHES
# ─────────────────────────────────────────────────────────────
with tabs[3]:
    branch_data = run_query(
        f"""SELECT b.BranchName AS Branch,
                   COUNT(wl.WasteLogID)              AS Entries,
                   ROUND(SUM(wl.Quantity),2)          AS Qty_kg,
                   ROUND(SUM(wl.EstimatedCost),2)     AS Cost_RM,
                   ROUND(AVG(wl.EstimatedCost),2)     AS Avg_Cost,
                   COUNT(DISTINCT wl.ItemID)           AS UniqueItems
            FROM   Branch b
            LEFT JOIN WasteLog wl ON wl.BranchID=b.BranchID
                  AND DATE(wl.WasteDateTime) BETWEEN %(fd)s AND %(td)s
            WHERE  b.IsActive=1
            GROUP  BY b.BranchID, b.BranchName
            ORDER  BY Qty_kg DESC""",
        params,
    )
    if branch_data:
        df_br = pd.DataFrame(branch_data)

        # grouped bar
        fig5 = go.Figure()
        fig5.add_trace(go.Bar(name="Waste (kg)", x=df_br["Branch"],
                              y=df_br["Qty_kg"], marker_color="#27AE60"))
        fig5.add_trace(go.Bar(name="Cost (RM)", x=df_br["Branch"],
                              y=df_br["Cost_RM"], marker_color="#a8d5b5",
                              yaxis="y2"))
        fig5.update_layout(
            barmode="group",
            yaxis =dict(title="Qty (kg)"),
            yaxis2=dict(title="Cost (RM)", overlaying="y", side="right"),
            legend=dict(orientation="h",y=-0.25),
```

```
            plot_bgcolor="white", paper_bgcolor="white",
            margin=dict(t=20,b=10),
            title="Branch Comparison – Waste Qty vs Cost",
        )
        st.plotly_chart(fig5, use_container_width=True)

        # radar chart – branch performance
        if len(df_br) >= 3:
            st.markdown("#### 🕸️ Branch Radar – Multi-Metric")
            metrics = ["Entries","Qty_kg","Cost_RM","Avg_Cost","UniqueItems"]
            df_norm = df_br[["Branch"]+metrics].copy()
            # cast every metric column to float (MySQL Decimal → Python object)
            for m in metrics:
                df_norm[m] = pd.to_numeric(df_norm[m], errors="coerce").fillna(0)
            for m in metrics:
                mx = float(df_norm[m].max())
                df_norm[m] = ((df_norm[m] / mx * 100).round(1)
                              if mx > 0 else pd.Series([0.0]*len(df_norm)))
            fig_rad = go.Figure()
            for _, row in df_norm.iterrows():
                fig_rad.add_trace(go.Scatterpolar(
                    r=[row[m] for m in metrics],
                    theta=metrics, fill="toself",
                    name=row["Branch"],
                ))
            fig_rad.update_layout(
                polar=dict(radialaxis=dict(visible=True, range=[0,100])),
                showlegend=True, paper_bgcolor="white",
                margin=dict(t=40,b=40),
                title="Branch Radar (normalised 0–100)",
            )
            st.plotly_chart(fig_rad, use_container_width=True)

        st.dataframe(df_br, use_container_width=True, hide_index=True)

# ═══════════════════════════════════════════════════════════════
# TAB 5 – EXPORT
# ═══════════════════════════════════════════════════════════════
with tabs[4]:
    st.subheader("💾 Export Data")

    export_data = run_query(
        f"""SELECT wl.WasteLogID, wl.WasteDateTime, b.BranchName,
                   i.ItemName, i.Unit, wc.TypeName AS Category,
                   wr.ReasonText AS Reason, wl.Quantity,
                   wl.EstimatedCost, u.UserName AS LoggedBy,
                   r.RoleName AS LoggedByRole, wl.Notes
            FROM WasteLog wl
            JOIN Branch        b  ON b.BranchID         = wl.BranchID
            JOIN Item          i  ON i.ItemID           = wl.ItemID
            JOIN WasteCategory wc ON wc.WasteCategoryID = wl.WasteCategoryID
            JOIN WasteReason   wr ON wr.ReasonID        = wl.ReasonID
            JOIN User          u  ON u.UserID           = wl.LoggedByUserID
            JOIN Role          r  ON r.RoleID           = u.RoleID
            WHERE 1=1 {extra_bf} {date_clause}
```

```
            ORDER BY wl.WasteDateTime DESC""",
        params,
    )

    if export_data:
        df_exp = pd.DataFrame(export_data)
        st.info(f"📄 **{len(df_exp):,}** records ready for export "
                f"({rpt_from} → {rpt_to})")
        st.dataframe(df_exp.head(25), use_container_width=True, hide_index=True)

        col1, col2 = st.columns(2)
        with col1:
            csv = df_exp.to_csv(index=False).encode("utf-8")
            st.download_button("⬇️ Download Full Waste Log CSV", data=csv,
                               file_name=f"wastelens_{rpt_from}_{rpt_to}.csv",
                               mime="text/csv", use_container_width=True)
        with col2:
            branch_sum = run_query(
                f"""SELECT b.BranchName AS Branch,
                           ROUND(SUM(wl.Quantity),2)      AS Total_Waste_kg,
                           ROUND(SUM(wl.EstimatedCost),2) AS Total_Cost_RM,
                           COUNT(*)                        AS Entries
                    FROM WasteLog wl JOIN Branch b ON b.BranchID=wl.BranchID
                    WHERE 1=1 {extra_bf} {date_clause}
                    GROUP BY b.BranchName ORDER BY Total_Cost_RM DESC""",
                params,
            )
            if branch_sum:
                csv2 = pd.DataFrame(branch_sum).to_csv(index=False).encode("utf-
8")
                st.download_button("⬇️ Download Branch Summary CSV", data=csv2,

file_name=f"wastelens_branch_{rpt_from}_{rpt_to}.csv",
                                   mime="text/csv", use_container_width=True)
    else:
        st.info("No data to export for the selected period.")
```

11. User Management (5_User_Management.py)

11.1 Design View

User Management is restricted to the Admin role. It provides a full suite of account administration tools across four tabs.

Stat Cards Row (always visible):

- 4 metrics: Total Users, Active Users, Pending Approval, Inactive Users.

Tab 1 – All Users:

- Search bar (name or email) and Role filter dropdown.
- Each user rendered as a styled card: avatar circle with initials (colour-coded by role), name, email, role badge, status badge (Active/Pending/Inactive), join date, and branch count.
- Below each card: action buttons – Approve (if pending), Activate/Deactivate toggle, Change Role selectbox + Save button.
- Role colour coding: Admin = purple, Manager = blue, Staff = green.

Tab 2 – Pending Approval:

- Lists only users where IsApproved=0 and IsActive=1.
- Shows name, email, role badge, branch, registered date.
- One-click Approve (sets IsApproved=1) and Reject (sets IsActive=0) buttons per row.
- If no pending users: success message "No pending users — all accounts are approved!"

Tab 3 – Add User:

- Admin creates a new account directly without going through the registration flow.
- First Name + Last Name (mirrors the Register form layout).
- Email, Role dropdown, Branch multiselect (can assign multiple branches).
- Password + Confirm Password fields.
- "Auto-approve account" checkbox (checked by default — admin-created accounts skip the approval queue).

Tab 4 – Branch Access:

- Select any active user from a dropdown.
- Multiselect of all active branches with current assignments pre-selected.
- Access Type dropdown (Write / Read / Admin).
- "Update Branch Access" button: DELETE existing access rows then re-INSERT the selected branches.

11.2 Execution View

The All Users cards are built as pure Python f-string concatenation. No HTML comments (<!-- -->) are used inside any st.markdown() block because Streamlit treats commented

HTML as a code fence and renders the entire block as raw text instead of HTML. Action buttons are rendered in a separate st.columns() row immediately below each card, outside the HTML.

The Add User form has all input widgets OUTSIDE st.form() (like the Register page) so validation feedback and the password strength check are visible before submission. Only the submit button is inside st.form(). On submit: email uniqueness is verified with a SELECT query before the INSERT.

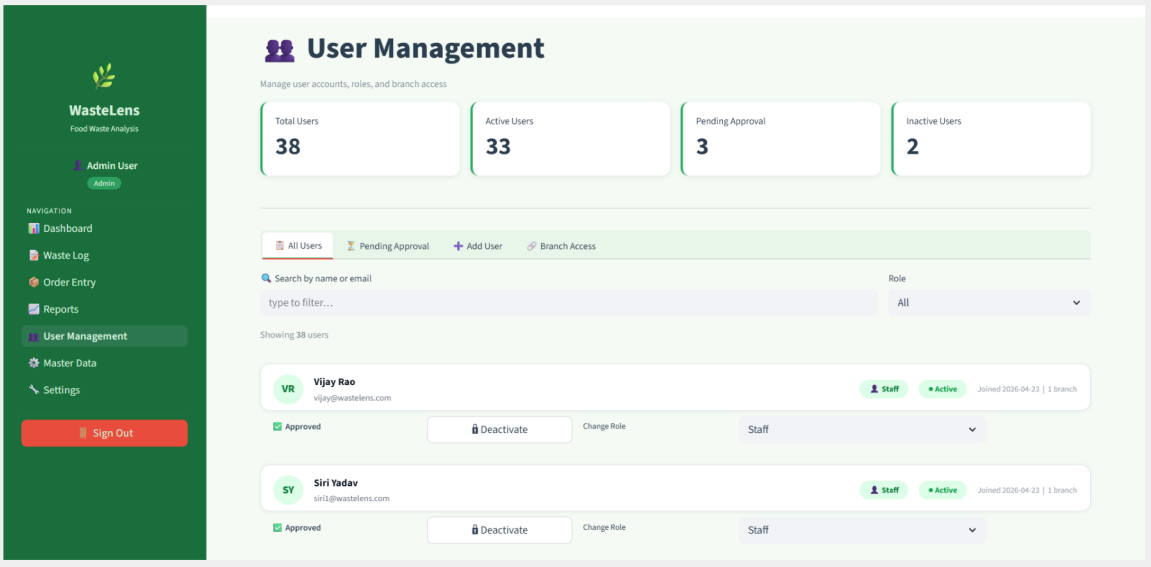
11.3 Field Validation Rules

- First Name + Last Name: both must be non-empty.
- Email: non-empty and must not already exist in the User table.
- Branch: at least one branch must be selected from the multiselect.
- Password: minimum 8 characters. Error: "Password must be at least 8 characters."
- Confirm Password: must exactly match Password. Error: "Passwords do not match."

11.4 Screenshots – User Management

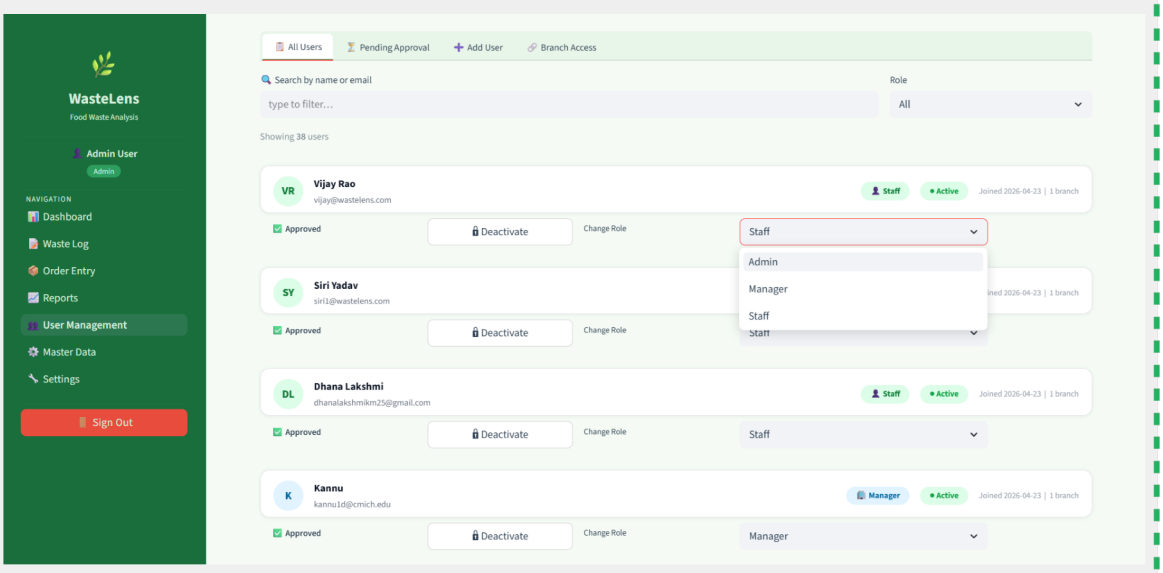
User Management – Stat Cards + All Users Tab

[SCREENSHOT — User Management – 4 stat cards (Total, Active, Pending, Inactive) and All Users card list]



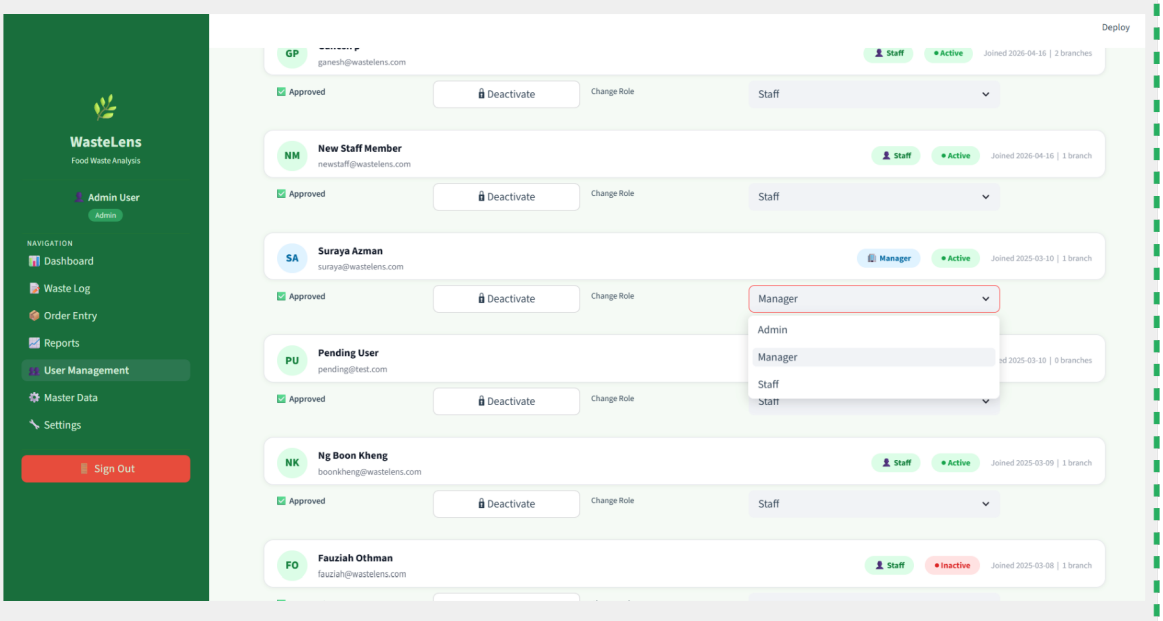
User Management – User Card (Expanded Actions)

[SCREENSHOT — Close-up of a single user card showing avatar, name, email, role badge, status badge, and action buttons row]



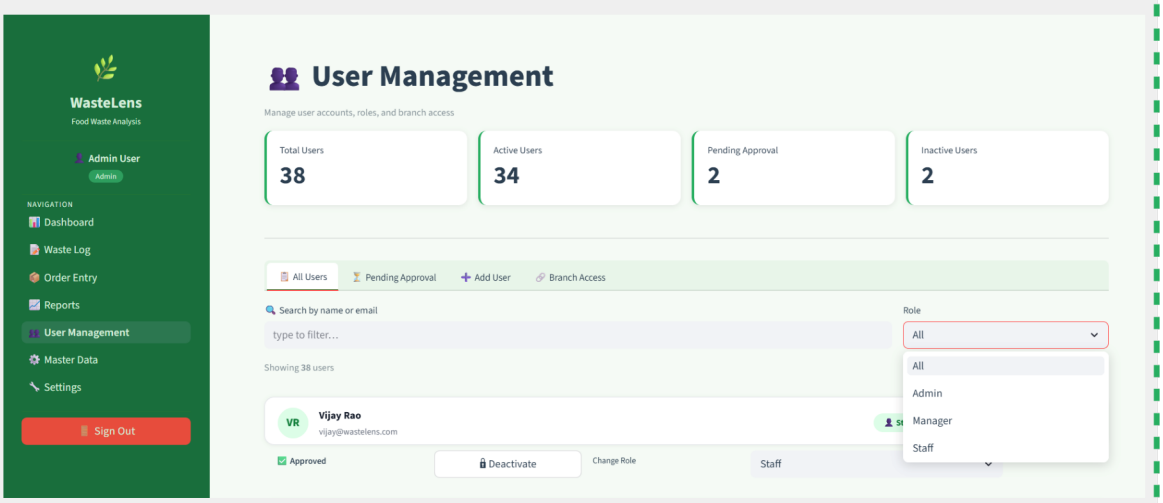
User Management – Change Role Dropdown + Save

[SCREENSHOT — User Management – Role selectbox open for a user with Save button visible]



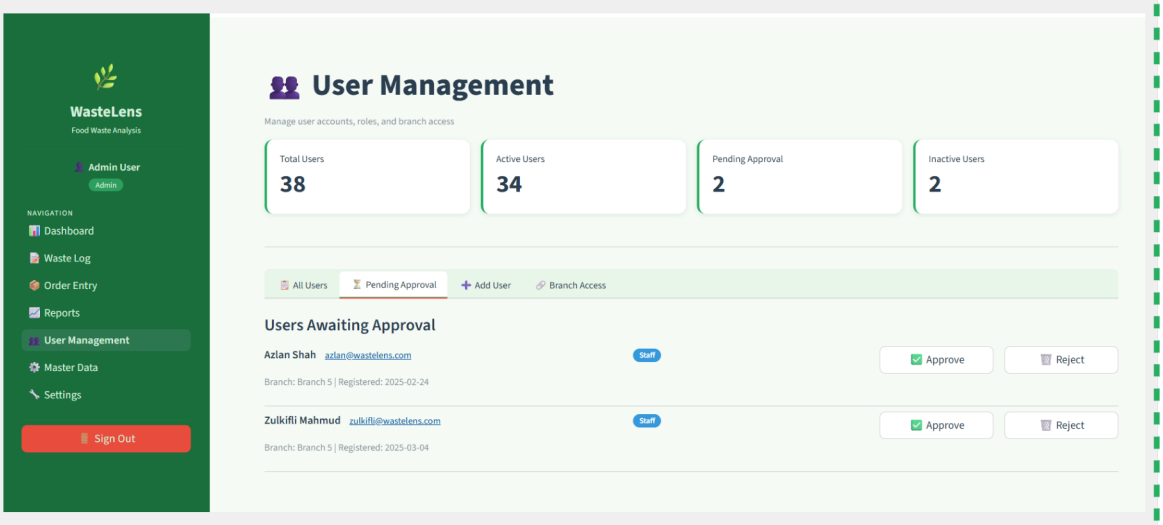
User Management – Pending Approval Tab

[SCREENSHOT — Pending Approval tab – list of unverified users with Approve and Reject buttons]



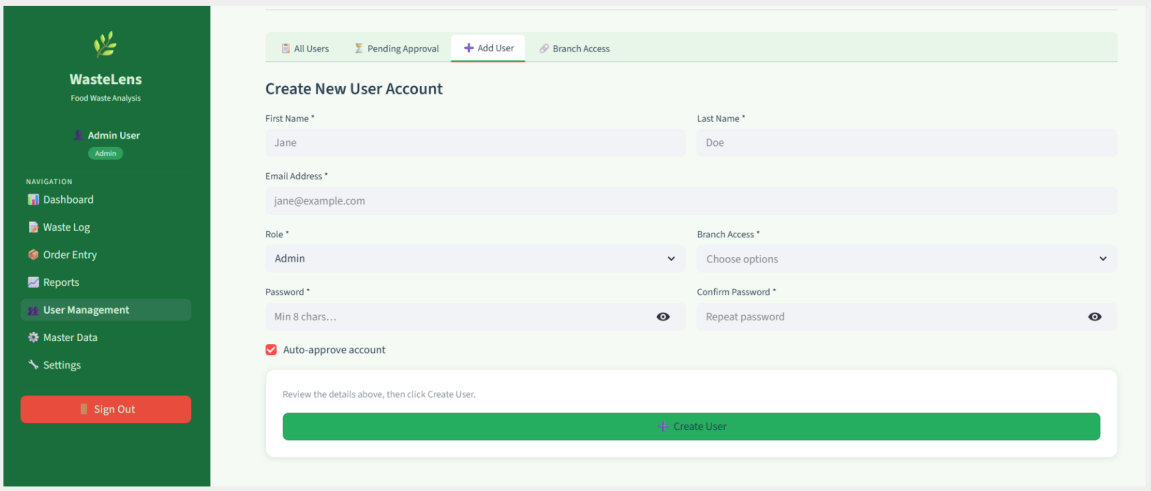
User Management – No Pending Users

[SCREENSHOT — Pending Approval tab – green success banner "No pending users — all accounts are approved!"]



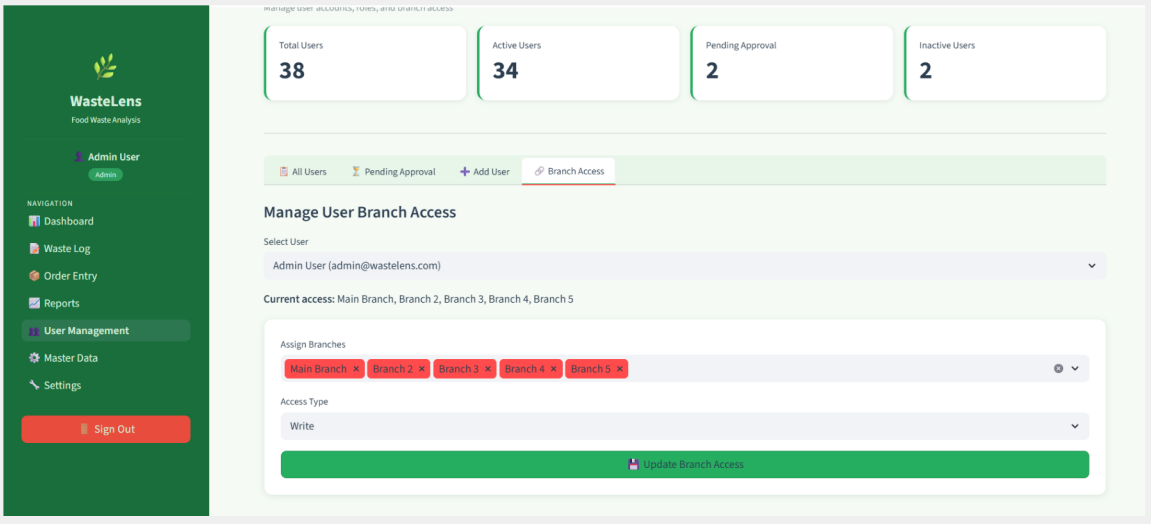
User Management – Add User Tab

[SCREENSHOT — Add User tab – First Name, Last Name, Email, Role, Branch multiselect, Password, Confirm, Auto-approve checkbox]



User Management – Branch Access Tab

[SCREENSHOT — Branch Access tab – user dropdown, current access shown, multiselect with pre-selected branches, Access Type dropdown, Update button]



11.5 Source Code – 5_User_Management.py

```
# ============================================================
#  WasteLens - User Management (Admin only)
```

```
# ============================================================
import streamlit as st

st.set_page_config(
    page_title="User Management | WasteLens",
    page_icon="👥",
    layout="wide",
)

import pandas as pd
from auth import require_role
from db import run_query, run_update
from utils import inject_css, render_sidebar, badge

inject_css()
user = require_role(["Admin"])
render_sidebar(user)


st.title("👥 User Management")
st.caption("Manage user accounts, roles, and branch access")

# == STAT CARDS ==============================================
stats = run_query(
    """SELECT COUNT(*)                                  AS Total,
              SUM(IsActive=1 AND IsApproved=1)       AS Active,
              SUM(IsApproved=0 AND IsActive=1)       AS Pending,
              SUM(IsActive=0)                        AS Inactive
       FROM User"""
)
if stats:
    s = stats[0]
    c1, c2, c3, c4 = st.columns(4)
    c1.metric("Total Users",    f"{s['Total']:,}")
    c2.metric("Active Users",   f"{s['Active']:,}")
    c3.metric("Pending Approval", f"{s['Pending']:,}")
    c4.metric("Inactive Users", f"{s['Inactive']:,}")

st.markdown("---")

# == TABS ====================================================
tab_all, tab_pending, tab_add, tab_branch = st.tabs(
    ["📋 All Users", "⏳ Pending Approval", "➕ Add User", "🔗 Branch Access"]
)

# ------------------------------------------------------------
# TAB 1 - ALL USERS
# ------------------------------------------------------------
with tab_all:
    # Search / filter
    sf1, sf2 = st.columns([3, 1])
    with sf1:
        search = st.text_input("🔍 Search by name or email", placeholder="type to
filter…")
    with sf2:
```

```
        role_filter = st.selectbox("Role", ["All", "Admin", "Manager", "Staff"])

    role_clause = "" if role_filter == "All" else f" AND r.RoleName =
'{role_filter}'"
    search_clause = ""
    search_params: list = []
    if search:
        search_clause = " AND (u.UserName LIKE %s OR u.Email LIKE %s)"
        search_params = [f"%{search}%", f"%{search}%"]

    all_users = run_query(
        f"""SELECT u.UserID, u.UserName, u.Email,
                   r.RoleName,
                   u.IsApproved, u.IsActive,
                   DATE(u.CreatedAt) AS Joined,
                   COUNT(uba.BranchID) AS Branches
            FROM   User u
            JOIN   Role r ON r.RoleID = u.RoleID
            LEFT JOIN UserBranchAccess uba ON uba.UserID = u.UserID
            WHERE  1=1 {role_clause} {search_clause}
            GROUP  BY u.UserID
            ORDER  BY u.CreatedAt DESC""",
        tuple(search_params),
    )

    if not all_users:
        st.info("No users found.")
    else:
        roles_avail = run_query("SELECT RoleID, RoleName FROM Role WHERE IsActive=1")
        role_map    = {r["RoleName"]: r["RoleID"] for r in roles_avail}

        total_shown = len(all_users)
        st.caption(f"Showing **{total_shown}** user{'s' if total_shown != 1 else
''}")
        st.markdown("<div style='height:6px'></div>", unsafe_allow_html=True)

        ROLE_META = {
            "Admin":   {"color": "#7c3aed", "bg": "#f5f3ff", "icon": "👑"},
            "Manager": {"color": "#0369a1", "bg": "#e0f2fe", "icon": "🏢"},
            "Staff":   {"color": "#15803d", "bg": "#dcfce7", "icon": "👤"},
        }

        for r in all_users:
            status_text  = ("Active"   if r["IsActive"] and r["IsApproved"]
                            else "Pending"  if not r["IsApproved"] and
r["IsActive"]
                            else "Inactive")
            status_cfg = {
                "Active":   {"dot": "●", "color": "#16a34a", "bg": "#dcfce7"},
                "Pending":  {"dot": "●", "color": "#d97706", "bg": "#fef9c3"},
                "Inactive": {"dot": "●", "color": "#dc2626", "bg": "#fee2e2"},
            }[status_text]

            rm = ROLE_META.get(r["RoleName"], {"color": "#555", "bg": "#eee", "icon":
"👤"})
```

```
            # ── initials avatar ──────────────────────────────
            parts    = r["UserName"].split()

            initials = (parts[0][0] + (parts[-1][0] if len(parts) > 1 else
"")).upper()

            # ── render card ──────────────────────────────────
            with st.container():
                card_html = (
                    f"<div style='background:white;border-radius:14px;"
f"border:1px solid #e5e7eb;"
                    f"box-shadow:0 1px 6px rgba(0,0,0,.06);"
                    f"padding:.9rem 1.2rem .5rem;margin-bottom:.5rem'>"
                    f"<div
style='display:flex;align-items:center;gap:1rem;flex-wrap:wrap'>"
                    f"<div style='width:44px;height:44px;border-radius:50%;"
                    f"background:{rm['bg']};color:{rm['color']};"
                    f"display:flex;align-items:center;justify-content:center;"
                    f"font-weight:800;font-size:1rem;flex-
shrink:0'>{initials}</div>"
                    f"<div style='flex:1;min-width:160px'>"
                    f"<div
style='font-weight:700;color:#111827;font-size:.95rem'>{r['UserName']}</div>"
                    f"<div
style='color:#6b7280;font-size:.80rem;margin-top:1px'>{r['Email']}</div>"
f"</div>"
                    f"<span style='background:{rm['bg']};color:{rm['color']};"
                    f"border-radius:20px;padding:4px 14px;"
                    f"font-size:.76rem;font-weight:700;white-space:nowrap'>"
                    f"{rm['icon']} {r['RoleName']}</span>"
                    f"<span
style='background:{status_cfg['bg']};color:{status_cfg['color']};"
                    f"border-radius:20px;padding:4px 14px;"
                    f"font-size:.76rem;font-weight:700;white-space:nowrap'>"
                    f"{status_cfg['dot']} {status_text}</span>"
                    f"<span
style='color:#9ca3af;font-size:.75rem;white-space:nowrap'>"
                    f"Joined {r['Joined']}  |  "
                    f"{r['Branches']}  branch{'es'  if  r['Branches']  !=  1  else
''}</span>"
                    f"</div></div>"
                )
                st.markdown(card_html, unsafe_allow_html=True)

                # action bar (pure Streamlit buttons — no mixing with HTML)
                _, ab1, ab2, ab3, ab4, ab5 = st.columns([0.05, 1.2, 1.2, 1.2, 2.0,
0.8])

                with ab1:
                    if not r["IsApproved"]:
                        if st.button("✅ Approve", key=f"app_{r['UserID']}",
                                     use_container_width=True):
                            run_update("UPDATE User SET IsApproved=1 WHERE
UserID=%s",
                                       (r["UserID"],))
                            st.rerun()
```

```
                    else:
                        st.markdown(
                            "<p
style='color:#16a34a;font-size:.8rem;font-weight:600;"
                            "margin:6px 0 0'>✅ Approved</p>",
                            unsafe_allow_html=True,
```

```
                        )

                with ab2:
                    if r["IsActive"]:
                        if st.button("🔒 Deactivate", key=f"deact_{r['UserID']}",
                                     use_container_width=True):
                            run_update("UPDATE User SET IsActive=0 WHERE
UserID=%s",
                                       (r["UserID"],))
                            st.rerun()
                    else:
                        if st.button("🔓 Activate", key=f"act_{r['UserID']}",
                                     use_container_width=True):
                            run_update("UPDATE User SET IsActive=1 WHERE
UserID=%s",
                                       (r["UserID"],))
                            st.rerun()

                with ab3:
                    st.markdown(
                        "<p    style='color:#6b7280;font-size:.76rem;margin:6px    0
2px'>"
                        "Change Role</p>",
                        unsafe_allow_html=True,
                    )

                with ab4:
                    cur_idx = list(role_map.keys()).index(r["RoleName"]) \
                              if r["RoleName"] in role_map else 0
                    new_role_sel = st.selectbox(
                        "role", list(role_map.keys()), index=cur_idx,
                        key=f"role_{r['UserID']}", label_visibility="collapsed",
                    )

                with ab5:
                    if new_role_sel != r["RoleName"]:
                        if st.button("💾 Save", key=f"saverole_{r['UserID']}",
                                     use_container_width=True):
                            run_update(
                                "UPDATE User SET RoleID=%s WHERE UserID=%s",
                                (role_map[new_role_sel], r["UserID"]),
                            )
                            st.rerun()

                st.markdown("<div                        style='height:2px'></div>",
unsafe_allow_html=True)

# ──────────────────────────────────────────────────────────────
# TAB 2 – PENDING APPROVAL
```

```
# ──────────────────────────────────────────────────────────
with tab_pending:
    st.markdown("#### Users Awaiting Approval")
pending = run_query(
        """SELECT u.UserID, u.UserName, u.Email,
                  r.RoleName, DATE(u.CreatedAt) AS Registered,
                  GROUP_CONCAT(b.BranchName SEPARATOR ', ') AS Branches
           FROM   User u
           JOIN   Role r ON r.RoleID = u.RoleID
           LEFT JOIN UserBranchAccess uba ON uba.UserID = u.UserID
           LEFT JOIN Branch b ON b.BranchID = uba.BranchID
           WHERE  u.IsApproved=0 AND u.IsActive=1
           GROUP  BY u.UserID
           ORDER  BY u.CreatedAt"""
    )

    if not pending:
        st.success("🎉 No pending users — all accounts are approved!")
    else:
        for p in pending:
            with st.container():
                pc1, pc2, pc3, pc4 = st.columns([3, 2, 1, 1])
                with pc1:
                    st.markdown(
                        f"**{p['UserName']}**    "
                        f"<span
style='color:#777;font-size:0.85rem'>{p['Email']}</span>",
                        unsafe_allow_html=True,
                    )
                    st.caption(f"Branch: {p['Branches'] or '—'}  |  Registered:
{p['Registered']}")
                with pc2:
                    st.markdown(
                        badge(p["RoleName"], "#3498db"),
unsafe_allow_html=True,
                    )
                with pc3:
                    if st.button("✅ Approve", key=f"ap_{p['UserID']}",
                                 use_container_width=True):
                        run_update(
                            "UPDATE User SET IsApproved=1 WHERE UserID=%s",
                            (p["UserID"],),
                        )
                        st.success(f"Approved {p['UserName']}.")
                        st.rerun()
                with pc4:
                    if st.button("🗑️ Reject", key=f"rj_{p['UserID']}",
                                 use_container_width=True):
                        run_update(
                            "UPDATE User SET IsActive=0 WHERE UserID=%s",
                            (p["UserID"],),
                        )
                        st.warning(f"Rejected {p['UserName']}.")
                        st.rerun()
                st.markdown("<hr style='margin:4px 0'>", unsafe_allow_html=True)
```

```
# ──────────────────────────────────────────────────────────
# TAB 3 – ADD USER (admin creates an account directly)
# ──────────────────────────────────────────────────────────
with tab_add:
    st.markdown("#### Create New User Account")
    roles_all    = run_query("SELECT RoleID, RoleName FROM Role WHERE IsActive=1")
    branches_all = run_query(
        "SELECT BranchID, BranchName FROM Branch WHERE IsActive=1 ORDER BY
BranchName"
    )
    role_map2   = {r["RoleName"]: r["RoleID"] for r in roles_all}
    branch_map2 = {b["BranchName"]: b["BranchID"] for b in branches_all}

    # ── Name row (mirrors Register form) ──────────────────
    fn_col, ln_col = st.columns(2)
```

```
    with fn_col:
        nu_first = st.text_input("First Name *", placeholder="Jane",  key="au_first")
    with ln_col:
        nu_last  = st.text_input("Last Name *",  placeholder="Doe",
key="au_last")

    # ── Email ─────────────────────────────────────────────
    nu_email  =  st.text_input("Email  Address  *",  placeholder="jane@example.com",
key="au_email")

    # ── Role + Branch row ─────────────────────────────────
    r1, r2 = st.columns(2)
    with r1:
        nu_role   = st.selectbox("Role *", list(role_map2.keys()), key="au_role")
    with r2:
        nu_branch = st.multiselect("Branch Access *", list(branch_map2.keys()),
key="au_branch")

    # ── Password row ──────────────────────────────────────
    p1, p2 = st.columns(2)
    with p1:
        nu_pass    =  st.text_input("Password  *",                 type="password",
placeholder="Min 8 chars…", key="au_pass")
    with p2:
        nu_pass2 = st.text_input("Confirm Password *", type="password",
                                 placeholder="Repeat password", key="au_pass2")

    nu_approve = st.checkbox("Auto-approve account", value=True, key="au_approve")

    with st.form("admin_add_user", clear_on_submit=False):
        st.caption("Review the details above, then click Create User.")
        add_sub = st.form_submit_button("✚ Create User", use_container_width=True)

    if add_sub:
        nu_name = f"{nu_first.strip()} {nu_last.strip()}".strip()
        from auth import hash_password        if not all([nu_first, nu_last,
nu_email, nu_pass, nu_pass2, nu_branch]):
            st.error("Please fill in all required fields.")
```

```
        elif nu_pass != nu_pass2:
            st.error("❌ Passwords do not match.")
        elif len(nu_pass) < 8:
            st.error("Password must be at least 8 characters.")
        elif run_query("SELECT UserID FROM User WHERE Email=%s",
(nu_email.strip().lower(),)):
            st.error("❌ That email address is already registered.")
        else:
            hashed  = hash_password(nu_pass)
            new_uid = run_update(
                """INSERT INTO User (UserName, Email, Password, IsApproved, IsActive,
RoleID)
                   VALUES (%s, %s, %s, %s, 1, %s)""",
                (nu_name, nu_email.strip().lower(), hashed,
                 1 if nu_approve else 0, role_map2[nu_role]),
            )
            for br in nu_branch:
                run_update(
                    "INSERT IGNORE INTO UserBranchAccess (UserID, BranchID,
AccessType)"
                    " VALUES (%s, %s, 'Write')",
                    (new_uid, branch_map2[br]),
                )
            st.success(f"✅ User **{nu_name}** created successfully!")
            st.rerun()

# ─────────────────────────────────────────────────────────────
# TAB 4 – BRANCH ACCESS
# ─────────────────────────────────────────────────────────────
with tab_branch:
    st.markdown("#### Manage User Branch Access")

    all_u2 = run_query(
        """SELECT u.UserID, u.UserName, u.Email, r.RoleName
           FROM User u JOIN Role r ON r.RoleID=u.RoleID
           WHERE u.IsActive=1 ORDER BY u.UserName"""
    )
    u_map2 = {
        f"{u['UserName']} ({u['Email']})": u["UserID"] for u in all_u2
    }

    sel_u_label = st.selectbox("Select User", list(u_map2.keys()), key="ba_user")
sel_u_id    = u_map2[sel_u_label]

    current_access = run_query(
        """SELECT uba.BranchID, b.BranchName, uba.AccessType
           FROM UserBranchAccess uba
           JOIN Branch b ON b.BranchID=uba.BranchID
           WHERE uba.UserID=%s""",
        (sel_u_id,),
    )

    all_br2 = run_query(
        "SELECT BranchID, BranchName FROM Branch WHERE IsActive=1 ORDER BY
BranchName"
```

```
    )
    br_map2 = {b["BranchName"]: b["BranchID"] for b in all_br2}
    current_br_names = [r["BranchName"] for r in current_access]

    st.markdown(f"**Current access:** {', '.join(current_br_names) or 'None'}")

    with st.form("branch_access_form"):
        new_branches = st.multiselect(
            "Assign Branches", list(br_map2.keys()),
            default=current_br_names,
        )
        access_type = st.selectbox("Access Type", ["Write", "Read", "Admin"])
        ba_sub = st.form_submit_button("💾 Update Branch Access",
use_container_width=True)

    if ba_sub:
        # remove old access and re-insert
        run_update("DELETE FROM UserBranchAccess WHERE UserID=%s", (sel_u_id,))
        for br in new_branches:
            run_update(
                "INSERT IGNORE INTO UserBranchAccess (UserID, BranchID,
AccessType)"
                " VALUES (%s, %s, %s)",
                (sel_u_id, br_map2[br], access_type),
            )
        st.success("✅ Branch access updated.")
        st.rerun()
```

12. Master Data (6_Master_Data.py)

12.1 Design View

The Master Data module lets Admin and Manager manage all reference lookup tables. It has 4 tabs for Admin (Food Items, Waste Categories, Waste Reasons, Branches) and 3 tabs for Manager (Branches tab is not created at all – not just hidden).

Every tab uses the same split-column pattern:

- Left column (3/5 width): st.data_editor with inline-editable rows and a Delete checkbox column.
- Right column (2/5 width): "Add New …" form for creating new records.

Food Items tab editable columns:

- ItemName – free text.
- Unit – SelectboxColumn with preset options: kg, pcs, litre, g, ml, box, pack.
- CostPerUnit – NumberColumn with RM prefix format, min_value=0.0. ● Delete checkbox column.

Waste Categories tab editable columns:

- TypeName and Description – both free text.

Waste Reasons tab editable columns:

- ReasonText and Description – both free text.

Branches tab (Admin only) editable columns:

- BranchName and Location – free text.
- IsActive – CheckboxColumn (tick to keep active, untick to deactivate).
- Delete checkbox column.

12.2 Execution View

On "Save … Changes", the handler iterates every row in the edited DataFrame. If the Delete checkbox is True, it attempts DELETE FROM … and catches Exception for FK constraint violations (e.g., cannot delete an item that has WasteLog records). Otherwise it compares the edited row values against the original df_* at the same index; if any column changed it issues UPDATE … and catches Exception for duplicate name violations. Friendly error messages are shown via st.error() for both cases.

Manager role is handled at the tab creation level: the st.tabs() call receives only 3 strings (no "Branches"). tab_branches = None is set explicitly. The Branches rendering block uses "if tab_branches is not None:", so the tab is never created — it does not even appear as a disabled or hidden tab.

12.3 Field Validation Rules

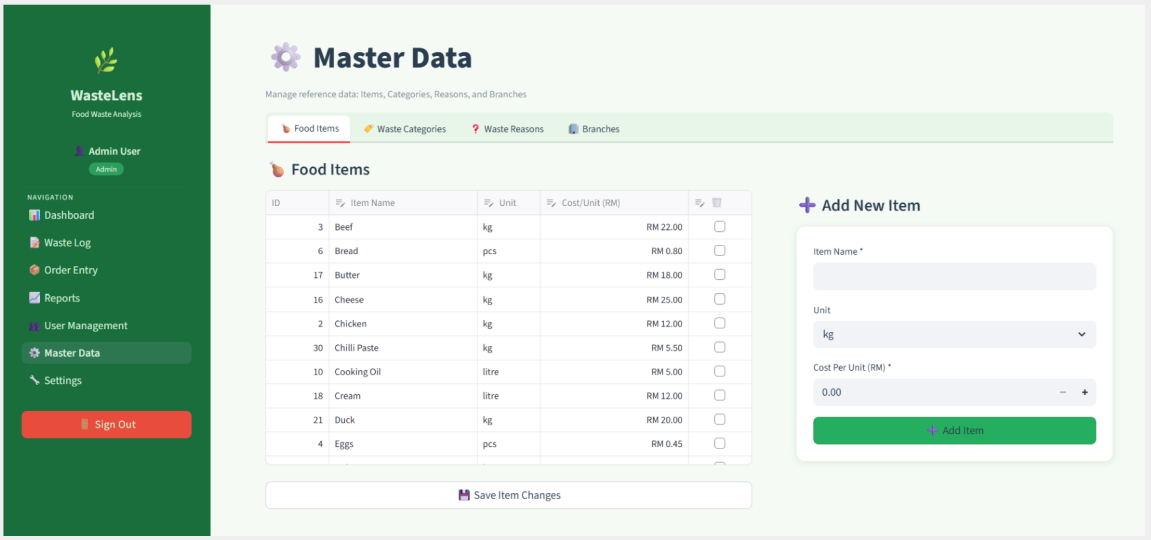
- All "Name" fields: required, non-empty. Error: "… name is required."

- Unique constraint violations: caught as Exception, shown as "… name may already exist." Error.
- FK constraint violations on delete: caught as Exception, shown as "Cannot delete … — it may be in use." Error.
- CostPerUnit: min_value=0.0 enforced by NumberColumn.
- Branch Location: required for new branch creation.

12.4 Screenshots – Master Data

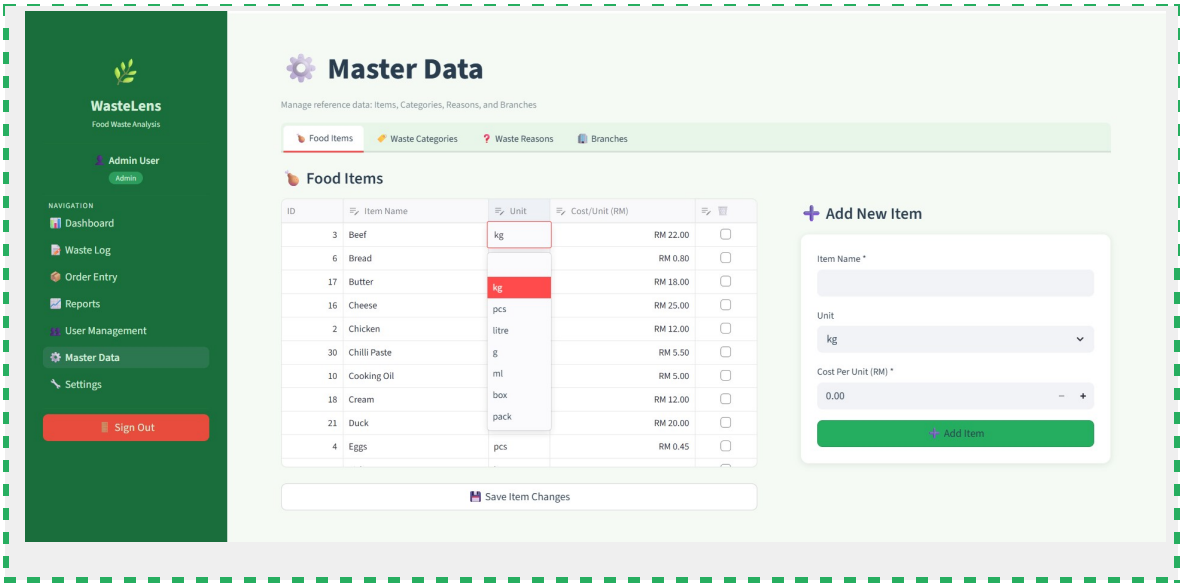
Master Data – Food Items Tab

[SCREENSHOT — Master Data – Food Items tab with inline-editable data_editor (ItemName, Unit dropdown, CostPerUnit) and Add Item form on right]



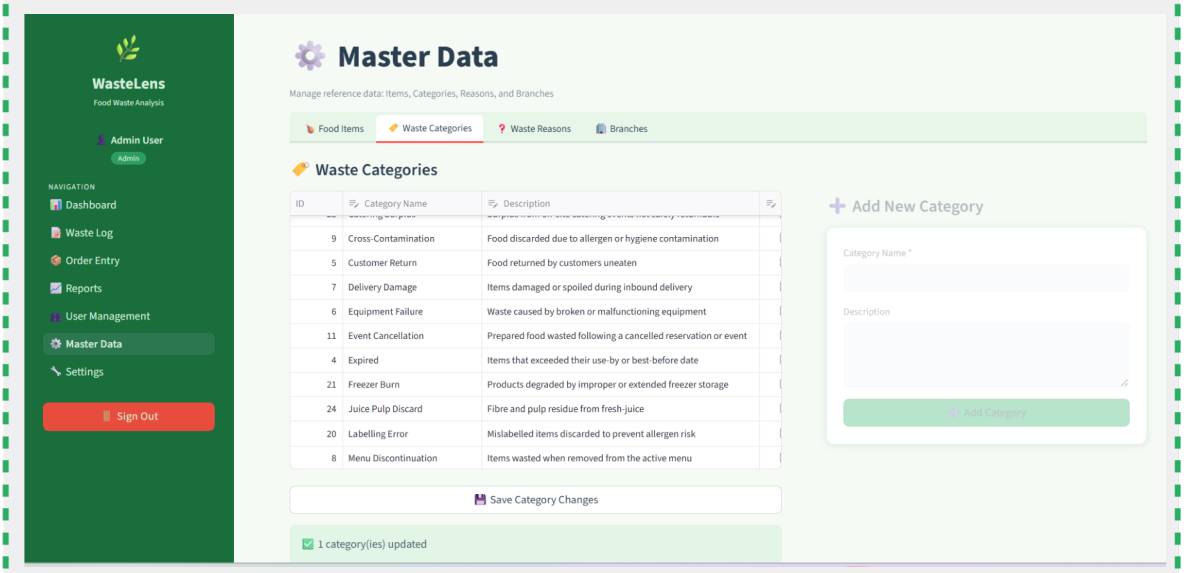
Master Data – Food Item Edited and Saved

[SCREENSHOT — Master Data – success message after saving updated food items]



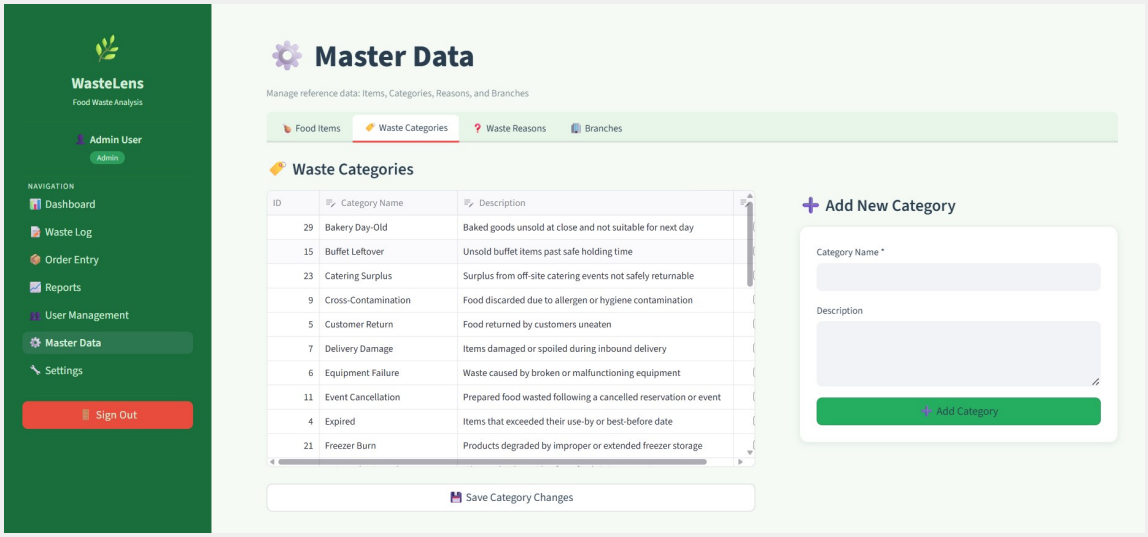
Master Data – FK Delete Error

[SCREENSHOT — Master Data – red error message "Cannot delete X — it may be in use" when deleting a referenced item]



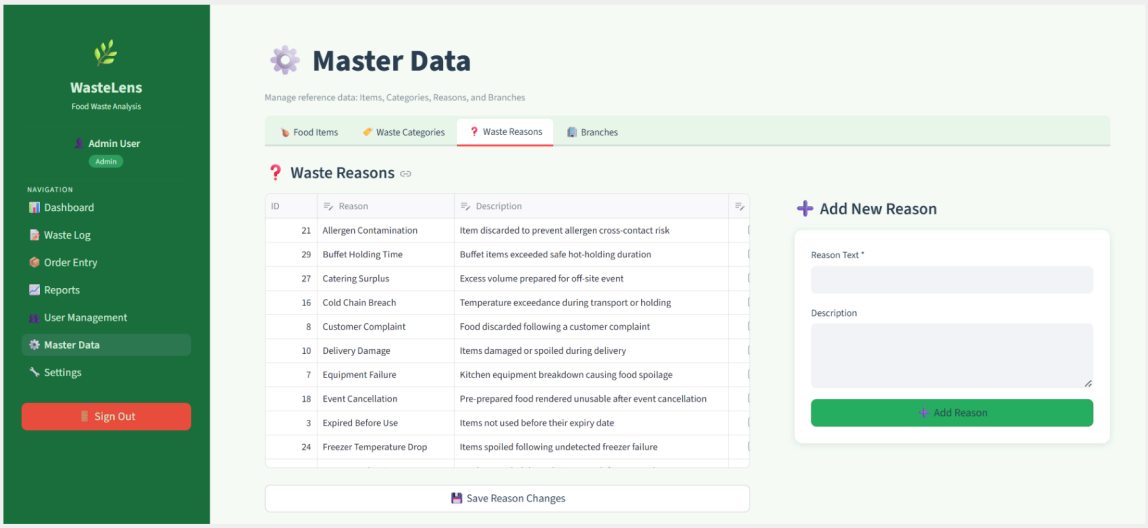
Master Data – Waste Categories Tab

[SCREENSHOT — Master Data – Waste Categories tab with editable TypeName and Description columns, Add Category form]



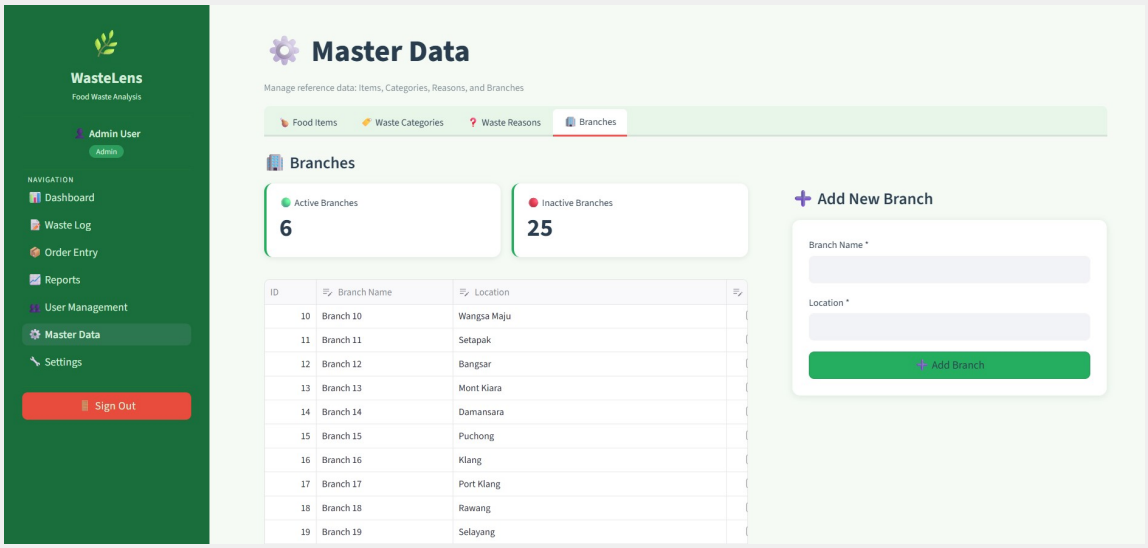
Master Data – Waste Reasons Tab

[SCREENSHOT — Master Data – Waste Reasons tab with editable ReasonText and Description columns, Add Reason form]



Master Data – Branches Tab (Admin Only)

[SCREENSHOT — Master Data – Branches tab (Admin) with BranchName, Location, IsActive checkbox, Delete checkbox, Add Branch form]



12.5 Source Code – 6_Master_Data.py

```
# ============================================================
#  WasteLens – Master Data Management (Admin / Manager)
# ============================================================
import streamlit as st

st.set_page_config(
    page_title="Master Data | WasteLens",
    page_icon="⚙️",
    layout="wide",
)

import pandas as pd
from auth import require_role from
db import run_query, run_update
from utils import inject_css, render_sidebar

inject_css()
user = require_role(["Admin", "Manager"])
render_sidebar(user)
```

```
role = user["RoleName"]

st.title("⚙️ Master Data")
st.caption("Manage reference data: Items, Categories, Reasons" + (", and Branches"
if
role == "Admin" else ""))

if role == "Admin":
    tab_items, tab_cats, tab_reasons, tab_branches = st.tabs(
        ["🍗 Food Items", "🏷️ Waste Categories", "❓ Waste Reasons", "🏬 Branches"]
    )
else:
```

```
    tab_items, tab_cats, tab_reasons = st.tabs(
        ["🍗 Food Items", "🏷️ Waste Categories", "❓ Waste Reasons"]
    )
    tab_branches = None   # not shown for Manager



# ═══════════════════════════════════════════════════════════════
# TAB 1 – FOOD ITEMS
# ═══════════════════════════════════════════════════════════════
with tab_items:
    st.markdown("#### 🍗 Food Items")

    items = run_query(
        "SELECT ItemID, ItemName, Unit, CostPerUnit FROM Item ORDER BY ItemName"
    )

    left_col, right_col = st.columns([3, 2], gap="large")

    with left_col:
        if not items:
            st.info("No items yet. Add one using the form →")
else:
            df_items = pd.DataFrame(items)
            df_items["CostPerUnit"] = df_items["CostPerUnit"].astype(float)
            df_items["🗑️ Delete"] = False

            edited_items = st.data_editor(
                df_items,
                use_container_width=True,
                hide_index=True,
                key="de_items",
                column_config={
                    "ItemID": st.column_config.NumberColumn("ID", disabled=True,
width="small"),
                    "ItemName":       st.column_config.TextColumn("Item       Name",
width="medium"),
                    "Unit": st.column_config.SelectboxColumn(
                        "Unit",
                        options=["kg", "pcs", "litre", "g", "ml", "box", "pack"],
                        width="small",
                    ),
                    "CostPerUnit": st.column_config.NumberColumn(
                        "Cost/Unit   (RM)",   min_value=0.0,   step=0.01,   format="RM
%.2f",
width="medium"
                    ),
                    "🗑️ Delete": st.column_config.CheckboxColumn("🗑️",
width="small"),
                },
                disabled=["ItemID"],
            )

            if st.button("💾 Save Item Changes", key="save_items",
use_container_width=True):
                saved = deleted = errors = 0
```

```
                for i, row in edited_items.iterrows():
                    orig = df_items.iloc[i]
                    if row["🗑 Delete"]:
                        try:
                            run_update("DELETE FROM Item WHERE ItemID=%s",
(int(row["ItemID"]),))
                            deleted += 1
                        except Exception as e:
                            st.error(f"Cannot delete **{row['ItemName']}** — it may
be in use. ({e})")
                            errors += 1
elif (
                        row["ItemName"] != orig["ItemName"]
                        or row["Unit"] != orig["Unit"]
                        or abs(float(row["CostPerUnit"]) -
float(orig["CostPerUnit"])) > 0.0001
                    ):
                        try:
                            run_update(
                                "UPDATE    Item    SET    ItemName=%s,    Unit=%s,
CostPerUnit=%s
WHERE ItemID=%s",
                                (row["ItemName"], row["Unit"],
float(row["CostPerUnit"]), int(row["ItemID"])),
                            )
                            saved += 1
except Exception as e:
                            st.error(f"Cannot  update  **{row['ItemName']}**  —  name
may
already exist. ({e})")
                            errors += 1

                msgs = []
                if saved:   msgs.append(f"✅ {saved} item(s) updated")
                if deleted: msgs.append(f"🗑 {deleted} item(s) deleted")
                if msgs:
                    st.success("  |  ".join(msgs))
                    st.rerun()
                elif not errors:
                    st.info("No changes detected.")
    with
right_col:
        st.markdown("#### ➕ Add New Item")
        with st.form("add_item", clear_on_submit=True):
            i_name = st.text_input("Item Name *")
            i_unit = st.selectbox("Unit", ["kg", "pcs", "litre", "g", "ml", "box",
"pack"])
            i_cpu  = st.number_input("Cost Per Unit (RM) *", min_value=0.0,
                                     step=0.01, format="%.2f")
            i_sub  = st.form_submit_button("➕ Add Item", use_container_width=True)
        if i_sub:
            if not i_name:
                st.error("Item name is required.")
            else:
try:
                    run_update(
```

```
                        "INSERT INTO Item (ItemName, Unit, CostPerUnit) VALUES
(%s,%s,%s)",
                        (i_name.strip(), i_unit, i_cpu),
                    )
                    st.success(f"✅ '{i_name}' added.")
                    st.rerun()
                except Exception as e:
                    st.error(f"Error: {e}")
```

```
# ═══════════════════════════════════════════════════════════════
# TAB 2 – WASTE CATEGORIES
# ═══════════════════════════════════════════════════════════════
with tab_cats:
    st.markdown("#### 🏷️ Waste Categories")

    cats = run_query(
        "SELECT WasteCategoryID, TypeName, Description FROM WasteCategory ORDER BY
TypeName"
    )

    left_col2, right_col2 = st.columns([3, 2], gap="large")

    with left_col2:
        if not cats:
            st.info("No categories yet. Add one using the form →")
        else:
            df_cats = pd.DataFrame(cats)
            df_cats["Description"] = df_cats["Description"].fillna("")
df_cats["🗑 Delete"] = False

            edited_cats = st.data_editor(
                df_cats,
                use_container_width=True,
                hide_index=True,
                key="de_cats",
                column_config={
                    "WasteCategoryID": st.column_config.NumberColumn("ID",
disabled=True, width="small"),
                    "TypeName": st.column_config.TextColumn("Category Name",
width="medium"),
                    "Description": st.column_config.TextColumn("Description",
width="large"),
                    "🗑 Delete": st.column_config.CheckboxColumn("🗑",
width="small"),
                },
                disabled=["WasteCategoryID"],
            )

            if st.button("💾 Save Category Changes", key="save_cats",
use_container_width=True):
                saved = deleted = errors = 0
                for i, row in edited_cats.iterrows():
                    orig = df_cats.iloc[i]
                    if row["🗑 Delete"]:
```

```
                        try:
                            run_update(
                                "DELETE       FROM       WasteCategory       WHERE
WasteCategoryID=%s",
                                (int(row["WasteCategoryID"]),),
                            )
                            deleted += 1
                        except Exception as e:
                            st.error(f"Cannot delete **{row['TypeName']}** — it
may
be in use. ({e})")
                            errors += 1
```

```
                    elif row["TypeName"] != orig["TypeName"] or row["Description"]
!=
orig["Description"]:
                        try:
                            run_update(
                                "UPDATE     WasteCategory     SET     TypeName=%s,
Description=%s
WHERE WasteCategoryID=%s",
                                (row["TypeName"],   row["Description"]   or   None,
int(row["WasteCategoryID"])),
                            )
                            saved += 1
                        except Exception as e:
                            st.error(f"Cannot  update  **{row['TypeName']}**  —  name
may
already exist. ({e})")
                            errors += 1

                msgs = []
                if saved:   msgs.append(f"✅ {saved} category(ies) updated")
                if deleted: msgs.append(f"🗑️ {deleted} category(ies) deleted")
                if msgs:
                    st.success("  |  ".join(msgs))
                    st.rerun()
elif not errors:
                    st.info("No changes detected.")

    with right_col2:
        st.markdown("#### ➕ Add New Category")
        with st.form("add_cat", clear_on_submit=True):
            c_name = st.text_input("Category Name *")
            c_desc = st.text_area("Description")
            c_sub  = st.form_submit_button("➕ Add Category",
use_container_width=True)
        if c_sub:
            if not c_name:
                st.error("Category name is required.")
            else:
try:
                    run_update(
                        "INSERT INTO WasteCategory (TypeName, Description) VALUES
(%s,%s)",
                        (c_name.strip(), c_desc or None),
                    )
                    st.success(f"✅ Category '{c_name}' added.")
```

```
                    st.rerun()
                except Exception as e:
                    st.error(f"Error: {e}")



# ═══════════════════════════════════════════════════════════════
# TAB 3 – WASTE REASONS
# ═══════════════════════════════════════════════════════════════
with tab_reasons:
    st.markdown("#### ❓ Waste Reasons")

    reasons = run_query(
        "SELECT ReasonID, ReasonText, Description FROM WasteReason ORDER BY
ReasonText"
    )
```

```
    left_col3, right_col3 = st.columns([3, 2], gap="large")

    with left_col3:
        if not reasons:
            st.info("No reasons yet. Add one using the form →")
        else:
            df_reasons = pd.DataFrame(reasons)
            df_reasons["Description"] = df_reasons["Description"].fillna("")
            df_reasons["🗑 Delete"] = False

            edited_reasons = st.data_editor(
                df_reasons,
                use_container_width=True,
                hide_index=True,
                key="de_reasons",
                column_config={
                    "ReasonID": st.column_config.NumberColumn("ID", disabled=True,
width="small"),
                    "ReasonText": st.column_config.TextColumn("Reason",
width="medium"),
                    "Description":       st.column_config.TextColumn("Description",
width="large"),
                    "🗑 Delete": st.column_config.CheckboxColumn("🗑",
width="small"),
                },
                disabled=["ReasonID"],
            )

            if st.button("💾 Save Reason Changes", key="save_reasons",
use_container_width=True):
                saved = deleted = errors = 0
                for i, row in edited_reasons.iterrows():
                    orig = df_reasons.iloc[i]
                    if row["🗑 Delete"]:
                        try:
                            run_update(
                                "DELETE FROM WasteReason WHERE ReasonID=%s",
                                (int(row["ReasonID"]),),
                            )
```

```
                            deleted += 1
                        except Exception as e:
                            st.error(f"Cannot  delete  **{row['ReasonText']}**  —  it
may
be in use. ({e})")
                            errors += 1
                    elif row["ReasonText"] != orig["ReasonText"] or
row["Description"] != orig["Description"]:
                        try:
                            run_update(
                                "UPDATE     WasteReason     SET     ReasonText=%s,
Description=%s
WHERE ReasonID=%s",
                                (row["ReasonText"], row["Description"] or None,
int(row["ReasonID"])),
                            )
                            saved += 1
                        except Exception as e:
                            st.error(f"Cannot update **{row['ReasonText']}** — name
may already exist. ({e})")
                            errors += 1
```

```
                msgs = []
                if saved:   msgs.append(f"✅ {saved} reason(s) updated")
                if deleted: msgs.append(f"🗑️ {deleted} reason(s) deleted")
                if msgs:
                    st.success("  |  ".join(msgs))
                    st.rerun()
elif not errors:
                    st.info("No changes detected.")

    with right_col3:
        st.markdown("#### ➕ Add New Reason")
        with st.form("add_reason", clear_on_submit=True):
            r_text = st.text_input("Reason Text *")
            r_desc = st.text_area("Description")
            r_sub           =      st.form_submit_button("➕      Add      Reason",
use_container_width=True)
        if r_sub:
            if not r_text:
                st.error("Reason text is required.")
            else:
                try:
                    run_update(
                        "INSERT INTO WasteReason (ReasonText, Description) VALUES
(%s,%s)",
                        (r_text.strip(), r_desc or None),
                    )
                    st.success(f"✅ Reason '{r_text}' added.")
                    st.rerun()
                except Exception as e:
                    st.error(f"Error: {e}")


# ═══════════════════════════════════════════════════════════
# TAB 4 - BRANCHES  (Admin only - tab not rendered for Manager)
```

```
# ═══════════════════════════════════════════════════════════════
if tab_branches is not None:
    with tab_branches:
        st.markdown("#### 🏢 Branches")

        branches = run_query(
            "SELECT BranchID, BranchName, Location, IsActive FROM Branch ORDER BY
BranchName"
        )

        left_col4, right_col4 = st.columns([3, 2], gap="large")

        with left_col4:
            if not branches:
                st.info("No branches found. Add one using the form →")
            else:
                active_cnt   = sum(1 for b in branches if b["IsActive"])
                inactive_cnt = len(branches) - active_cnt
                m1, m2 = st.columns(2)
                m1.metric("🟢 Active Branches",   active_cnt)
                m2.metric("🔴 Inactive Branches", inactive_cnt)
                st.markdown("")

                df_br = pd.DataFrame(branches)
```

```
                df_br["IsActive"]    = df_br["IsActive"].astype(bool)
                df_br["🗑 Delete"] = False

                edited_br = st.data_editor(
                    df_br,
                    use_container_width=True,
                    hide_index=True,
key="de_branches",
                    column_config={
                        "BranchID":   st.column_config.NumberColumn("ID",
disabled=True, width="small"),
                        "BranchName": st.column_config.TextColumn("Branch Name",
width="medium"),
                        "Location":   st.column_config.TextColumn("Location",
width="large"),
                        "IsActive":   st.column_config.CheckboxColumn("Active?",
width="small"),
                        "🗑 Delete": st.column_config.CheckboxColumn("🗑",
width="small"),
                    },
                    disabled=["BranchID"],
                )

                if st.button("💾 Save Branch Changes", key="save_branches",
use_container_width=True):
                    saved = deleted = errors = 0
                    for i, row in edited_br.iterrows():
                        orig = df_br.iloc[i]
                        if row["🗑 Delete"]:
                            try:
                                run_update(
```

```
                                    "DELETE FROM Branch WHERE BranchID=%s",
                                    (int(row["BranchID"]),),
                                )
                                deleted += 1
except Exception as e:
                                st.error(f"Cannot  delete  **{row['BranchName']}**  —
it
may be in use. ({e})")
                                errors += 1
                        elif (
                            row["BranchName"] != orig["BranchName"]
                            or row["Location"] != orig["Location"]
                            or bool(row["IsActive"]) != bool(orig["IsActive"])
                        ):
                            try:
                                run_update(
                                    "UPDATE Branch SET BranchName=%s, Location=%s,
IsActive=%s WHERE BranchID=%s",
                                    (row["BranchName"], row["Location"],
                                     1 if row["IsActive"] else 0,
int(row["BranchID"])),
                                )
                                saved += 1
                            except Exception as e:
                                st.error(f"Cannot update **{row['BranchName']}** —
name may already exist. ({e})")
                                errors += 1

                    msgs = []
                    if saved:   msgs.append(f"✅ {saved} branch(es) updated")
                    if deleted: msgs.append(f"🗑️ {deleted} branch(es) deleted")
                    if msgs:
                        st.success("  |  ".join(msgs))
                        st.rerun()
                    elif not errors:
                        st.info("No changes detected.")

        with right_col4:
            st.markdown("#### ➕ Add New Branch")
            with st.form("add_branch", clear_on_submit=True):
                br_name = st.text_input("Branch Name *")
                br_loc  = st.text_input("Location *")
                br_sub  = st.form_submit_button("➕ Add Branch",
use_container_width=True)
            if br_sub:
                if not br_name or not br_loc:
                    st.error("Branch name and location are required.")
                else:
                    try:
                        run_update(
                            "INSERT INTO Branch (BranchName, Location, IsActive)
VALUES (%s,%s,1)",
                            (br_name.strip(), br_loc.strip()),
                        )
                        st.success(f"✅ Branch '{br_name}' added.")
                        st.rerun()
                    except Exception as e:
```

```
                    st.error(f"Error: {e}")
```

13. Settings (7_Settings.py)

13.1 Design View

The Settings page is accessible to Admin only. It provides a Gmail SMTP configuration interface so each team member can set up their own Gmail App Password for OTP password-reset emails, without modifying the shared config.py file.

Page structure:

- A green instruction banner explaining the 2-step process to get a Gmail App Password (links to Google account pages).
- Two input fields side by side: Gmail Address and Gmail App Password (masked, type="password").
- Sender Display Name field (single line).
- "Save Settings" button: validates input and writes to local_settings.json.
- "Send Test Email" button: connects to Gmail SMTP and sends a test email to the configured sender address.
- Status indicator at the bottom: green card with sender email if configured; amber warning card if not configured.

13.2 Execution View

Settings are read from and written to local_settings.json located one level above the pages/ directory. The load_settings() function returns {} if the file does not exist. The save_settings() function writes the dict as formatted JSON.

The "Send Test Email" button opens an smtplib.SMTP connection directly with the stored credentials and sends an HTML email to the sender's own address. SMTPAuthenticationError is caught specifically to display a clear "check your App Password" message. Any other exception is displayed with its message string.

The email_utils.py module reads local_settings.json first (if it exists and has valid credentials), so any email sent by the app (OTP resets) uses the locally saved Gmail account — not the config.py defaults.

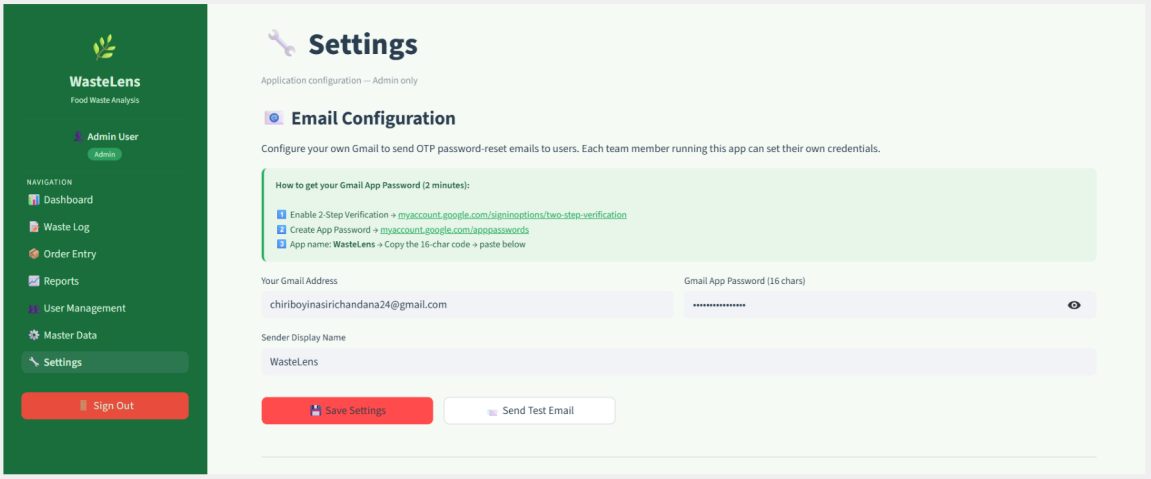
13.3 Field Validation Rules

- Gmail Address: must contain "@" and must not be empty. Error: "Please enter a valid Gmail address."
- App Password: after removing spaces, must be at least 16 characters. Error: "App Password must be 16 characters (remove any spaces)."
- Sender Name: falls back silently to "WasteLens" if left blank. No error.

13.4 Screenshots – Settings

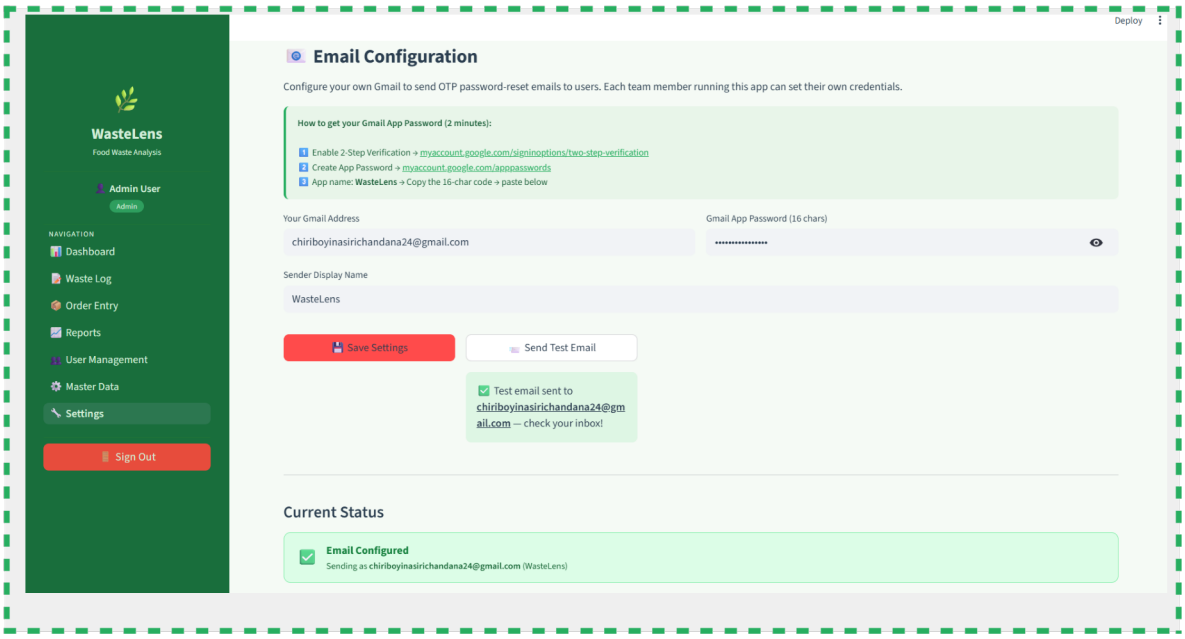
Settings – Email Configuration Form (Not Configured)

[SCREENSHOT — Settings page – Gmail instruction banner, empty email/password/name fields, Save and Test buttons, amber "Email Not Configured" status at bottom]



Settings – Test Email Sent Success

[SCREENSHOT — Settings page – green success message "Test email sent to <email> — check your inbox!" after clicking Send Test Email]



13.5 Source Code – 7_Settings.py

```
# ============================================================
#  WasteLens – Settings  (Admin only)
# ============================================================
import streamlit as st

st.set_page_config(
page_title="Settings | WasteLens",
    page_icon="🔧",
    layout="wide",
)

import json, os, smtplib, random
from email.mime.multipart import MIMEMultipart
from email.mime.text      import MIMEText
from auth   import require_role
from utils  import inject_css, render_sidebar

inject_css()
user = require_role(["Admin"])
render_sidebar(user)

SETTINGS_FILE = os.path.join(os.path.dirname(__file__), "..",
"local_settings.json")

# ── Load / Save helpers ──────────────────────────────────────
def load_settings() -> dict:
    if os.path.exists(SETTINGS_FILE):
        try:
            with open(SETTINGS_FILE, "r") as f:
                return json.load(f)
        except Exception:
```

```
            pass
    return {}


def save_settings(data: dict):
    with open(SETTINGS_FILE, "w") as f:
        json.dump(data, f, indent=2)

cfg = load_settings()

st.title("🔧 Settings")
st.caption("Application configuration — Admin only")

# ═══════════════════════════════════════════════════════════
#  EMAIL / GMAIL CONFIGURATION
# ═══════════════════════════════════════════════════════════
st.markdown("### 📧 Email Configuration")
st.markdown(
    "Configure your own Gmail to send OTP password-reset emails to users. "
    "Each team member running this app can set their own credentials."
)

with st.container():
    st.markdown("""
    <div style='background:#e8f5e9;border-left:4px solid #27AE60;border-
radius:8px;                padding:.9rem
1.1rem;font-size:.85rem;color:#1a5e38;margin-bottom:1rem'>
      <b>How to get your Gmail App Password (2 minutes):</b><br><br>
      1️⃣ Enable 2-Step Verification →
         <a href='https://myaccount.google.com/signinoptions/two-step-
verification'
            target='_blank' style='color:#27AE60'>
            myaccount.google.com/signinoptions/two-step-verification</a><br>
      2️⃣ Create App Password →
         <a href='https://myaccount.google.com/apppasswords'
            target='_blank' style='color:#27AE60'>
            myaccount.google.com/apppasswords</a><br>
      3️⃣ App name: <b>WasteLens</b> → Copy the 16-char code → paste below
    </div>
    """, unsafe_allow_html=True)

    col1, col2 = st.columns(2)

    with col1:
        gmail_addr = st.text_input(
            "Your Gmail Address",
            value=cfg.get("sender_email", ""),
            placeholder="yourname@gmail.com",
        )
    with col2:
        app_pwd = st.text_input(
            "Gmail App Password (16 chars)",
            value=cfg.get("app_password", ""),
placeholder="abcdefghijklmnop",
            type="password",
        )
```

```
    sender_name = st.text_input(
        "Sender Display Name",
        value=cfg.get("sender_name", "WasteLens"),
        placeholder="WasteLens",
    )
```

```
    st.markdown("<div style='height:4px'></div>", unsafe_allow_html=True)

    save_col, test_col, _ = st.columns([1.5, 1.5, 4])

    with save_col:
        if st.button("💾 Save Settings", use_container_width=True,
type="primary"):
            if not gmail_addr or "@" not in gmail_addr:
                st.error("Please enter a valid Gmail address.")
            elif not app_pwd or len(app_pwd.replace(" ", "")) < 16:
                st.error("App Password must be 16 characters (remove any
spaces).")
            else:
                cfg["sender_email"] = gmail_addr.strip()
                cfg["app_password"] = app_pwd.strip().replace(" ", "")
                cfg["sender_name"]  = sender_name.strip() or "WasteLens"
                cfg["smtp_host"]    = "smtp.gmail.com"
                cfg["smtp_port"]    = 587
                save_settings(cfg)
                st.success("✅ Email settings saved!")
                st.rerun()

    with test_col:
        if st.button("📨 Send Test Email", use_container_width=True):
            if not cfg.get("app_password") or not cfg.get("sender_email"):
                st.error("Save your settings first before testing.")
            else:
                test_otp = str(random.randint(100000, 999999))
                try:
                    msg = MIMEMultipart("alternative")
                    msg["Subject"] = "WasteLens – Test Email ✅"
                    msg["From"]    = f"{cfg['sender_name']}
<{cfg['sender_email']}>"
                    msg["To"]      = cfg["sender_email"]

                    body = (
                        f"<div style='font-family:sans-serif;padding:20px'>"
                        f"<h2 style='color:#27AE60'>🌿 WasteLens Test Email</h2>"
f"<p>Your email configuration is working correctly!</p>"
                        f"<p>Sample OTP: <b
style='font-size:1.4rem;letter-spacing:5px;"
                        f"color:#1a5e38'>{test_otp}</b></p>"
                        f"<p style='color:#888;font-size:.8rem'>Sent from
WasteLens
Settings</p>"
                        f"</div>"
                    )
                    msg.attach(MIMEText("WasteLens test email — config is working!",
```

```
"plain"))
                    msg.attach(MIMEText(body, "html"))

                    with smtplib.SMTP("smtp.gmail.com", 587) as server:
                        server.ehlo()
                        server.starttls()
                        server.login(cfg["sender_email"], cfg["app_password"])
                        server.sendmail(cfg["sender_email"], cfg["sender_email"],
msg.as_string())

                    st.success(f"✅ Test email sent to **{cfg['sender_email']}** —
check your inbox!")
                except smtplib.SMTPAuthenticationError:
                    st.error("❌ Authentication failed — check your App
Password.")
                except Exception as e:
                    st.error(f"❌ Failed to send: {e}")

# ── Current status indicator ──────────────────────────────
st.markdown("---")
st.markdown("#### Current Status")
 if cfg.get("app_password") and
cfg.get("sender_email"):
    st.markdown(f"""
    <div style='background:#dcfce7;border:1px solid #86efac;border-radius:10px;
                padding:.8rem 1.2rem;display:flex;align-items:center;gap:.8rem'>
      <span style='font-size:1.4rem'>✅</span>
      <div>
        <div style='font-weight:700;color:#15803d'>Email Configured</div>
        <div style='font-size:.83rem;color:#166534'>
          Sending as <b>{cfg['sender_email']}</b>
({cfg.get('sender_name','WasteLens')})
        </div>
      </div>
    </div>
    """, unsafe_allow_html=True)
else:
    st.markdown("""
    <div style='background:#fef9c3;border:1px solid #fde047;border-radius:10px;
                padding:.8rem 1.2rem;display:flex;align-items:center;gap:.8rem'>
      <span style='font-size:1.4rem'>⚠️</span>
      <div>
        <div style='font-weight:700;color:#854d0e'>Email Not Configured</div>
        <div style='font-size:.83rem;color:#92400e'>
          OTP emails are in Demo Mode — enter your Gmail above to enable real
sending.
        </div>
      </div>
    </div>
    """, unsafe_allow_html=True)
```

14. Database Schema & Design

14.1 Entity Overview

The WasteLens database ("wastelens") contains 9 tables. The central fact table is WasteLog, surrounded by dimension tables for users, branches, items, categories, and reasons. The OrderEntry table is a secondary fact table for daily order counts. A supporting view vw_DailyBranchSummary pre-aggregates daily totals for the waste-per-order KPI chart.

14.2 Table Descriptions

Role

- Primary key: RoleID (INT, AUTO_INCREMENT)
- Columns: RoleName (VARCHAR 50, UNIQUE), IsActive (TINYINT, default 1) ● Purpose: lookup table for the three system roles – Admin, Manager, Staff.

User

- Primary key: UserID (INT, AUTO_INCREMENT)
- Columns: UserName, Email (UNIQUE), Password (bcrypt hash), IsApproved (TINYINT), IsActive (TINYINT), CreatedAt (TIMESTAMP default CURRENT_TIMESTAMP), RoleID (FK → Role)
- Purpose: all application users with bcrypt-hashed passwords and approval/active flags.

UserBranchAccess

- Composite primary key: (UserID, BranchID)
- Columns: AccessType (VARCHAR 20 – Write / Read / Admin)
- Purpose: junction table controlling which branches each user can access. Many-to-many between User and Branch.

Branch

- Primary key: BranchID (INT, AUTO_INCREMENT)
- Columns: BranchName (VARCHAR 100, UNIQUE), Location (VARCHAR 200), IsActive (TINYINT, default 1)
- Purpose: physical restaurant or food-service branches.

Item

- Primary key: ItemID (INT, AUTO_INCREMENT)
- Columns: ItemName (VARCHAR 100, UNIQUE), Unit (VARCHAR 20), CostPerUnit (DECIMAL 10,2)
- Purpose: food items with unit of measurement and cost for automatic waste cost estimation.

WasteCategory

- Primary key: WasteCategoryID (INT, AUTO_INCREMENT)
- Columns: TypeName (VARCHAR 100, UNIQUE), Description (TEXT, nullable) ● Purpose: types of waste – e.g., Overproduction, Spoilage, Preparation Waste.

WasteReason

- Primary key: ReasonID (INT, AUTO_INCREMENT)
- Columns: ReasonText (VARCHAR 200, UNIQUE), Description (TEXT, nullable) ● Purpose: specific reasons for waste – e.g., Expired, Over-ordered, Customer Return.

WasteLog

- Primary key: WasteLogID (INT, AUTO_INCREMENT)
- Columns: WasteDateTime (DATETIME), Quantity (DECIMAL 10,3), EstimatedCost (DECIMAL 10,2), Notes (TEXT, nullable), LoggedByUserID (FK → User), BranchID (FK → Branch), WasteCategoryID (FK → WasteCategory), ReasonID (FK → WasteReason), ItemID (FK → Item)
- Purpose: central fact table, one row per waste event.

OrderEntry

- Primary key: OrderID (INT, AUTO_INCREMENT)
- Columns: OrderDate (DATE), OrderCount (INT), Notes (TEXT, nullable), EnteredByUserID (FK → User), BranchID (FK → Branch)
- Unique constraint: (BranchID, OrderDate) – one record per branch per day.
- Purpose: daily customer order counts per branch, used for waste-per-order ratio KPI.

14.3 Key Foreign-Key Relationships

- User.RoleID → Role.RoleID
- UserBranchAccess.UserID → User.UserID (ON DELETE CASCADE)
- UserBranchAccess.BranchID → Branch.BranchID
- WasteLog.LoggedByUserID → User.UserID
- WasteLog.BranchID → Branch.BranchID
- WasteLog.ItemID → Item.ItemID
- WasteLog.WasteCategoryID → WasteCategory.WasteCategoryID
- WasteLog.ReasonID → WasteReason.ReasonID
- OrderEntry.EnteredByUserID → User.UserID
- OrderEntry.BranchID → Branch.BranchID

15. Role-Based Access Control (RBAC)

15.1 Role Overview

WasteLens enforces three roles. Access is checked on every page load by require_auth() and require_role(). The sidebar navigation also filters the visible page links by role, so users only see links to pages they are permitted to access.

Admin role:

- Full access to all 7 pages and all features.
- Sees all branches and all users.
- Can manage users, branches, settings, and all master data.

Manager role:

- Access to Dashboard, Waste Log, Order Entry, Reports, and Master Data (Items/Categories/Reasons only – no Branches tab).
- Data scoped to assigned branches only.
- Cannot access User Management or Settings.

Staff role:

- Access to Dashboard, Waste Log, and Order Entry only.
- Data scoped to assigned branches only.
- Can only edit or delete their own waste log entries.
- Cannot access Reports, User Management, Master Data, or Settings.

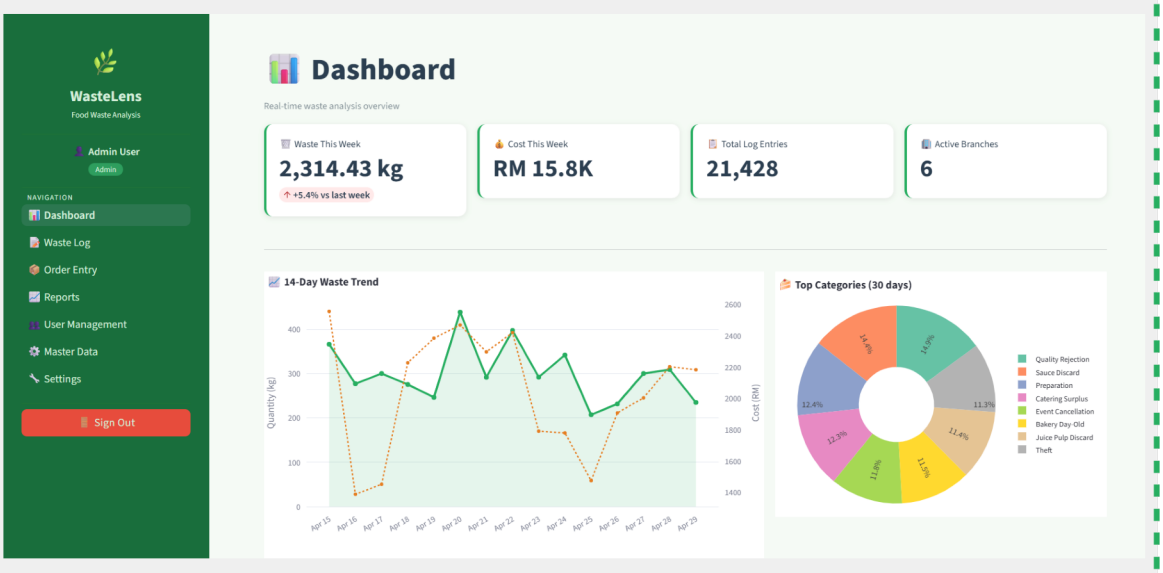
15.2 Sidebar Navigation by Role

Sidebar – Admin View (All 7 links)

[SCREENSHOT — Sidebar showing all 7 navigation links: Dashboard, Waste Log, Order Entry, Reports, User Management, Master Data, Settings – Admin logged in]

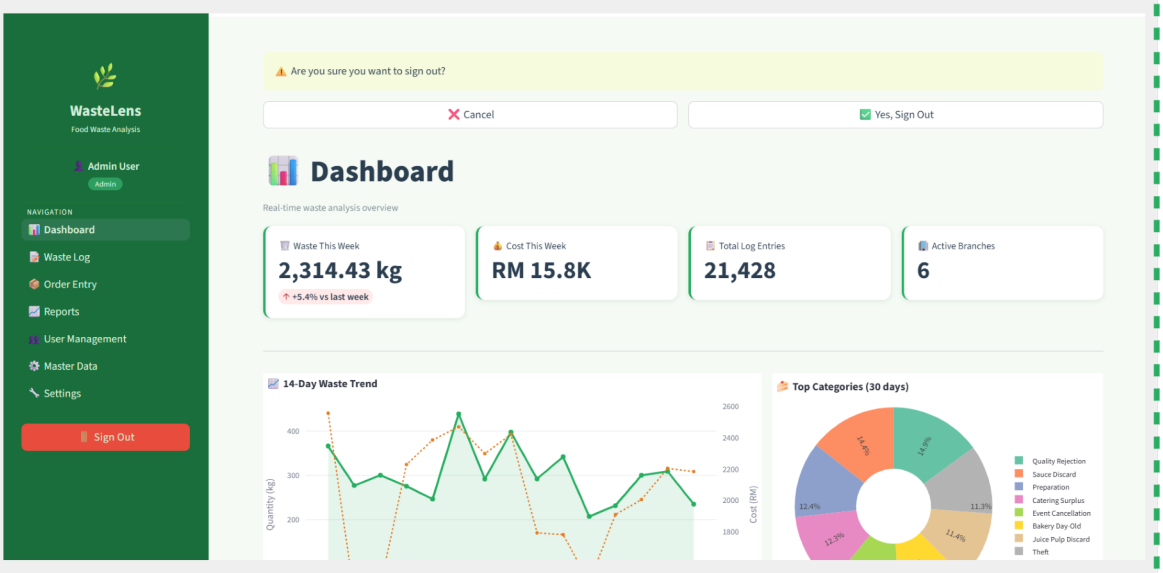
Sidebar – Manager View (5 links)

[SCREENSHOT — Sidebar showing 5 navigation links for Manager: Dashboard, Waste Log, Order Entry, Reports, Master Data – no User Management, no Settings]



Sign Out Confirmation

[SCREENSHOT — Sign Out confirmation banner in main area with "Yes, Sign Out" and "Cancel" buttons]



16. Installation & Setup Guide

16.1 Prerequisites

- Python 3.11 or higher installed and on the system PATH.
- MySQL 8.0 server running locally on port 3306.
- MySQL database "wastelens" created (run setup_db.py).
- A Gmail account with 2-Step Verification enabled and an App Password created (for OTP emails).

16.2 Step 1 – Install Python Dependencies

```
pip install streamlit mysql-connector-python bcrypt plotly pandas python-docx
```

16.3 Step 2 – Configure Database Connection

Edit the file food_/config.py and set your MySQL password:

```
DB_CONFIG = {
    "host":     "localhost",
    "port":     3306,
    "database": "wastelens",
    "user":     "root",
    "password": "YourMySQLPassword",
}
```

16.4 Step 3 – Initialise the Database

```
cd "C:/Users/dhana/Desktop/food management/food_"

python setup_db.py              # creates all 9 tables and the view
python seed_data.py             # inserts default roles, branches, items,
categories
python inject_sample_data.py    # injects realistic waste log data for chart demos
python add_team_members.py      # creates/updates the 3 team member accounts
```

16.5 Step 4 – Run the Application

```
# Option A – if streamlit is on your PATH
streamlit run app.py

# Option B – if streamlit is not on PATH
python -m streamlit run app.py
```

The application opens automatically at http://localhost:8501 in your default browser.

16.6 Step 5 – Configure Email (Optional but Recommended)

9. Log in as Admin (admin@wastelens.com / Admin@123).
10. Navigate to Settings (gear icon in the sidebar).
11. Enter your Gmail address and 16-character App Password.
12. Click "Save Settings".
13. Click "Send Test Email" to verify the connection.

After this, OTP password-reset emails will be sent from your Gmail. Teammates can do the same on their own machines.

17. Demo Credentials

The following accounts are seeded by default and can be used to explore all roles:

Admin Accounts:

- Email: admin@wastelens.com Password: Admin@123

Manager Account:

- Email: manager@wastelens.com Password: Manager@123
- Email: kannu1d@cmich.edu Password: 123456

Staff Account:

- Email: staff@wastelens.com Password: Staff@123
- Email: chiri1s@cmich.edu Password: Chiri@123

Note: On first run, init_passwords() in db.py automatically converts the SQL placeholder hashes to real bcrypt hashes and creates team-member accounts if they do not yet exist. No manual SQL needed.

18. Conclusion

WasteLens delivers a full-featured, production-quality food waste management system within a Python Streamlit and MySQL stack. The application addresses all core requirements of the BIS 698 capstone project: role-based multi-user access, live data visualisation, comprehensive CRUD operations, secure authentication with OTP-based password reset, and a clean, responsive UI with a consistent green brand theme across all pages.

Engineering highlights:

- All SQL uses parameterised queries (%s / %(key)s). No user input is ever concatenated into SQL strings.
- bcrypt with random salt is used for all passwords. Plaintext is never stored or logged.
- OTP codes are time-limited to 10 minutes and stored only in server-side session state, not in the database.
- branch_filter_sql() provides a single, DRY data-isolation mechanism reused across Dashboard, Waste Log, Reports, and Order Entry.
- st.data_editor enables professional inline table editing for both Waste Log and Master Data without any custom JavaScript.
- Gmail App Password is stored per-machine in local_settings.json, so each teammate configures their own without touching shared config.py.
- FK-constraint errors on Master Data delete are caught gracefully and shown as friendly messages instead of Python tracebacks.
- Global CSS is defined once in utils.py and injected with inject_css() on every page, ensuring a consistent look throughout.

The system is ready for live demonstration and is extensible. Potential future enhancements include automated daily summary emails, predictive waste forecasting using historical trend data, a mobile-responsive layout, and integration with inventory or POS systems for fully automated waste detection.

— End of Documentation —